

Chapter- 1

Computer Graphics



Computer Graphics:

Types of computer graphics,-Graphic Displays, Random scan displays, Raster scan displays, Frame buffer and video controller, Line and Circle generating algorithms, Transformations, Windowing and Clipping, Three Dimensional graphics, Curves and Surfaces, Hidden Lines and Surfaces.

Introduction

- **Computer graphics** is the discipline of generating images with the aid of computers.
- The phrase was coined in 1960 by computer graphics researchers William Fetter of Boeing.
- **William Fetter**, also known as William Alan Fetter or Bill Fetter (March 14, 1928 – June 23, 2002), was an American graphic designer and pioneer in the field of computer graphics.
- He explored the perspective fundamentals of computer animation of a human figure from 1960 on and was the first to create a human figure as a 3D model.
- The *First Man* was a pilot in a short 1964 computer animation, also known as *Boeing Man* and now as *Boeman* by the Boeing company.



- The term computer graphics has been used in a broad sense to describe "almost everything on computers that is not text or sound". The term computer graphics refers to several different things:

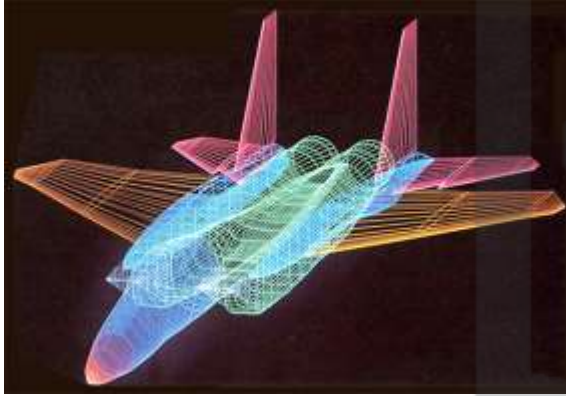
- The representation and manipulation of image data by a computer.

- The various technologies used to create and manipulate images.



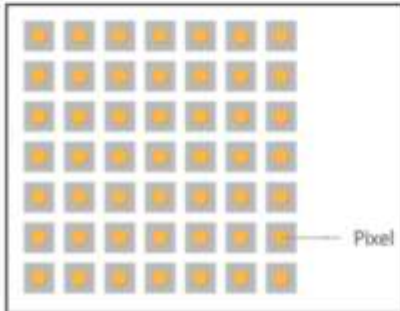
Application

Today, computer graphics is a core technology in -- digital photography, animation, movies, advertising, video games, cell phone and computer displays, and many specialized applications.

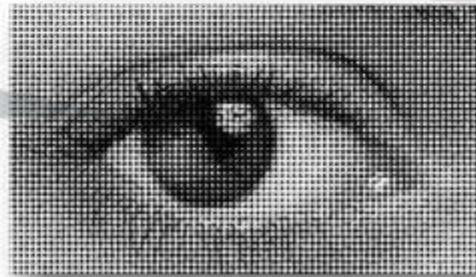


Types of Computer Graphics

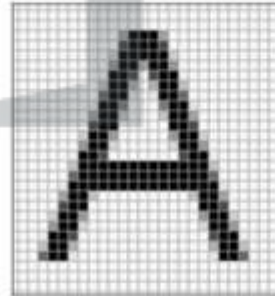
- **Raster Graphics:** In raster, graphics pixels are used for an image to be drawn. It is also known as a bitmap image in which a sequence of images is into smaller pixels. Basically, a bitmap indicates a large number of pixels together.
- Raster is good for photographs. Raster graphics are great when creating rich and detailed images. Every pixel in a raster image can be a different color creating a complex image with all kinds of color and variations.



Computer screen with pixels is a raster

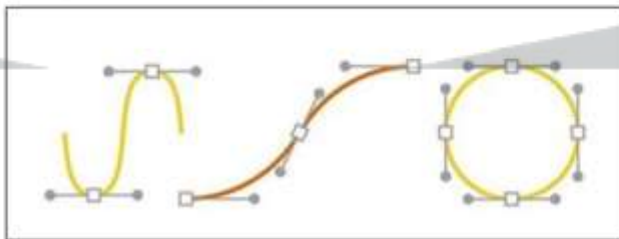


Computer screen with pixels is a raster



Computer screen with pixels is a raster

Vector Graphics: The vector graphics consist of anchored dots and connected by lines and curves, similar to **the Also, vector graphics are made up of paths, each with a mathematical formula (vector) that tells the path how it is shaped.** Because these graphics are not based on pixels, they are known as resolution independent, which makes them infinitely scalable. Their lines are sharp, without any loss in quality or detail, no matter what their size. These graphics are also device independent, which means their quality doesn't depend on the number of dots available on a printer or the number of pixels on a screen. Because they consist of lines and anchor points, the size of the files are relatively small. This makes vector files the best format for graphic assets such as illustrations, icons and company logos, as the same file can be used for designs ranging from a mobile app to a large billboard without sacrificing quality or increasing file size.

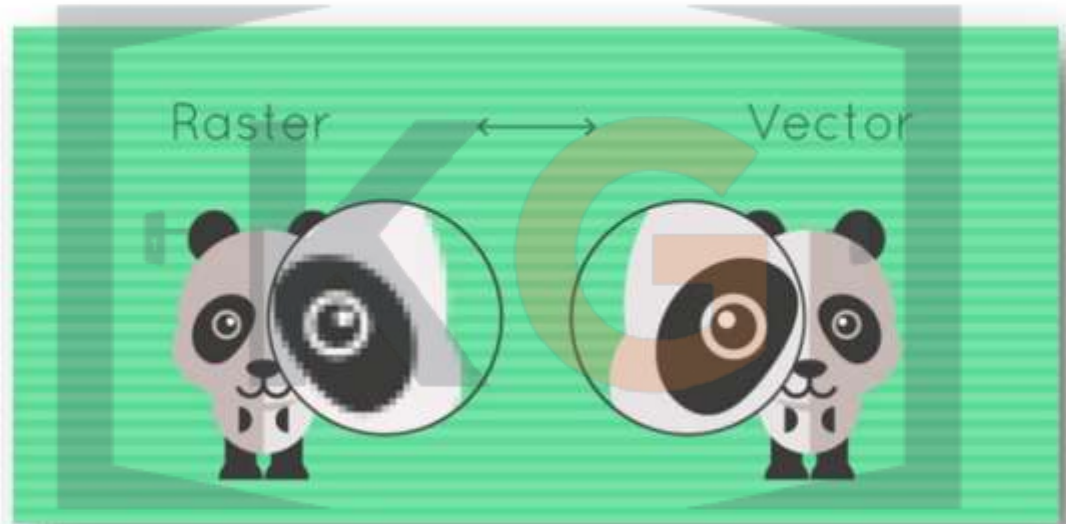


Vector shapes are made up of points and lines that create paths.



Vector image

- When the raster image is zoomed in or enlarged pixels appear like little squares on graph paper. Note the figure below. We will deal with Raster Graphics.



Raster image Vs Vector image

- It is known that Raster graphics is more popular than Vector graphics, but if your project requires scalable shapes and solid colors, vector is the best choice, but if your project requires complex color blends, raster is the preferred format.



Types of Computer Graphics

In addition to the two primary classifications, computer graphics can be categorized based on interactivity. Interactive computer graphics involve real-time user engagement and manipulation, while non-interactive computer graphics encompass pre-generated, static visual content.

Interactive Computer Graphics: Interactive computer graphics involve real-time user communication and visual content. This technique enables users to interact with virtual environments by manipulating objects or navigating virtual worlds. Interactive computer graphics rely on input devices like keyboards, mouse, touchscreens, or motion sensors to capture user actions. Interaction techniques play a crucial role in enhancing user experiences. From touch gestures to voice commands, these techniques enable users to perform actions and navigate through virtual environments effortlessly. Application logic ensures that user interactions are processed correctly and produce the desired results.

Types of Computer Graphics

Non-Interactive Computer Graphics: While interactive computer graphics involve real-time manipulation of visual content, non-interactive computer graphics utilise pre-generated visual content without real-time manipulation. This technique is commonly used in the entertainment, advertising, and architecture industries.

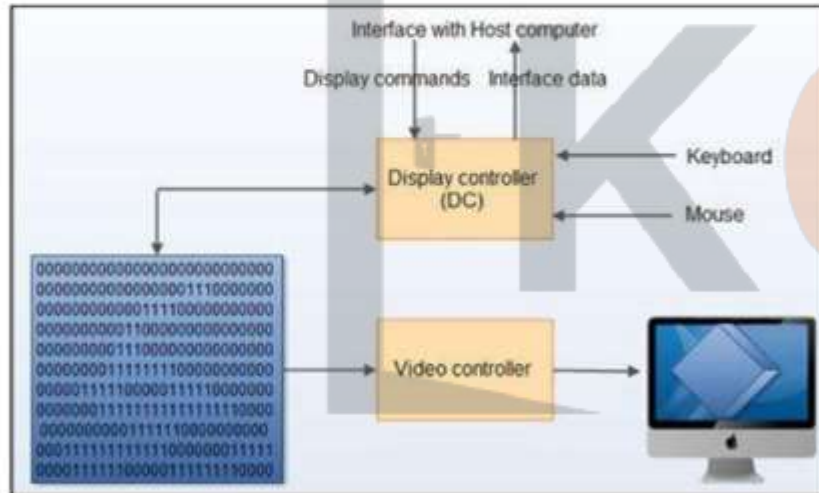
Non-interactive computer graphics use advanced techniques such as global illumination, ray tracing, high-resolution rendering, and shaders. Global illumination simulates the interaction of light with objects to create realistic lighting effects. Ray tracing algorithms trace the path of light rays to generate highly detailed and accurate images.

High-resolution rendering techniques produce visuals with exceptional detail and clarity. Shaders allow for the creation of complex materials and textures.

Graphics Display System

It consists of four components:

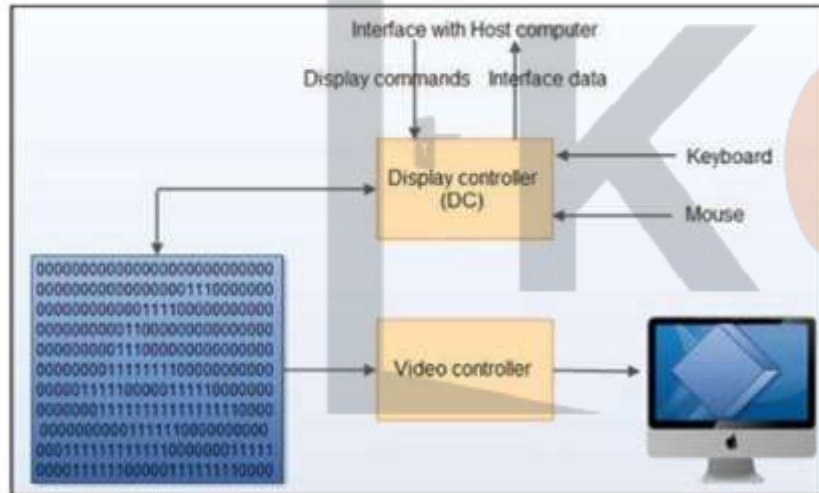
1. Display controller
2. Digital memory (frame buffer or Bitmap)
3. Monitor (computer screen)
4. Video controller



Graphics Display System.

Graphics Display System

It consists of four components:



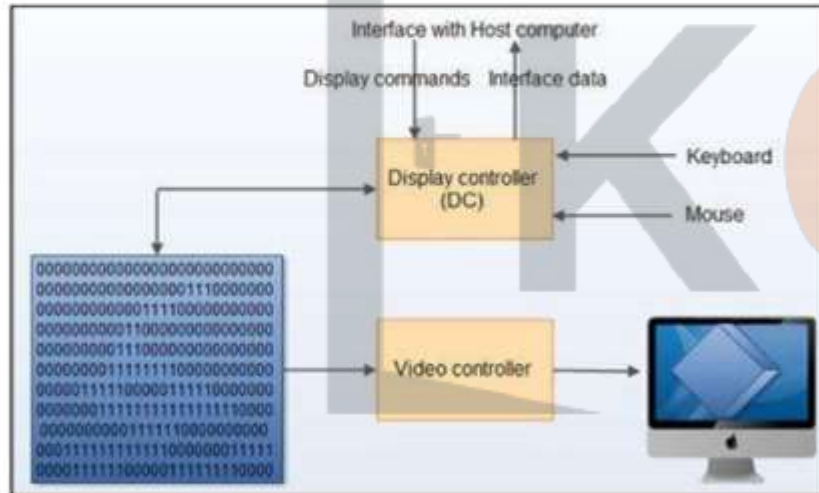
Graphics Display System.

Display controller

A display controller that gets the inputs and commands from the user and determines the image to be displayed on the monitor. The display controller will divide the image into a number of pixels. This image which is to be displayed is stored in the frame buffer. The image will be stored as a matrix of intensity values

Graphics Display System

It consists of four components:



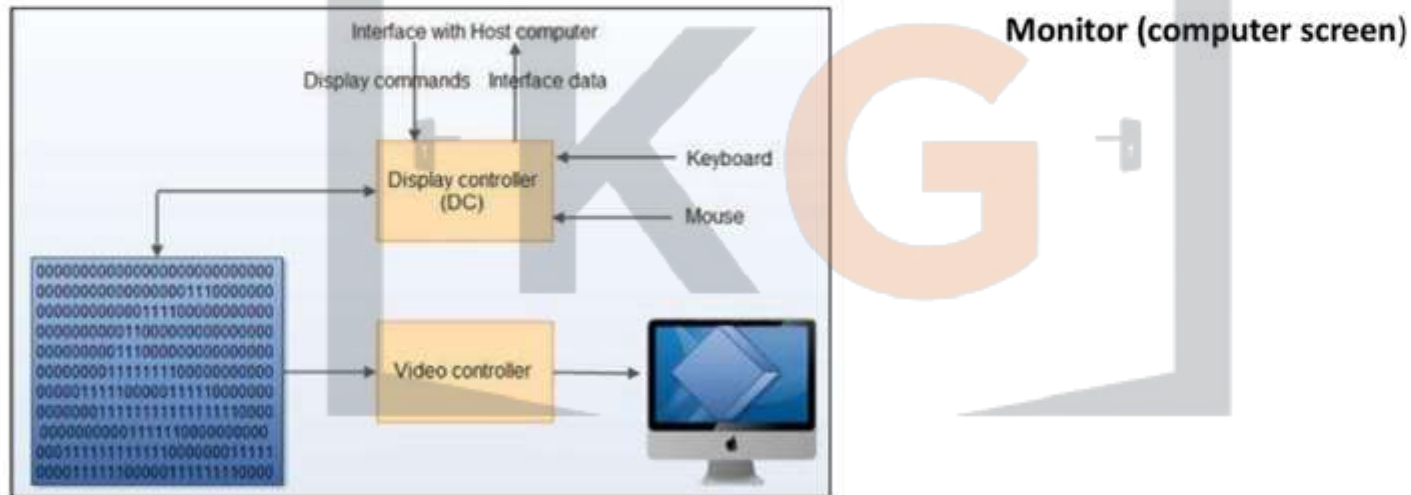
Graphics Display System.

Digital memory (frame buffer or Bitmap)

A digital memory (frame buffer or Bitmap), in which the displayed Image is stored as a matrix of intensity values. The number of rows in the frame buffer array equal the number of raster lines on the display screen, and the number of columns in this array equals the number of pixels on each raster line.

Graphics Display System

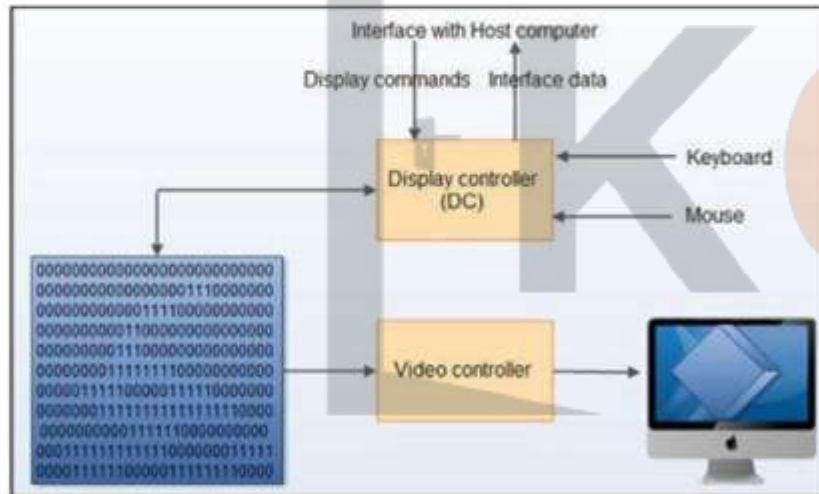
It consists of four components:



Graphics Display System.

Graphics Display System

It consists of four components:



Graphics Display System.

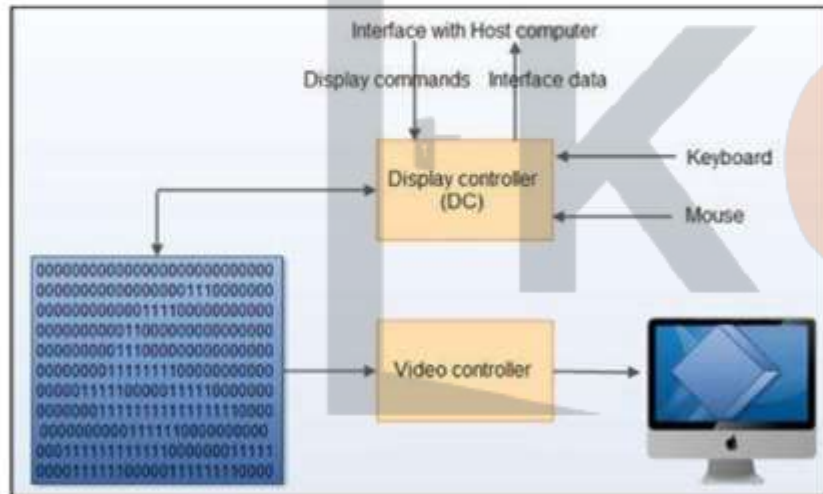
Video controller

which is a simple interface that passes the contents of the frame buffer to the monitor. Inside the frame buffer the image is stored as a pattern of binary digital numbers, which represent a rectangular array of picture elements, or pixel.

In the Simplest case where we wish to store only black and white images, we can represent black pixels by 0's in the frame buffer and white Pixels by 1's. The display controller simply reads each successive byte of data from the frame buffer and converts each 0 and 1 to the corresponding video signal. This signal is then fed to the monitor. If we wish to change the displayed picture all we need to do is to change of modify the frame buffer contents to represent the new pattern of pixels.

Graphics Display System

It consists of four components:



Graphics Display System.

Video controller

A video controller, often referred to as a video or graphics card, is a key hardware component that allows computers to generate graphic information to any video display devices, such as a monitor or projector. Some modern computers do not include video cards, but rather have graphics processing units directly integrated into the computer's motherboard

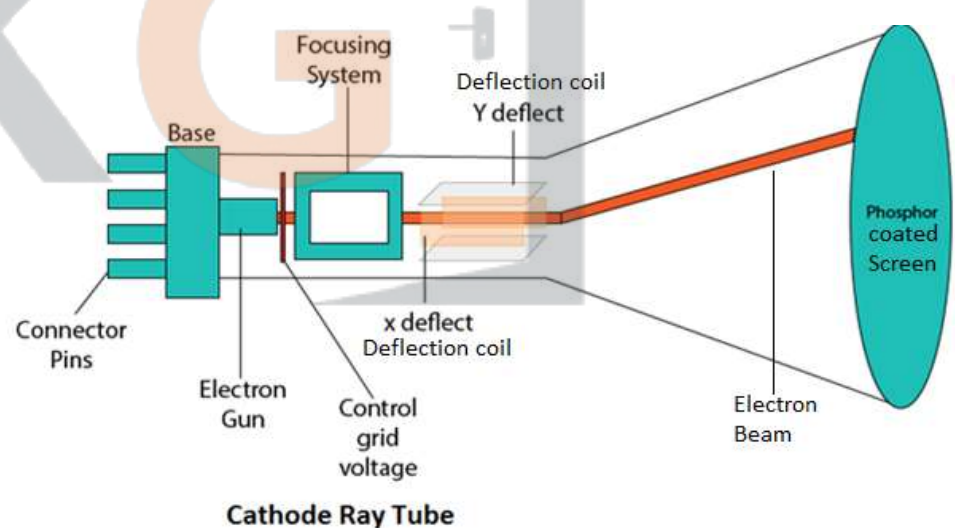
VIDEO DISPLAY DEVICES

- The primary output device in a graphics system is a video monitor.
- The operation of most video monitors is based on the standard **cathode-ray tube (CRT)** design.



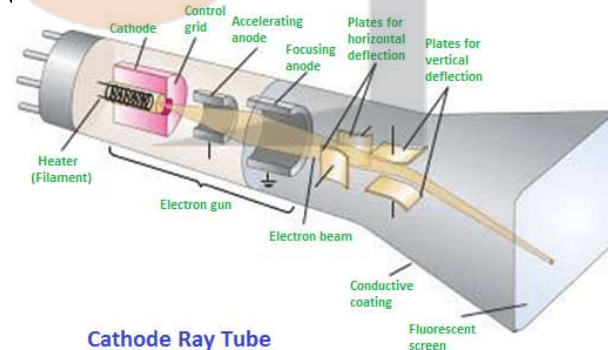
Refresh Cathode-Ray Tubes

1. A beam of electrons (cathode rays), emitted by an electron gun, passes through focusing and deflection systems that direct the beam toward specified positions on the phosphor-coated screen.
2. The phosphor then emits a small spot of light at each position contacted by the electron beam.
3. Since the light emitted by the phosphor fades very rapidly, some method is needed for maintaining the screen picture.
4. One way to keep the phosphor glowing is to redraw the picture repeatedly by quickly directing the electron beam back over the same points.
5. This type of display is called a refresh CRT.



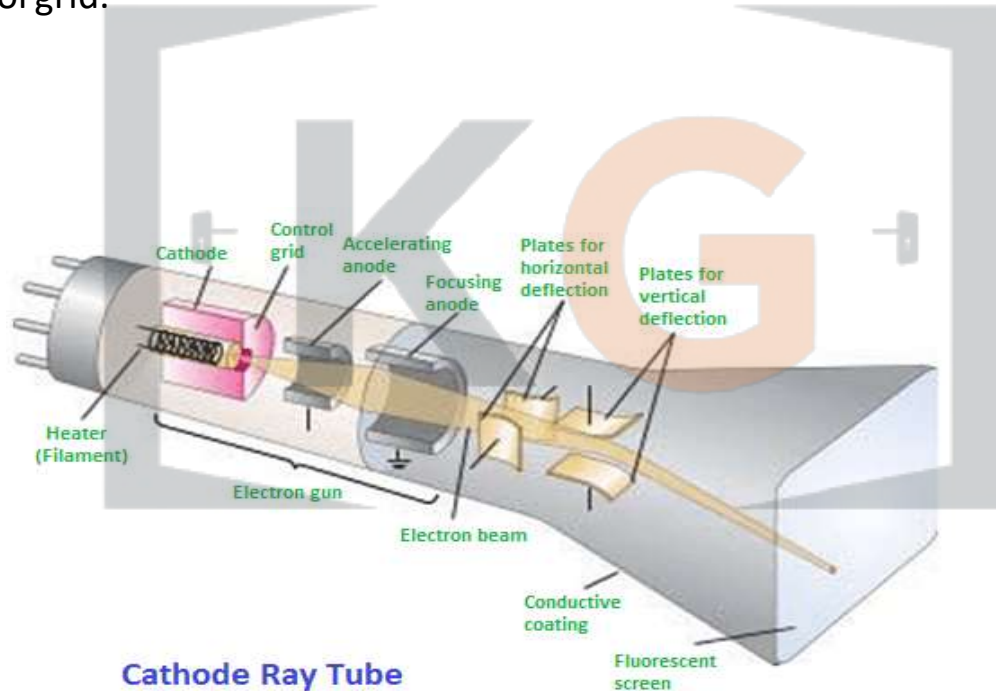
Electron Gun

1. The primary components of an electron gun in a CRT are the **heated metal cathode** and a **control grid**.
2. Heat is supplied to the cathode (- ve charged) by directing a current through a coil of wire, called the **heating filament**, inside the cylindrical cathode structure.
3. Heat causes electrons to be 'boiled off' the hot cathode surface.
4. In the vacuum inside the CRT envelope, the free, negatively charged electrons are then accelerated toward the phosphor coating by a high positive voltage.
5. The accelerating voltage can be generated with a positively charged metal coating on the side of the CRT envelope near the phosphor screen, or an **accelerating anode** can be used.
6. Intensity of the electron beam is controlled by setting voltage levels on the control grid, which is a metal cylinder that fits over the cathode.



Cathode Ray Tube

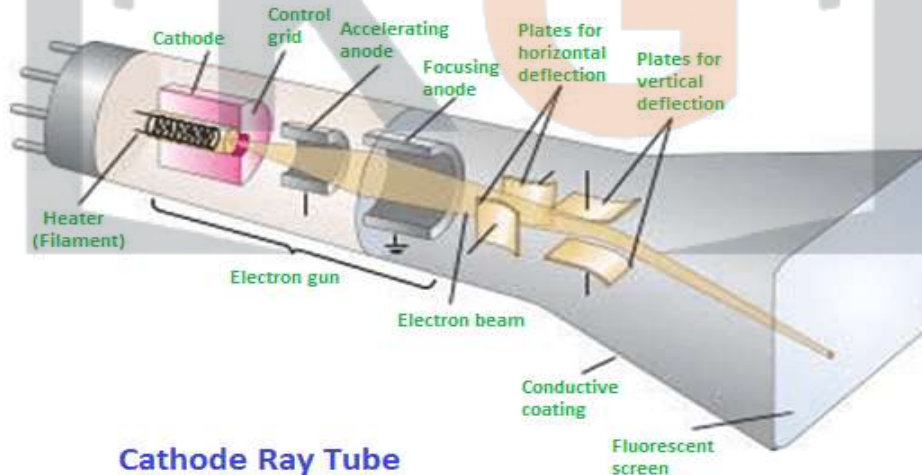
- A high negative voltage applied to the control grid will shut off the beam by repelling electrons and stopping them from passing through the small hole at the end of the control grid structure.
- Since the amount of light emitted by the phosphor coating depends on the number of electrons striking the screen, we control the brightness of a display by varying the voltage on the control grid.



Focusing System

Why Focusing is needed?

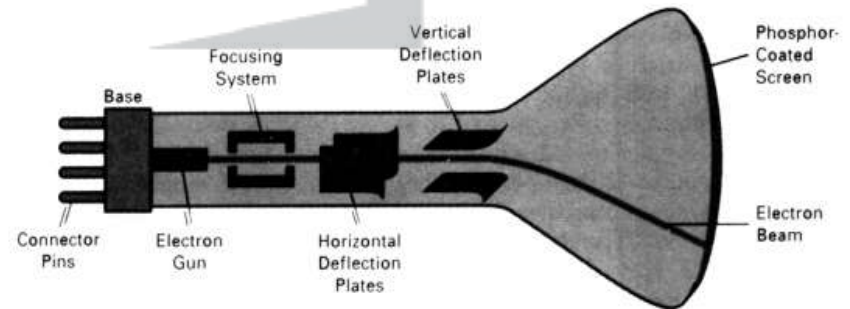
- The **focusing system** in a CRT is needed to force the electron beam to converge into a small spot as it strikes the phosphor. Otherwise, the electrons would repel each other, and the beam would spread out as it approaches the screen.
- Also the distance that the electron beam must travel to different points on the screen varies because the **radius of curvature** for most CRTs is greater than the distance from the focusing system to the screen centre.
- As the beam moves to the outer edges of the screen, displayed images become blurred.
- To compensate for this, the system can adjust the focusing according to the screen position of the beam
- Focusing is accomplished with either **electric** or **magnetic fields**.



Cathode Ray Tube

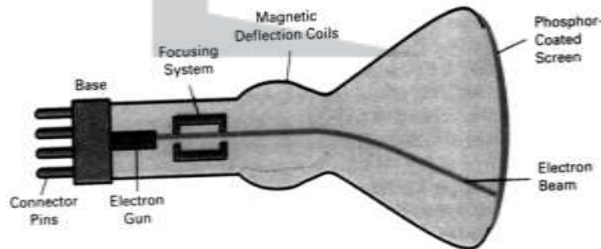
Electrostatic Focusing

- **Electrostatic focusing** is commonly used in television and computer graphics monitors. With electrostatic focusing, the electron beam passes through a positively charged metal cylinder that forms an electrostatic lens.
- The action of the electrostatic lens focuses the electron beam at the screen, in exactly the same way that an optical lens focuses a beam of light at a particular focal distance.
- In electrostatic deflection -- two pairs of parallel plates are mounted inside the CRT envelope.
- One pair of plates is mounted horizontally to control the vertical deflection, and the other pair is mounted vertically to control horizontal deflection.



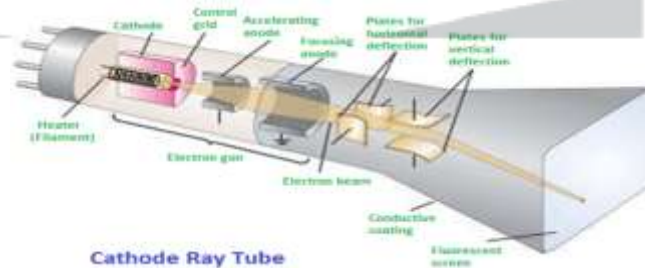
Magnetic Deflection

- Similar lens focusing effects can also be accomplished with **a magnetic field** set up by a coil mounted around the outside of the CRT envelope.
- Two pairs of coils are used, with the coils in each pair mounted on opposite sides of the neck of the CRT envelope.
- One pair is mounted on the top and bottom of the neck, and the other pair is mounted on opposite sides of the neck.
- The magnetic field produced by each pair of coils results in a **transverse deflection force** that is perpendicular both to the direction of the magnetic field and to the direction of travel of the electron beam.
- Horizontal deflection is accomplished with one pair of coils, and vertical deflection by the other pair.
- The proper deflection amounts are attained by adjusting the current through the coils.



What happens when the electrons collide with screen

- Spots of light are produced on the screen by the transfer of the CRT beam energy to the phosphor.
- When the electrons in the beam collide with the phosphor coating, they are stopped and then kinetic energy is absorbed by the phosphor.
- Part of the beam energy is converted by friction into heat energy, and the remainder causes electrons in the phosphor atoms to move up to higher quantum- energy levels.
- After a short time, the "excited phosphor electrons begin dropping back to their stable ground state, giving up their extra energy as small quantum of Light energy.
- What we see on the screen is the combined effect of all the electron light emissions: **a glowing spot that quickly fades after all the excited phosphor electrons have returned to their ground energy level.**



Some terms related to Graphics

Persistence

- Persistence is defined as the time it takes the emitted light from the screen to decay to one-tenth of its original intensity.
- Lower persistence phosphors require higher refresh rates to maintain a picture on the screen without flicker.
- Low persistence phosphor is useful for animation; a high-persistence phosphor is useful for displaying highly complex, static pictures.

Resolution

- The maximum number of points that can be displayed without overlap on a CRT is referred to as the resolution.
- The number of points per centimetre that can be plotted horizontally and vertically.

Aspect Ratio

- The **aspect ratio** of an image is the ratio of its width to its height.
- It is commonly expressed as two numbers separated by a colon, as in *16:9*. For an *x:y* aspect ratio, the image is *x* units wide and *y* units high.
- Widely used aspect ratios include 1.85:1 and 2.39:1 in film photography, 4:3 and 16:9 in television, and 3:2 in still camera photography.

Question: Find out the aspect ratio of the raster system using 8 x 10 inches screen and 100 pixel/inch.



Question: Find out the aspect ratio of the raster system using 8 x 10 inches screen and 100 pixel/inch.

Ans. We know that,
Aspect ratio = Width / Height

$$= 8 \times 100 / 10 \times 100 = 4 / 5$$

Aspect ratio = 4 : 5



Bit map and pix map

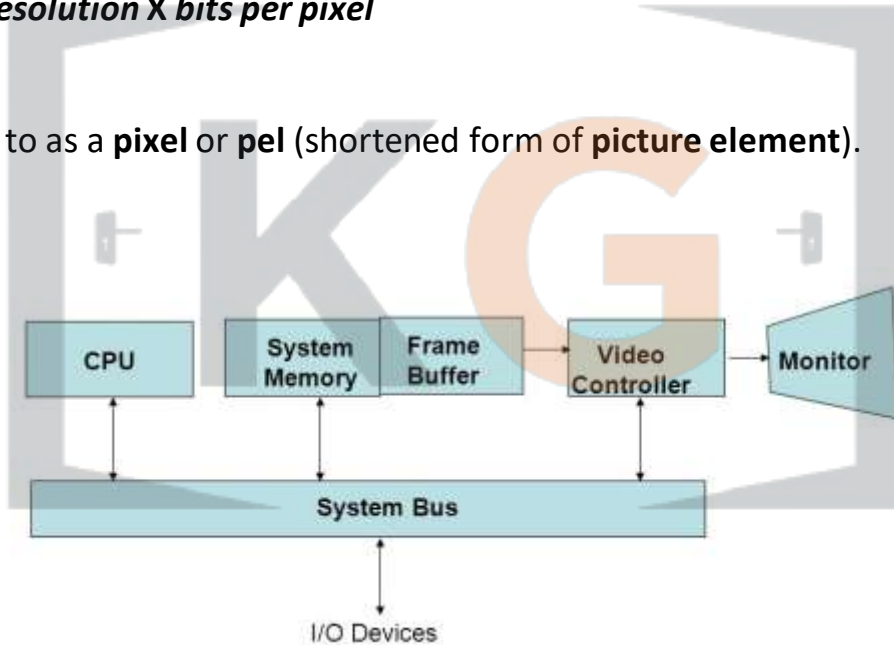
- On a black-and-white system with one bit per pixel, the frame buffer is commonly called a **bitmap**.
- For systems with multiple bits per pixel, the frame buffer is often referred to as a **pixmap**.



- Picture definition is stored in a memory area called the **refresh buffer** or **frame buffer**.
- This memory area holds the set of intensity values for all the screen points. Stored intensity values are then retrieved from the refresh buffer and "painted" on the screen one row (scan line) at a time.
- It is accessed by the video controller to refresh the image after a point of time.
- **The size of frame buffer = Resolution X bits per pixel**

Pixel

- Each screen point is referred to as a **pixel** or **pel** (shortened form of **picture element**).



Example: How Many KB's does a frame buffer needs in a 600 x 400 pixel ?

Suppose 1 pixel can store n bits

Example: A system with 24 bits per pixel and a screen resolution of 1024 x 1024 requires _____ megabytes of storage for the frame buffer.

Example: How Many KB's does a frame buffer needs in a 600 x 400 pixel ?

Suppose 1 pixel can store n bits

Since we know that,

Frame Buffer = Resolution x bits per Pixels

Frame Buffer = $600 * 400 * n$ bits = 240000 n bits

$$= \frac{240000 \text{ } n \text{ bits}}{1024 * 8 \text{ bits}} = 29.29 \text{ } n \text{ KB}$$

Example: A system with 24 bits per pixel and a screen resolution of 1024 x 1024 requires _____ megabytes of storage for the frame buffer.

In the above example, since 8 bits constitutes a byte, frame buffer size can be calculated as:

Since we know that,

Frame Buffer = Resolution x Pixels

$$\frac{1024 * 1024 * 24 \text{ bits}}{8 \text{ bits}} = 2^{10} * 2^{10} * 3 = 3 \text{ MB}$$

Example: Suppose RGB raster system is to be designed using on 8 inch X 10 inch screen with a resolution of 100 pixels per inch in each direction. If we want to store 6 bits per pixel in the frame buffer, how much storage (in bytes) do we need for frame buffer?



Example: Consider three different raster systems with resolutions of 640 x 480, 1280 x 1024, and 2560 x 2048. What size is frame buffer (in kilobytes) for each of these systems to store 12 bits per pixel?



Example: Consider three different raster systems with resolutions of 640 x 480, 1280 x 1024, and 2560 x 2048. What size is frame buffer (in kilobytes) for each of these systems to store 12 bits per pixel?

Ans. frame-buffer sizes of the systems are as follows:

$(640 \times 480 \times 12 \text{ bits}) / (1024 \times 8) = 450\text{KB};$

$1280 \times 1024 \times 12 \text{ bits} / (1024 \times 8) = 1920\text{KB};$

$2560 \times 2048 \times 12 \text{ bits} / (1024 \times 8) = 7680\text{KB};$



Raster Scan System



Random Scan System



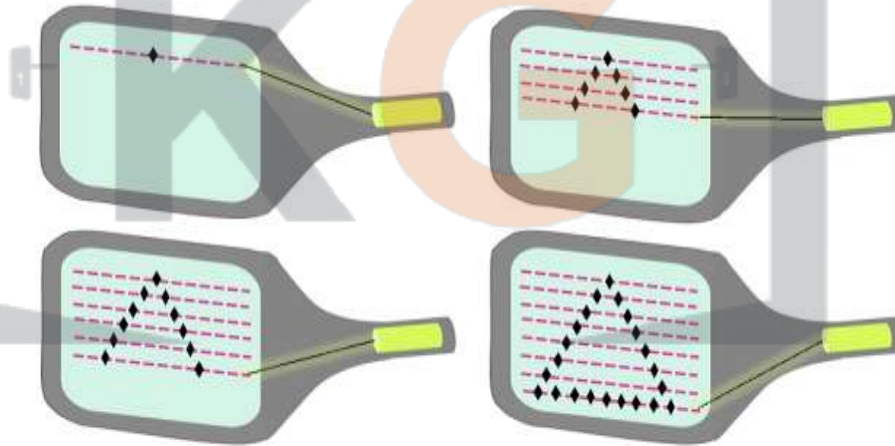
VS

Raster and Random-Scan Displays

Raster-Scan Displays

Working

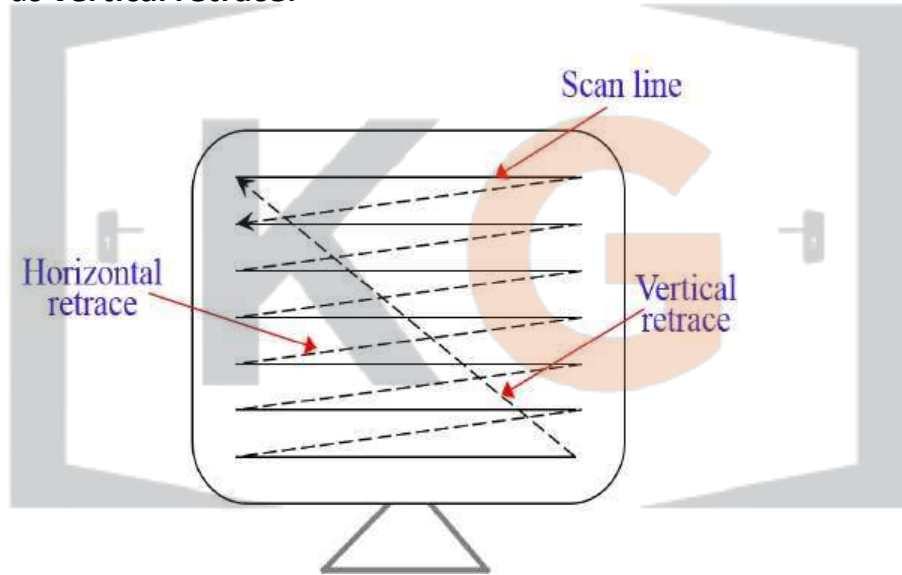
- In a raster-scan system, the electron beam is swept across the screen, one row at a time from top to bottom as shown in the figure below.
- As the electron beam moves across each row, the beam intensity is turned on and off to create a pattern of illuminated spots.



Raster Scan: A raster scan system displays an item as a group of separate points along each screen line

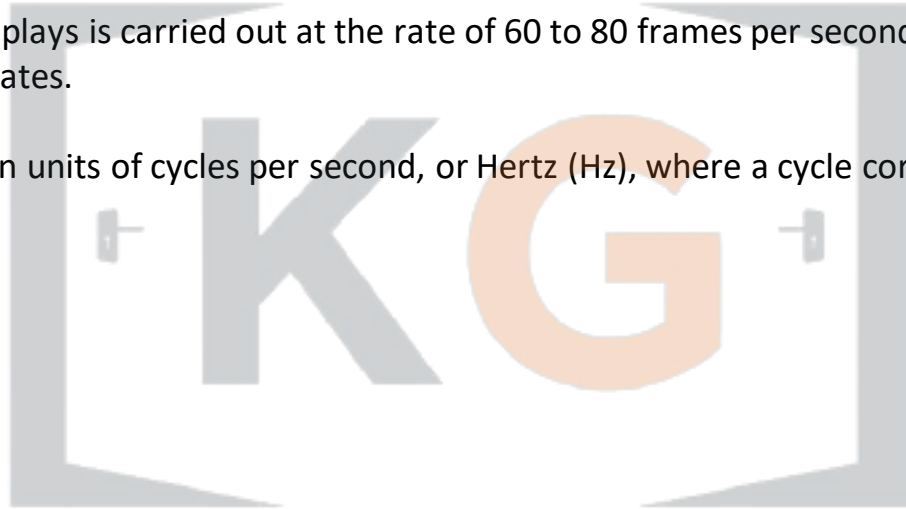
Horizontal and Vertical Retrace

- At the end of each scan line, the electron beam returns to the left side of the screen to begin displaying the next scan line known as **Horizontal Retrace**.
- At the end of each frame the electron beam returns to the top left corner of the screen to begin the next frame known as **vertical retrace**.



Refreshing on Raster Scans

- In a computer display, to refresh is to redraw the image information from memory .
- Computer or television displays have to be refreshed because they don't have the capacity to hold a stable image as the intensity of a spot lowers down with time.
- Refreshing on raster-scan displays is carried out at the rate of 60 to 80 frames per second, although some systems are designed for higher refresh rates.
- Refresh rates are described in units of cycles per second, or Hertz (Hz), where a cycle corresponds to one frame.



Ques: With respect to CRT, the horizontal retrace is defined as:

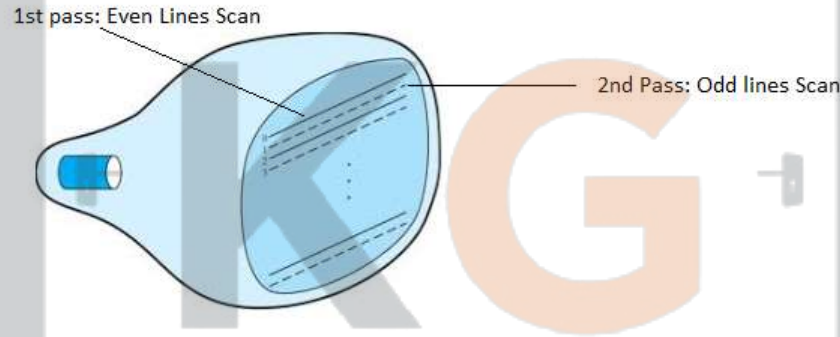
- a)** The path an electron beam takes when returning to the left side of the CRT.
- b)** The path an electron beam takes when returning to the right side of the CRT.
- c)** The technique of turning, the electron beam off while retracing.
- d)** The technique of turning, the electron beam on/ off while retracing.

Ans. a



Interlaced Refresh Procedure

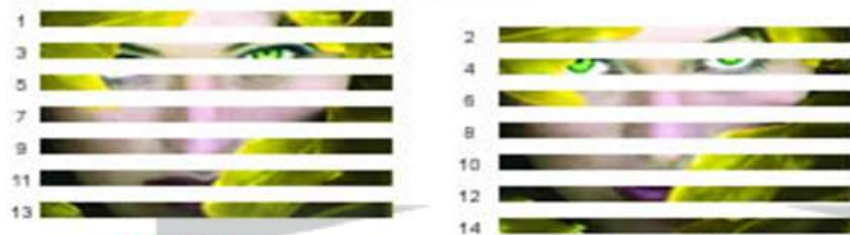
- On some raster-scan systems (and in TV sets), each frame is displayed in two passes using an interlaced refresh procedure.
- In the first pass, the beam sweeps across every other (even) scan line from top to bottom. Then after the vertical retrace, the beam sweeps out the remaining (i.e. odd) scan lines.



Advantages:

- Interlacing of the scan lines allows us to see the entire screen displayed in one-half the time it would have taken to sweep across all the lines at once from top to bottom.
- Effective technique for avoiding flicker.

Interlaced



Odd lines
Field 1

Even lines
Field 2



Field 1 + Field 2 = Frame (complete image)

Display Rate: 60 fields per second (North America)

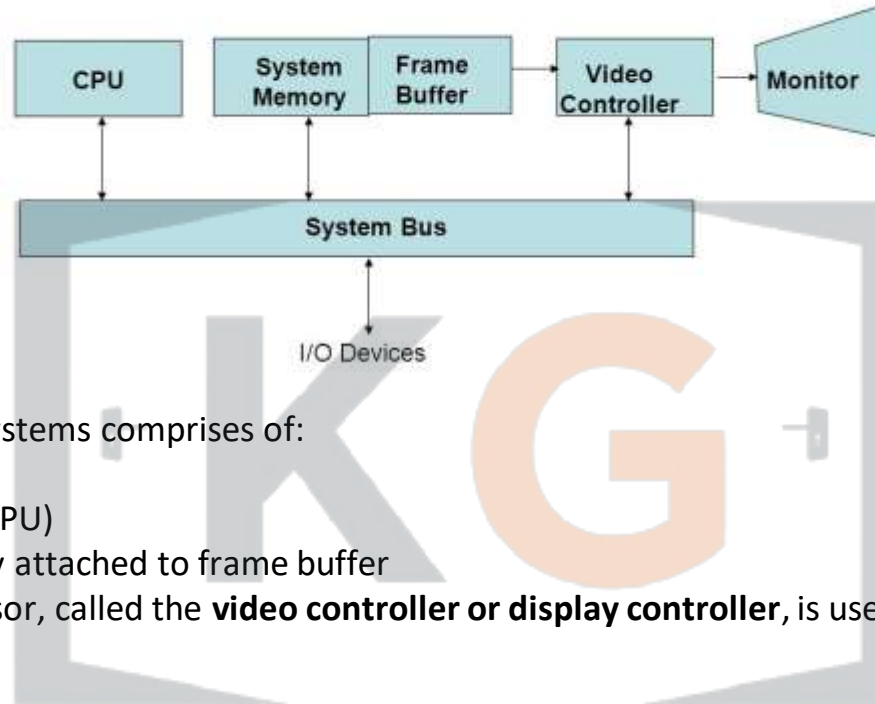
Example:

On an older, 30 frames per- second, noninterlaced display, some flicker is noticeable.

But with interlacing, each of the two passes can be accomplished in $1/60^{\text{th}}$ of a second ($1/60$ for even lines and $1/60$ for odd lines), which brings the refresh rate nearer to 60 frames per second



Architecture of Raster Scan Systems

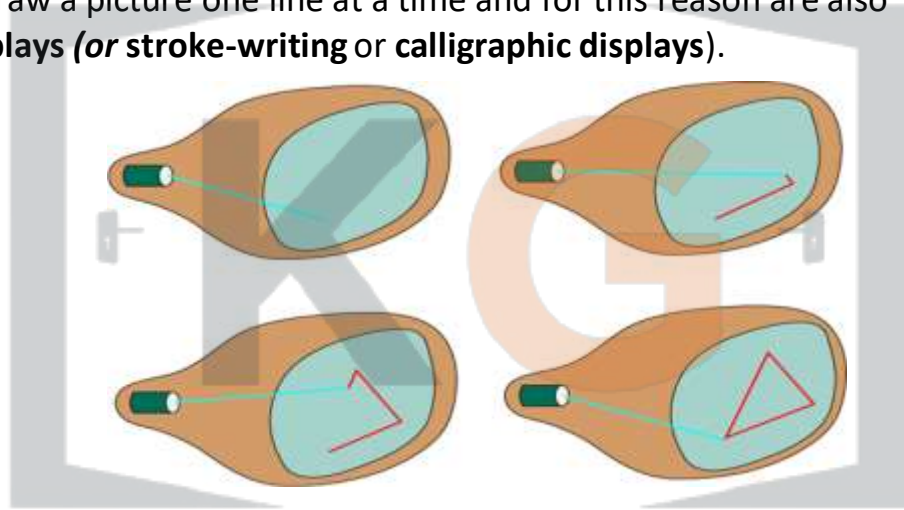


The architecture of Raster systems comprises of:

- Central processing unit (CPU)
- System Memory generally attached to frame buffer
- A special-purpose processor, called the **video controller or display controller**, is used to control the operation of the display device.
- The video controller accesses the frame buffer to refresh the screen.
- Entire image is scanned out sequentially by video controller.

Random-Scan Displays

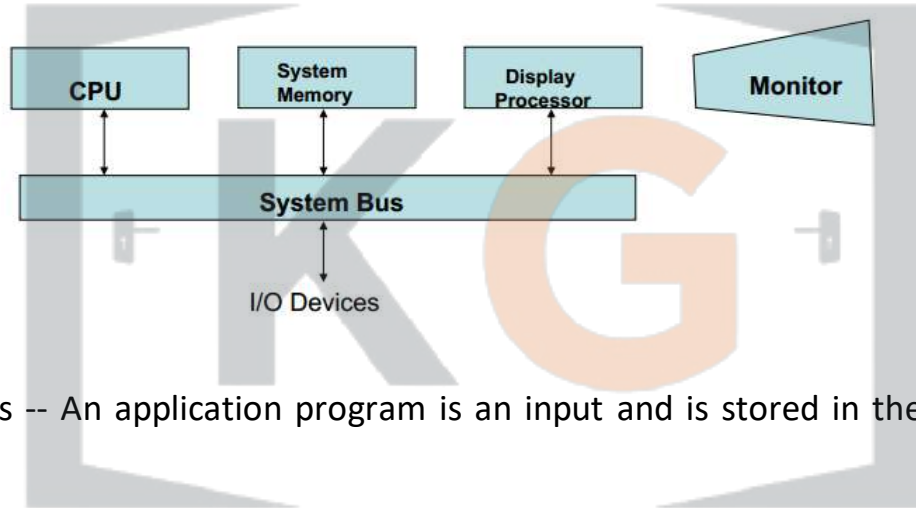
- In random-scan display unit, a CRT has the electron beam directed only to the parts of the screen where a picture is to be drawn.
- Random scan monitors draw a picture one line at a time and for this reason are also referred to as **vector displays (or stroke-writing or calligraphic displays)**.



- The component lines of a picture can be drawn and refreshed by a random-scan system in any specified order as shown above.

- Picture definition is stored as a set of line **drawing commands** in an area of memory referred to as the **refresh display file or display list or display program or refresh buffer**
- Random-scan systems are designed for line drawing applications and cannot display realistic shaded scenes.

RANDOM-SCAN SYSTEM



- In Random Scan Systems -- An application program is an input and is stored in the system memory along with a graphics package.
- Graphics commands in the application program are translated by the graphics package into a display file stored in the system memory.
- This display file is then accessed by the display processor to refresh the screen.

Comparison of Raster and Random Scan

BASIS FOR COMPARISON	RASTER SCAN	RANDOM SCAN
Electron beam	Swept across the screen and handles one row at a time and in downward direction.	Directed to the portions of the screen where a picture is to be rendered.
Resolution	Poor, since it generates meander lines which are organized as distinct point sets.	Good, as this produces even lines drawing.
Picture definition	Stored as the combination of intensity values for all screen points.	Stored as a group of line drawing instructions in a display file.
Realistic display	Effectively displays realistic scenes.	Unable to display realistic shaded scenes.

Q. Raster images are commonly called (UP LT - 2018)

- A) pixmap**
- B) bitmap**
- C) Both (a) and (b)**
- D) None of the above**



Output Primitives

- Functions provided by graphics packages to describe a scene in terms of basic geometric structures such as straight-line segments and polygon area colours, referred to as output primitives.
- Each output primitive is specified with input coordinate data and other information about the way that object is to be displayed.
- Points and straight-line segments are the simplest geometric components of pictures.

Point Plotting

- Point plotting is done by converting a single coordinate position into appropriate operations for the output device in use.

Example: In a CRT monitor the electron beam is turned on to illuminate the screen phosphor at the selected location.

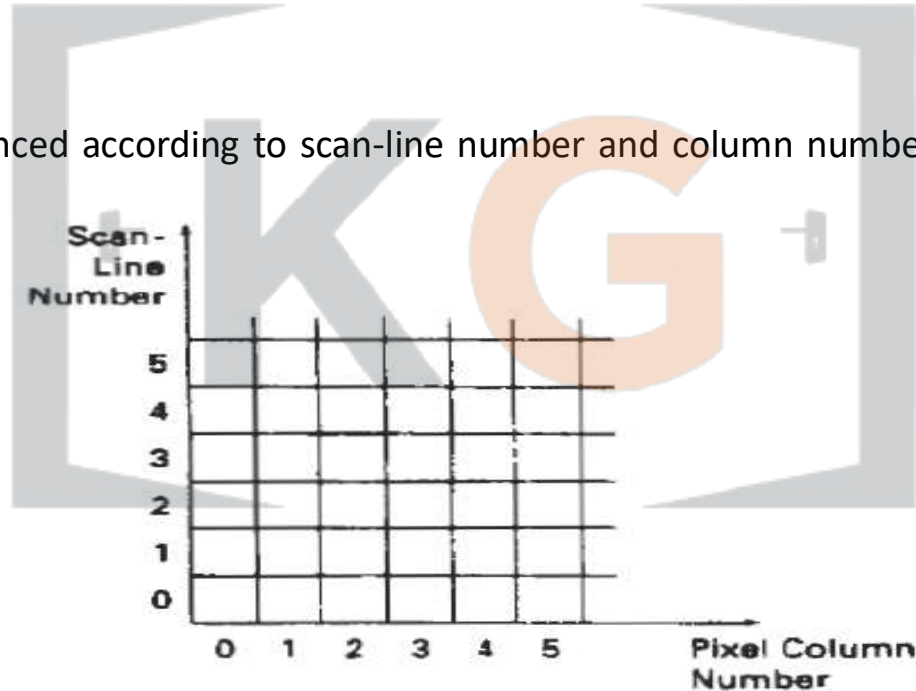


Line Drawing

- Line drawing is accomplished by calculating intermediate positions along the line path between two specified endpoint positions.
- An output device is then directed to fill in these positions between the endpoints.

Positions of Pixels

- Pixel positions are referenced according to scan-line number and column number (pixel position across a scan line).



LINE-DRAWING ALGORITHMS

- Digital Differential Analyzer (DDA) Algorithm
- Bresenham's Line Algorithm
- Midpoint Line Algorithm

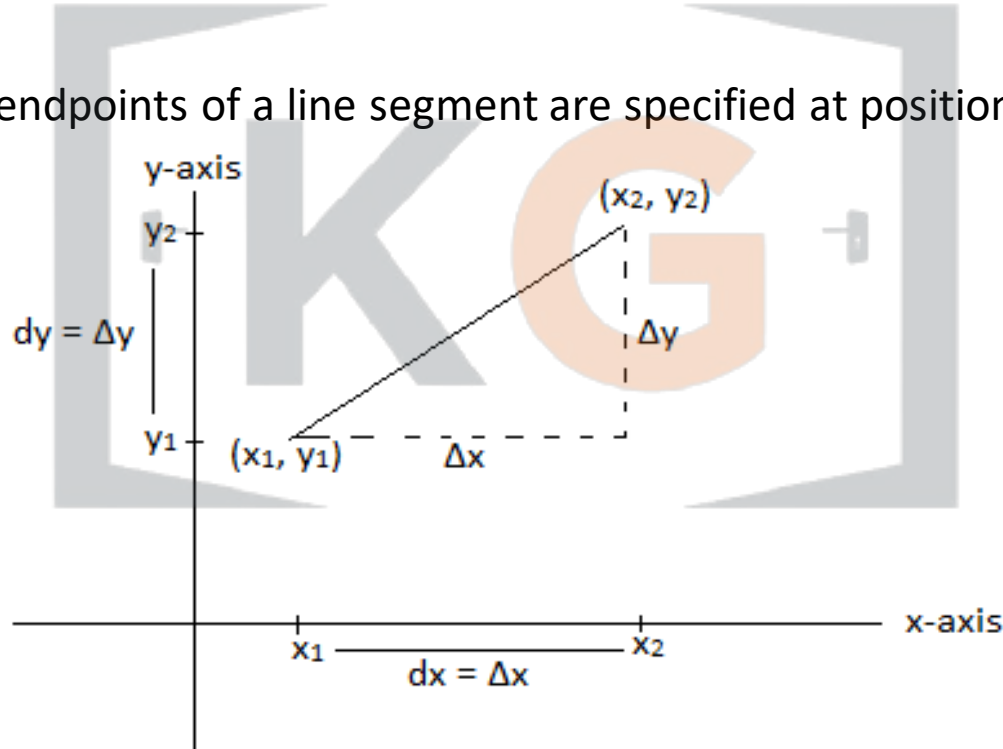


LINE-DRAWING ALGORITHMS

- The Cartesian ***slope-intercept equation*** for a straight line is

$$y = m \cdot x + c$$

- Where, m represents the slope of the line and c represents the y intercept.
- Given that the two endpoints of a line segment are specified at positions (x_1, y_1) and (x_2, y_2)



We determine values for the slope m and y intercept b with the following calculations:

$$m = \frac{y_2 - y_1}{x_2 - x_1} \dots \dots \dots (1)$$

$$c = y_1 - m \cdot x_1 \dots \dots \dots (2)$$

Also,

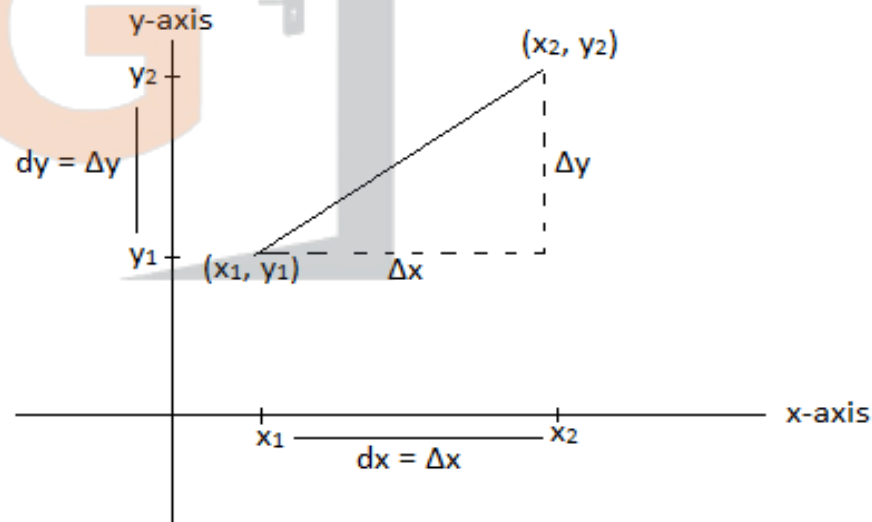
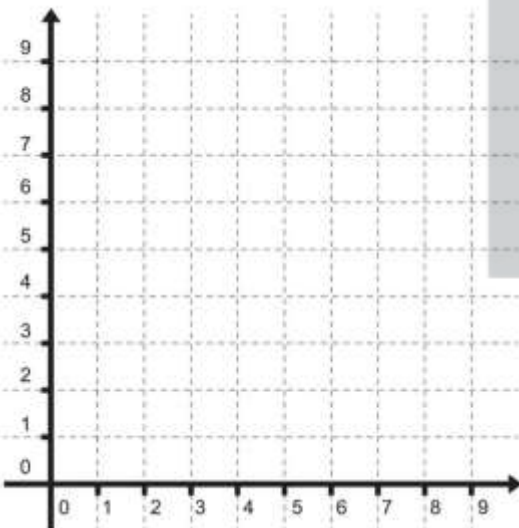
$$m = \frac{\Delta y}{\Delta x} \quad \text{As } \Delta y = y_2 - y_1 \text{ and } \Delta x = x_2 - x_1$$

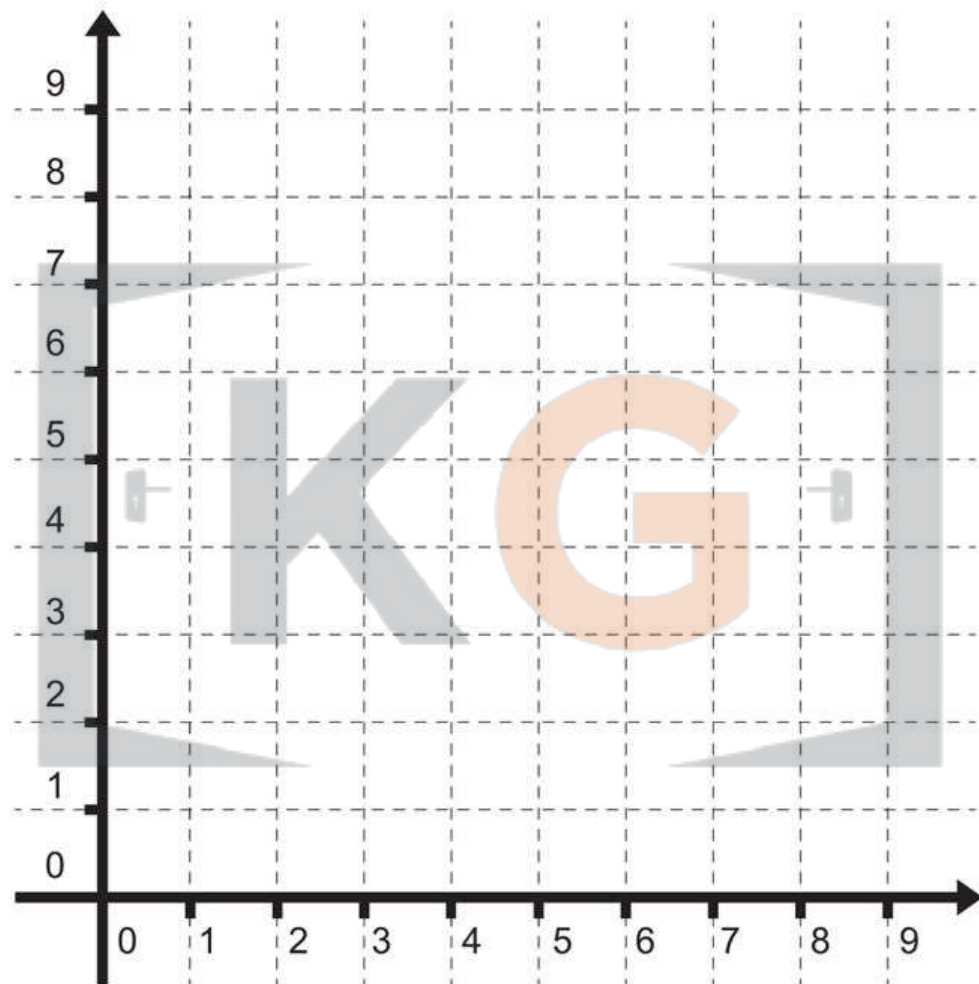
- For any given x interval Δx along a line, we can compute the corresponding y interval Δy from equation:

$$\Delta y = m \Delta x$$

- Similarly, we can obtain the x interval Δx corresponding to a specified Δy as

$$\Delta x = \frac{\Delta y}{m}$$



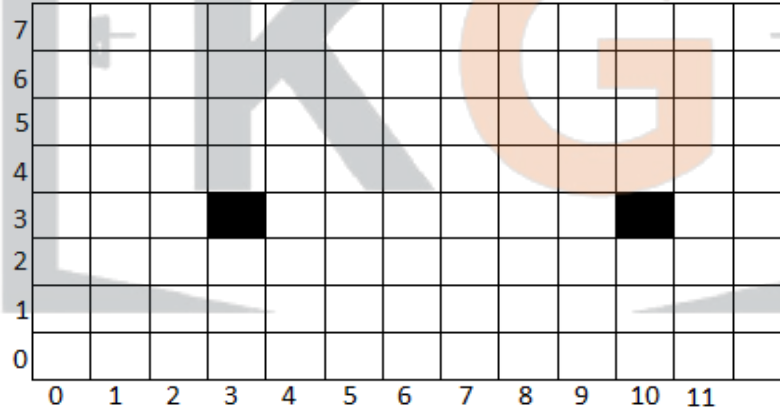


DDA Algorithm (Digital Differential Analyzer)

- The *digital differential analyser* (DDA) is a scan-conversion line algorithm based on calculating either Δy or Δx .
- **Examples to understand the DDA Algorithm**

Case 1: For Slope = 0

Consider the points (x_1, y_1) and (x_2, y_2) to be: $(3, 3)$ and $(10, 3)$. If we look at the display the points will be as –



Step 1: Find $\Delta x = x_2 - x_1 = 10 - 3 = 7$

Step 2: Find $\Delta y = y_2 - y_1 = 3 - 3 = 0$

Step 3: Find Slope $m = \Delta y / \Delta x = 0 / 7 = 0$ (i.e. it is a horizontal line)

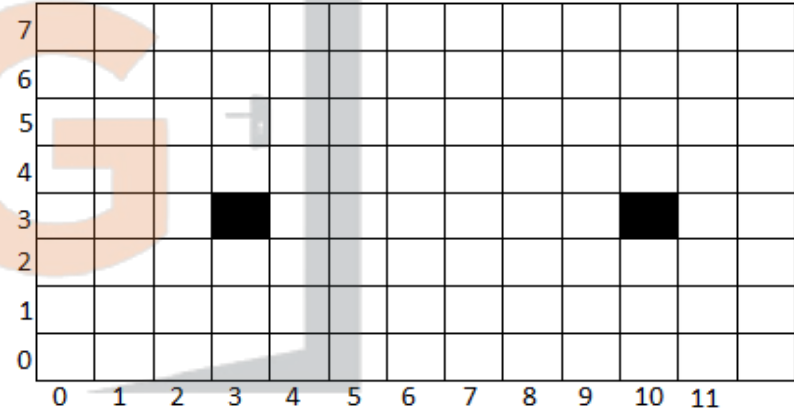
Step 4: Find total number of steps to draw line -- Compare the values of Δx and Δy whichever is greater will be the number of steps to draw a line.

Total steps to draw the line will be = 7

Step 5: Find Increments to be made in x-axis and y-axis in

$$x_{\text{inc}} = \Delta x / \text{total steps} = 7/7 = 1$$

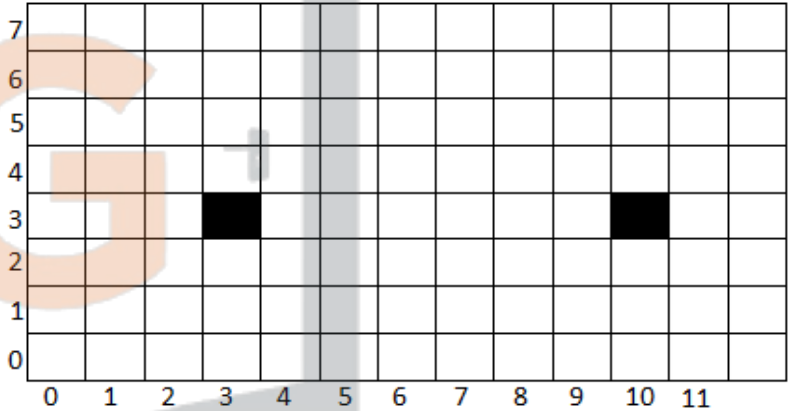
$$y_{\text{inc}} = \Delta y / \text{total steps} = 0/7 = 0$$



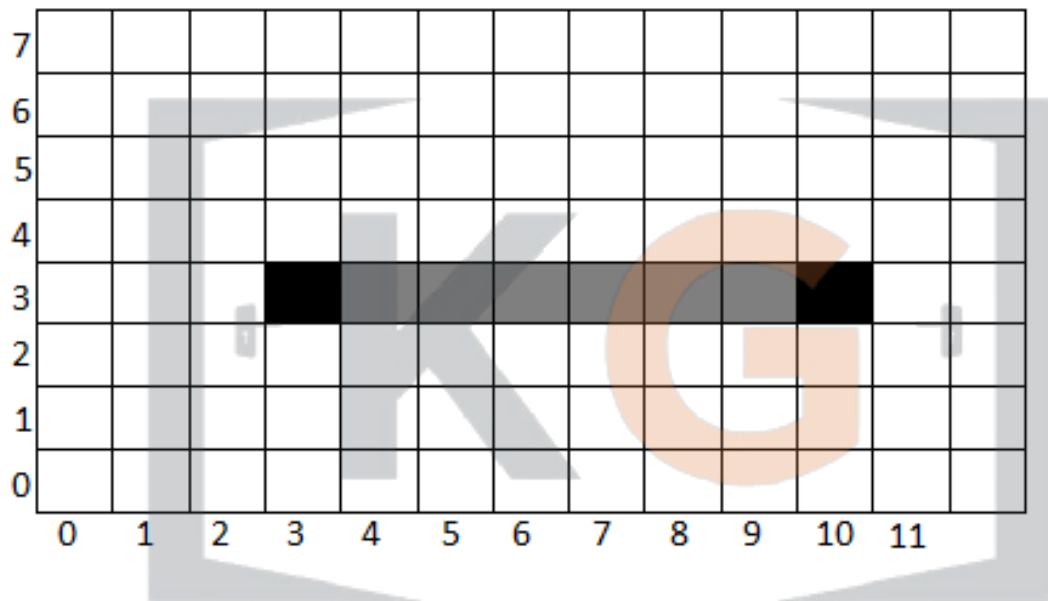
Step 6: Find the corresponding values of x and y

We get the corresponding values for x and y in each steps as:

x	y
3	3
4	3
5	3
6	3
7	3
8	3
9	3
10	3



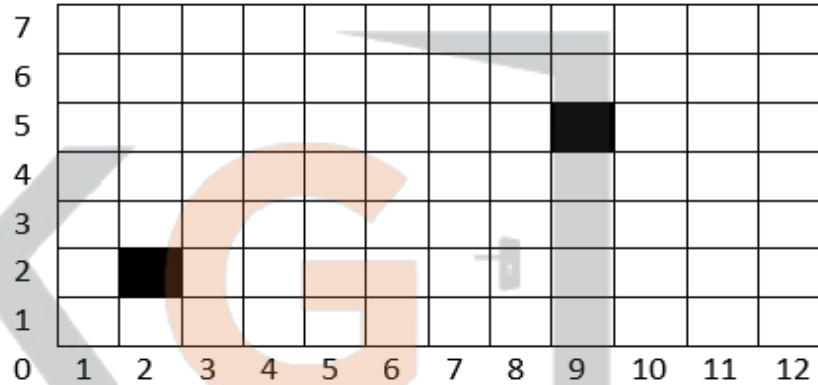
The final line will be:



Case 2: For slope < 1

Consider the points (x_1, y_1) and (x_2, y_2) to be: $(2, 2)$ and $(9, 5)$

If we look at the display the points will be as –



Here

$$\Delta x = x_2 - x_1 = 9 - 2 = 7$$

$$\Delta y = y_2 - y_1 = 5 - 2 = 3$$

$$\text{Slope } m = \Delta y / \Delta x = 3 / 7 = 0.4$$

Total steps to draw the line will be = 7

Now to draw the line the increment in

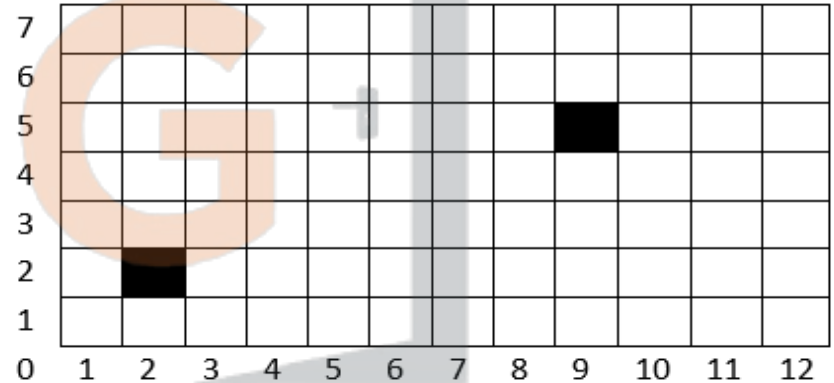
$$x_{\text{inc}} = \Delta x / \text{total steps} = 7/7 = 1$$

$$y_{\text{inc}} = \Delta y / \text{total steps} = 3/7 = 0.42$$

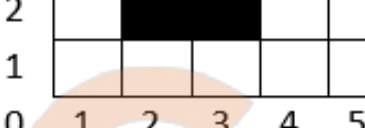
Thus, we get the corresponding values for x and y in each steps as:

Note that we will round off the values when necessary

x	y
2	2
3	2.4 = 2
4	2.8 = 3
5	3.2 = 3
6	3.6 = 4
7	4 = 4
8	4.4 = 4
9	4.8 = 5

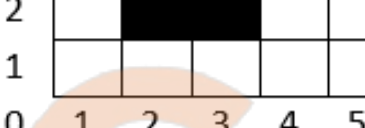


the x and y value as:

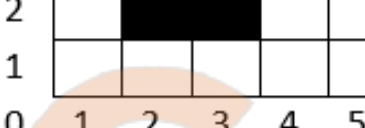


	1	2	3	4	5	6
0						
1						
2						

the x and y value as:

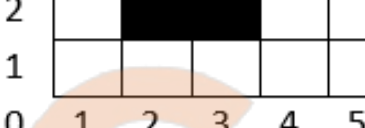


the x and y value as:



	1	2	3	4	5	6
0						
1						
2						

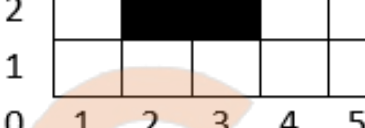
the x and y value as:

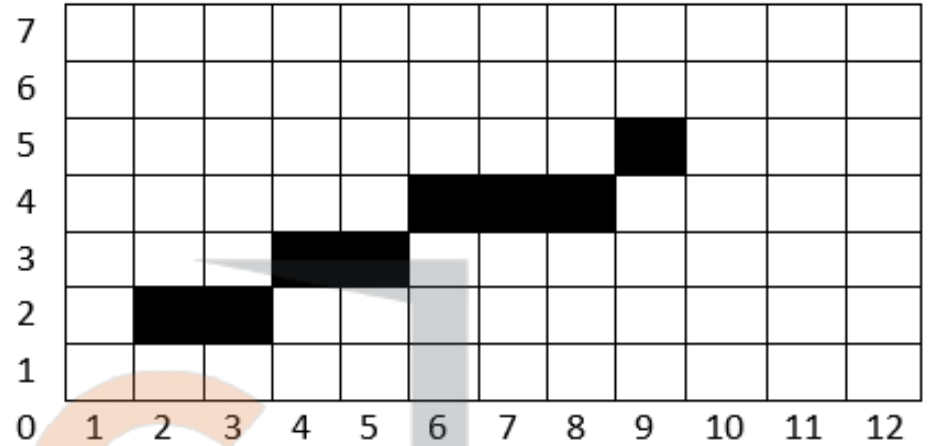


the x and y value as:

	1	2	3	4	5	6
0						
1						
2						
3						

the x and y value as:

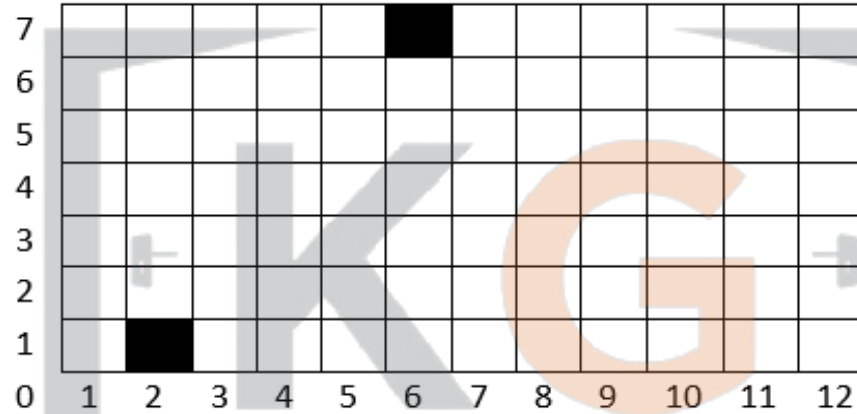




Case 3: For Slope > 1

Consider the points (x_1, y_1) and (x_2, y_2) to be: $(2, 1)$ and $(6, 7)$

The initial points and the final points are like –



Here

$$\Delta x = x_2 - x_1 = 6 - 2 = 4$$

$$\Delta y = y_2 - y_1 = 7 - 1 = 6$$

$$\text{Slope } m = \Delta y / \Delta x = 6 / 4 = 1.5$$

Total steps to draw the line will be = 6

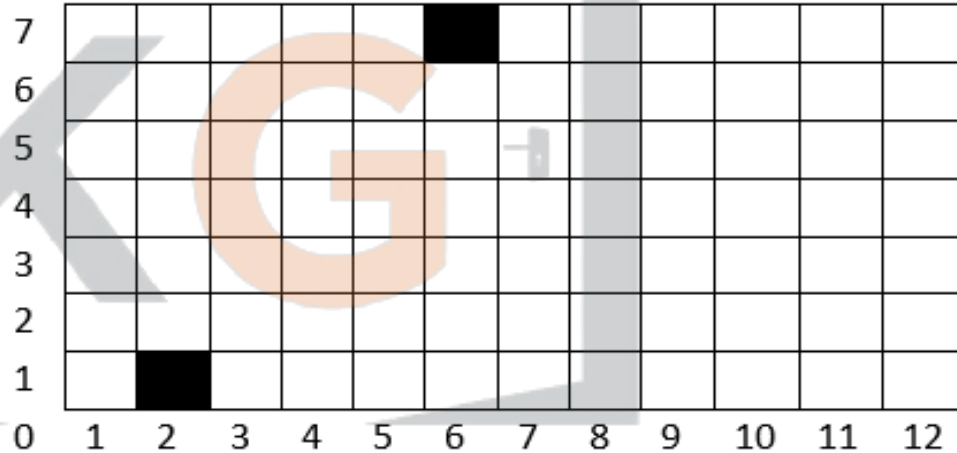
Now to draw the line the increment in

$$x_{\text{inc}} = \Delta x / \text{total steps} = 4/6 = 0.7$$

$$y_{\text{inc}} = \Delta y / \text{total steps} = 6/6 = 1$$

Thus, we get the corresponding values for x and y in each steps as:

x	y
2	1
$2.7 = 3$	2
$3.4 = 3$	3
$4.1 = 4$	4
$4.8 = 5$	5
$5.5 = 5$	6
$6.2 = 6$	7



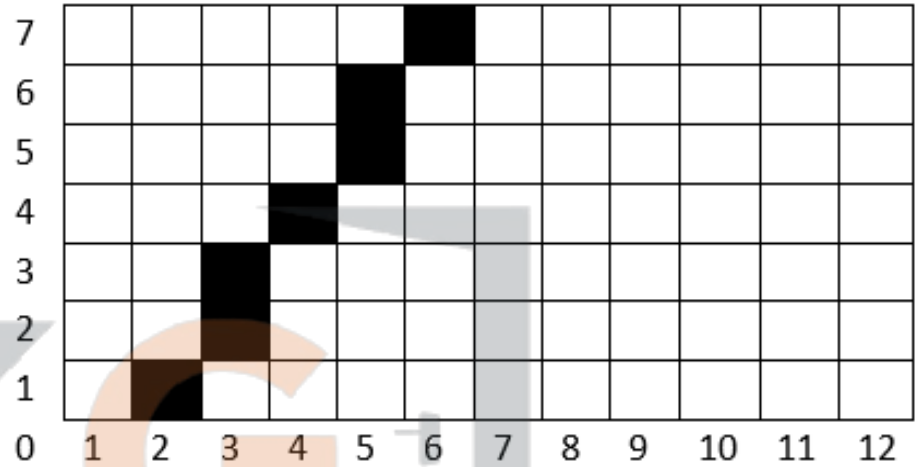
Give x and y value as:



4							
3							
2							
1							
0							
	1	2	3	4	5	6	7

If the slope is greater than 1,
we compute each successive x and y value as:

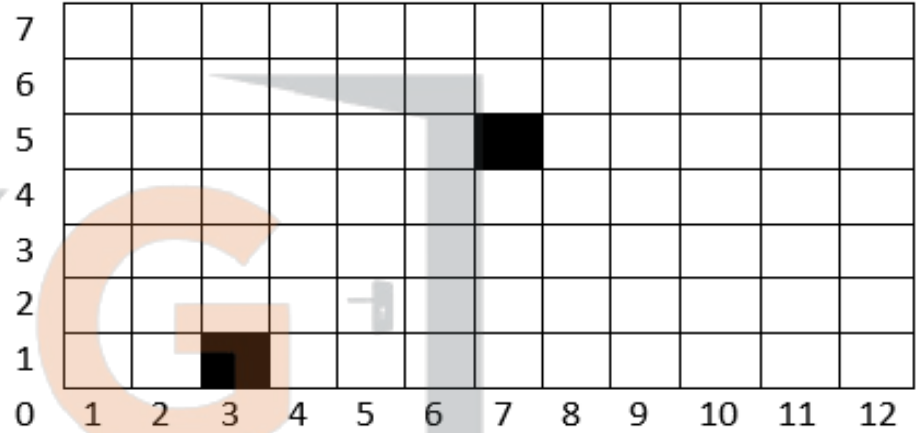
$$\mathbf{y}_{k+1} = \mathbf{y}_k + \mathbf{1}$$



Case 4: For, slope = 1

Let us take another example for -- (x_1, y_1) and (x_2, y_2) to be: (3, 1) and (7, 5)

The initial figure for the required line will be –



Here

$$\Delta x = x_2 - x_1 = 7 - 3 = 4$$

$$\Delta y = y_2 - y_1 = 5 - 1 = 4$$

$$\text{Slope } m = \Delta y / \Delta x = 4 / 4 = 1$$

Here total steps will be = 4

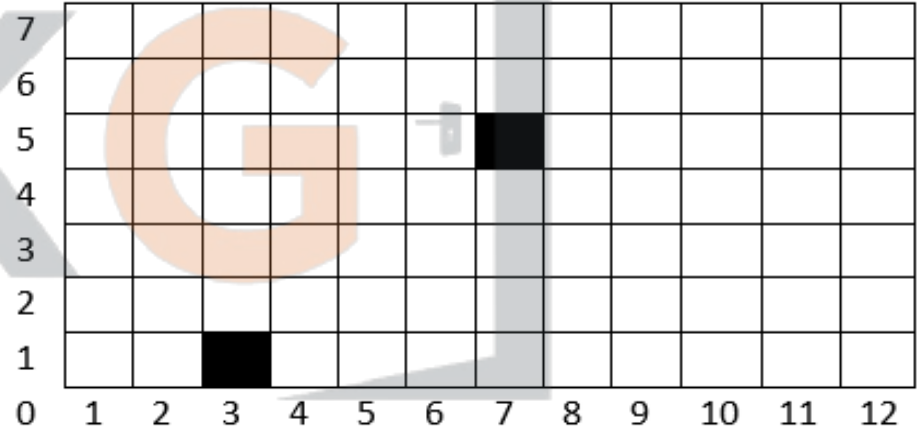
Now to draw the line the increment in

$$x_{\text{inc}} = \Delta x / \text{total steps} = 4/4 = 1$$

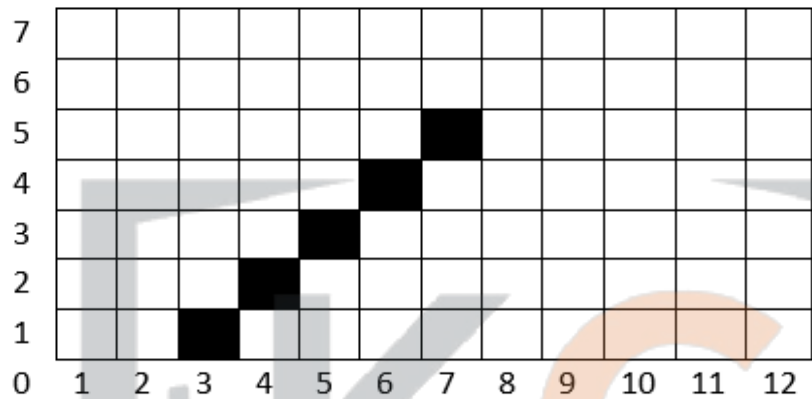
$$y_{\text{inc}} = \Delta y / \text{total steps} = 4/4 = 1$$

Thus, we get the corresponding values for x and y in each steps as:

x	y
3	1
4	2
5	3
6	4
7	5



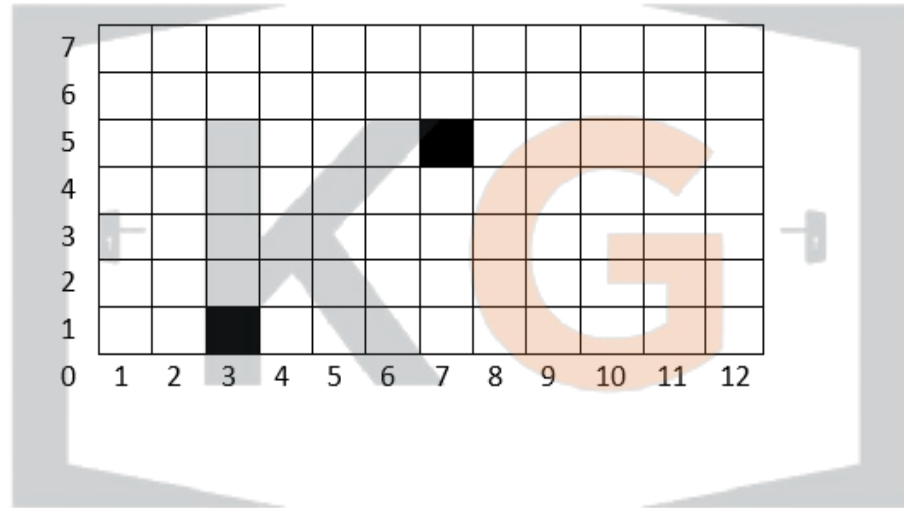
Thus, the line drawn will be like:



Special Case to note (Negative Increment)

Consider the points (x_1, y_1) and (x_2, y_2) to be: $(7, 5)$ and $(3, 1)$ i.e we reversed starting and end points of the previous example.

The initial figure for the required line will be –



Here

$$\Delta x = x_2 - x_1 = 3 - 7 = -4$$

$$\Delta y = y_2 - y_1 = 1 - 5 = -4$$

$$\text{Slope } m = \Delta y / \Delta x = -4 / -4 = 1$$

Here total steps will be = 4

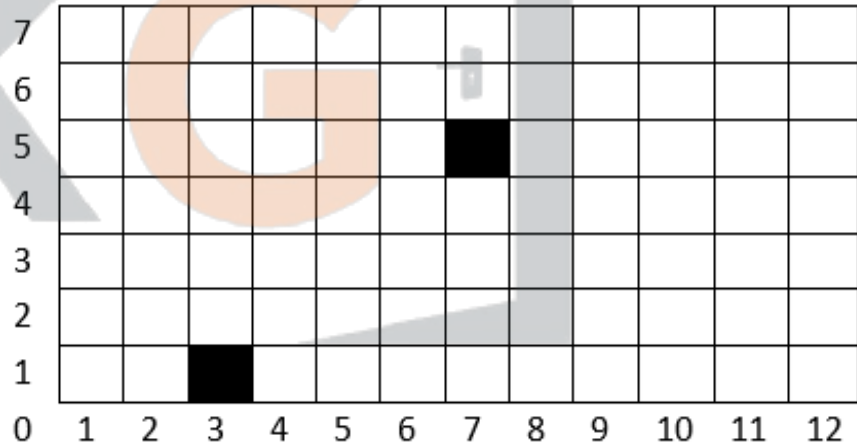
Now to draw the line the increment in

$$x_{\text{inc}} = \Delta x / \text{total steps} = -4 / 4 = -1$$

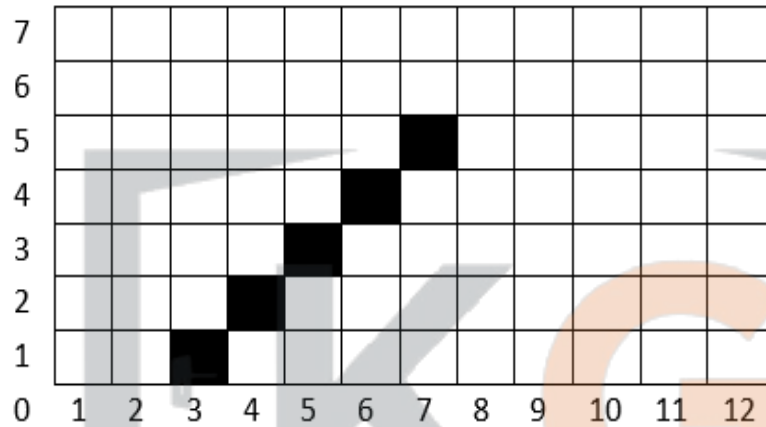
$$y_{\text{inc}} = \Delta y / \text{total steps} = -4 / 4 = -1$$

In this case the DDA works perfectly fine the only change is that we will now have to subtract values in order to draw line.

x	y
7	5
6	4
5	3
4	2
3	1



Thus, the diagram will remain same



Algorithm:

```
void lineDDA (int x1, int y1, int x2, int y2)
```

```
{
```

```
    int dx = x2 - x1, dy = y2 - y1, steps, k;
```

```
    float xincrement, yIncrement ;
```

```
    if (abs (dx) > abs (dy)
```

```
        steps = abs (dx) ;
```

```
    else
```

```
        steps = abs (dy);
```

```
    xincrement = dx / (float) steps;
```

```
    yincrement = dy / (float) steps
```

```
    for (k=0; k<steps; k++) {
```

```
        x += xincrement;
```

```
        y += yincrement;
```

```
        setPixel (ROUND(x), ROUND(y));
```

```
    }
```

```
}
```


Disadvantages of DDA

1. **Floating-point increment** can cause the calculated pixel positions to drift away from the true line path for long line segments as we have to round off values to determine which pixels to select.
2. The rounding operations and floating-point arithmetic in DDA are time-consuming.



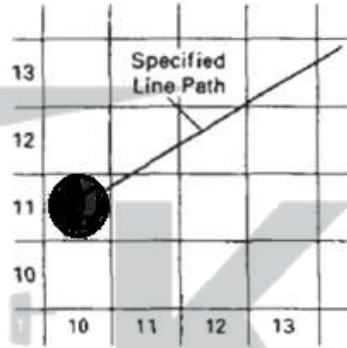
Bresenham's Line Algorithm

- An accurate and efficient raster line-generating algorithm was developed by Bresenham.
- It scan converts lines using only **incremental integer calculations (No floating point increment used)**.
- Thus, the floating-point increment problem by DDA is dealt by Bresenham.



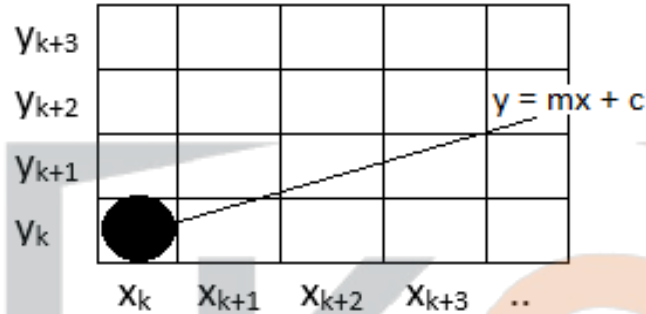
Basic Idea

Consider the figure below:

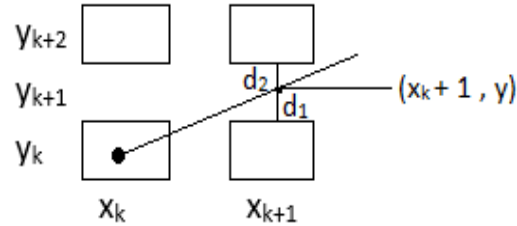


- In each step we need to decide which of two possible pixel positions is closer to the line path.
- In above figure; starting from the left endpoint, we need to determine at the next sample position whether to plot the pixel at position (11, 11) or the one at (11, 12).
- The question is answered with Bresenham's line algorithm by testing the sign of an integer parameter, whose value is proportional to the difference between the separations of the two-pixel positions from the actual line path.

For derivation purpose we consider the scan conversion process for lines with positive slope less than 1.



- Starting from the left endpoint (x_k, y_k) of a given line, we step to each successive column (x position) and plot the pixel whose scan-line y value is closest to the line path.
- Assuming we have determined that the pixel at (x_k, y_k) is to be displayed, we next need to decide which pixel to plot in column x_{k+1} .
- Our choices are the pixels at positions $(x_k + 1, y_k)$ and $(x_k + 1, y_k + 1)$.



- At sampling position x_{k+1} , we assume vertical pixel separations from the mathematical line path as d_1 and d_2 .
- We are assuming that the line instead of going through the pixel is actually passing through a gap and we have to now decide which pixel is closest to the line.
- We will calculate the pixel distance d_1 and d_2 , and whichever pixel is nearest will then be selected.

Derivation

Since we are working on a line with slope < 1 ;

Thus, we know that;

x_{next} will be $x_k + 1$ and y_{next} can be either y_k or $y_k + 1$

We know that the line equation is:

$$y = mx + c$$

Since we need to find the next value at $x_k + 1$, we can rewrite the above equation as:

$$y = m(x_k + 1) + c \dots (1)$$

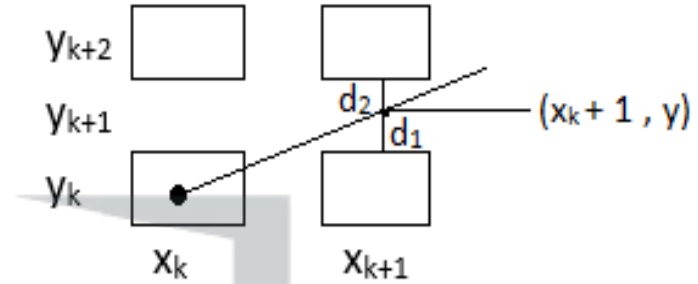
The actual point from which the distances d_1 and d_2 needs to be calculated is: $(x_k + 1, y)$

Thus we can conclude that,

$$d_1 = (y - y_k) \dots (2)$$

Substituting the value of y from eqn. 1 into eqn. 2 we get,

$$d_1 = m(x_k + 1) + c - y_k$$



and,

$$d_2 = (y_k + 1) - y \text{ (3)}$$

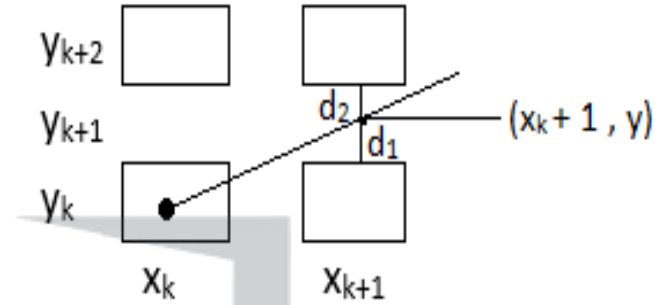
Substituting the value of y from eqn. 1 in eqn. 3 we get;

$$d_2 = (y_k + 1) - m (x_k + 1) - c$$

Now, calculating the difference between d_1 and d_2 , we get

$$d_1 - d_2 = [m (x_k + 1) + c - y_k] - [(y_k + 1) - m (x_k + 1) - c]$$

$$d_1 - d_2 = 2m (x_k + 1) - 2y_k + 2c - 1 \text{ (4)}$$



$$d_1 - d_2 = 2m (x_k + 1) - 2y_k + 2c - 1 \dots\dots (4)$$

Since there is m in equation (4) i.e. slope variable in the equation thus it will again lead to floating-point calculations and our aim is to convert it into an integer calculation.

We substitute $m = \Delta y / \Delta x$ and multiply the LHS and RHS of the equation by Δx .

$$\Delta x (d_1 - d_2) = \Delta x [2(\Delta y / \Delta x) (x_k + 1) - 2y_k + 2c - 1]$$

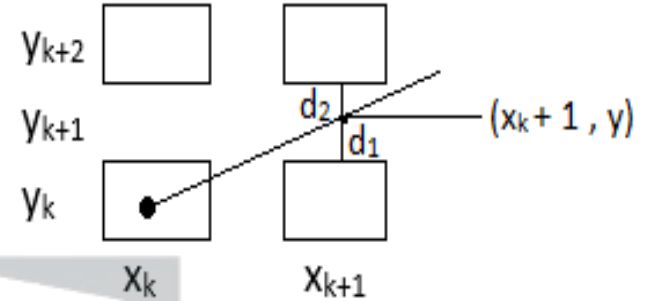
Simplifying this equation, we get,

$$\Delta x (d_1 - d_2) = 2 \Delta y x_k - 2 \Delta x y_k + \mathbf{2 \Delta y + 2 \Delta x c - \Delta x} \dots\dots (4.5)$$

Since the variables in bold red in above equation are nothing but constant values, we can ignore them.

We get the equation as:

$$\Delta x (d_1 - d_2) = 2 \Delta y x_k - 2 \Delta x y_k$$



The LHS of the above equation is a decision parameter (p_k) using which we will be taking a decision on which closest pixel do we need to select.

$$p_k = 2 \Delta y x_k - 2 \Delta x y_k \dots\dots (5)$$

- If the pixel at y_k is closer to the line path than the pixel at $y_k + 1$ (that is, $d_1 < d_2$), then decision parameter p_k is negative. In that case, we plot the lower pixel; otherwise, we plot the upper pixel.

To obtain the values of successive decision parameters using incremental integer calculations. At step $k + 1$, the decision parameter is evaluated as:

$$p_{k+1} = 2 \Delta y x_{k+1} - 2 \Delta x y_{\text{next}}$$

Subtracting Eq. 5 from the above equation we get,

$$p_{k+1} - p_k = 2 \Delta y (x_{k+1} - x_k) - 2 \Delta x (y_{\text{next}} - y_k)$$

$$p_{k+1} = p_k + 2 \Delta y (x_{k+1} - x_k) - 2 \Delta x (y_{\text{next}} - y_k)$$

$$\mathbf{p}_{k+1} = \mathbf{p}_k + 2 \Delta \mathbf{y} (\mathbf{x}_{k+1} - \mathbf{x}_k) - 2 \Delta \mathbf{x} (\mathbf{y}_{\text{next}} - \mathbf{y}_k)$$

Case 1: If $\mathbf{p}_k < 0$ (in this case $\mathbf{y}_{\text{next}} = \mathbf{y}_k$)

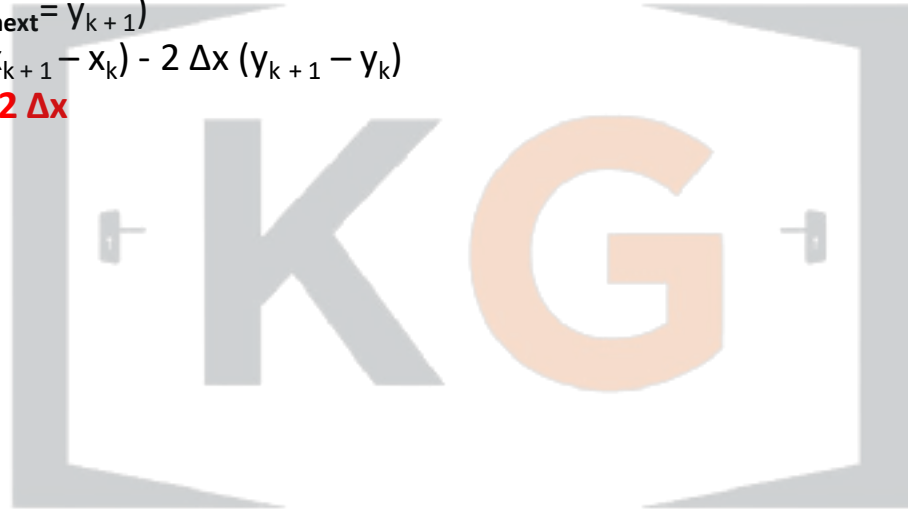
$$\mathbf{p}_{k+1} = \mathbf{p}_k + 2 \Delta \mathbf{y} (\mathbf{x}_{k+1} - \mathbf{x}_k) - 2 \Delta \mathbf{x} (\mathbf{y}_k - \mathbf{y}_k)$$

$$\mathbf{p}_{k+1} = \mathbf{p}_k + 2 \Delta \mathbf{y}$$

Case 2: if $\mathbf{p}_k > 1$ (in this case $\mathbf{y}_{\text{next}} = \mathbf{y}_{k+1}$)

$$\mathbf{p}_{k+1} = \mathbf{p}_k + 2 \Delta \mathbf{y} (\mathbf{x}_{k+1} - \mathbf{x}_k) - 2 \Delta \mathbf{x} (\mathbf{y}_{k+1} - \mathbf{y}_k)$$

$$\mathbf{p}_{k+1} = \mathbf{p}_k + 2 \Delta \mathbf{y} - 2 \Delta \mathbf{x}$$



Case 3: The first parameter, p_0 is evaluated from Eq. 3 at the starting pixel position (x_0, y_0) and with m evaluated as $m = \Delta y / \Delta x$

Since we have the equation;

$$p_k = 2 \Delta y x_k - 2 \Delta x y_k + 2 \Delta y + 2 \Delta x c - \Delta x$$

We can write c as;

$$c = y_0 - (\Delta y / \Delta x) x_0 \quad (\text{From } y_0 = mx_0 + c \text{ and } m = \Delta y / \Delta x)$$

Putting c in equation above we get;

$$p_0 = 2 \Delta y x_0 - 2 \Delta x y_0 + 2 \Delta y + 2 \Delta x [y_0 - (\Delta y / \Delta x) x_0] - \Delta x$$

$$p_0 = \mathbf{2 \Delta y x_0 - 2 \Delta x y_0} + 2 \Delta y + \mathbf{2 \Delta x y_0 - 2 \Delta y x_0} - \Delta x$$

The bold values gets cancelled out;

$$p_0 = \mathbf{2 \Delta y - \Delta x}$$

Bresenham's Line Drawing Algorithm(x_1, y_1, x_2, y_2)

```
{
```

```
    x = x1;
```

```
    y = y1;
```

```
    dx = x2 - x1;
```

```
    dy = y2 - y1;
```

```
    p = 2*dy - dx;
```

```
    while(x < x2)
```

```
    {
```

```
        putpixel(x, y)
```

```
        x++
```

```
        if(p < 0)
```

```
        {
```

```
            p = p + 2*dy
```

```
        }
```

```
        else
```

```
        {
```

```
            p = p + 2*dy - 2*dx;
```

```
            y++
```

```
        }
```

```
    }
```

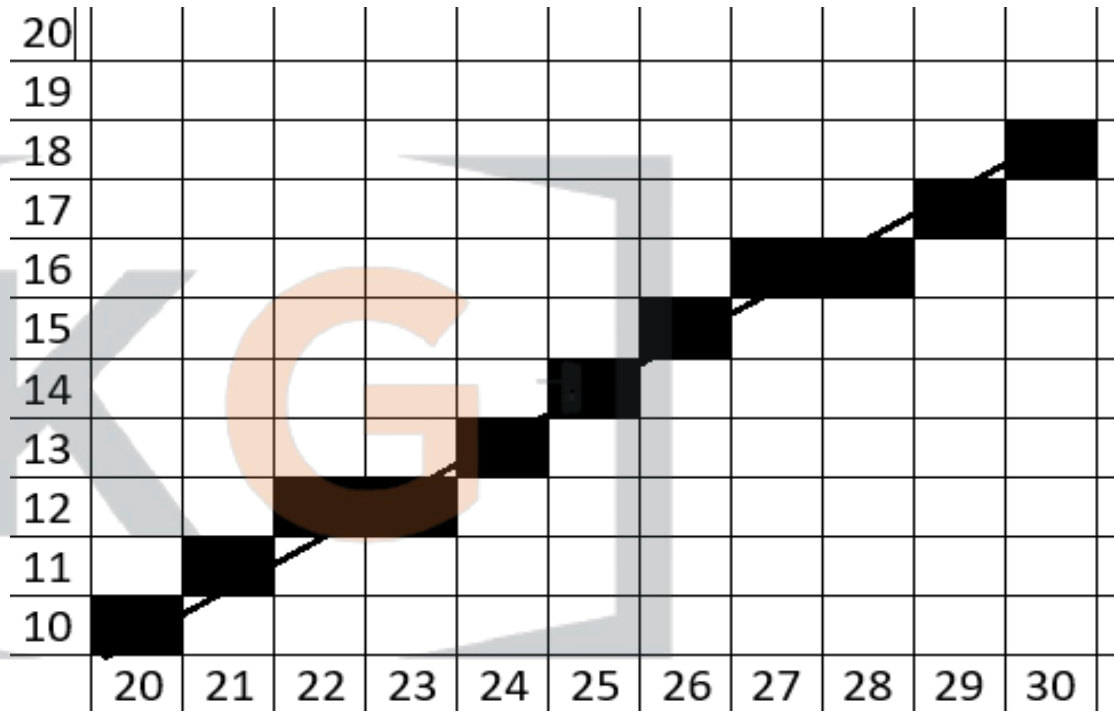
```
}
```

Example: Digitize the line with endpoints (20, 10) and (30, 18).

x	y	p
20	10	

```
Bresenham's Line Drawing Algorithm( $x_1, y_1, x_2, y_2$ )
{
     $x = x_1;$ 
     $y = y_1;$ 
     $d_x = x_2 - x_1;$ 
     $d_y = y_2 - y_1;$ 
     $p = 2 * d_y - d_x;$ 
    while( $x < x_2$ )
    {
        putpixel( $x, y$ )
         $X++$ 
        if( $p < 0$ )
        {
             $p = p + 2 * d_y;$ 
        }
        else
        {
             $p = p + 2 * d_y - d_x;$ 
             $Y++$ 
        }
    }
}
```

x	y	p
20	10	6
21	11	2
22	12	-2
23	12	14
24	13	10
25	14	6
26	15	2
27	16	-2
28	16	14
29	17	10
30	18	6



Q. Consider the midpoint (or Bresenham) algorithm for rasterizing lines given below: (NET-DEC-2018)

```
(1)   Input  $(x_1, y_1)$  and  $(x_2, y_2)$ 
(2)    $y = y_1$ 
(3)    $d = f(x_1 + 1, y_1 + 1/2)$  //f is the implicit form of a line
(4)   for  $x = x_1$  to  $x_2$ 
(5)   do
(6)     plot(x, y)
(7)     if  $d < 0$ 
(8)     then
(9)        $y = y + 1$ 
(10)     $d = d + (y_1 - y_2) + (x_2 - x_1)$ 
(11)   else
(12)     $d = d + (y_1 - y_2)$ 
(13)   end
(14) end
```

Which statements are true?

P: For a line with slope $m > 1$, we should change the outer line in line (4) to be over y

Q: Lines (10) and (12) update the decision variable d through an incremental evaluation of the line equation f

R: The algorithm fails if d is over 0

Choose the correct answer from the code given below:

a) P only

b) P and Q only

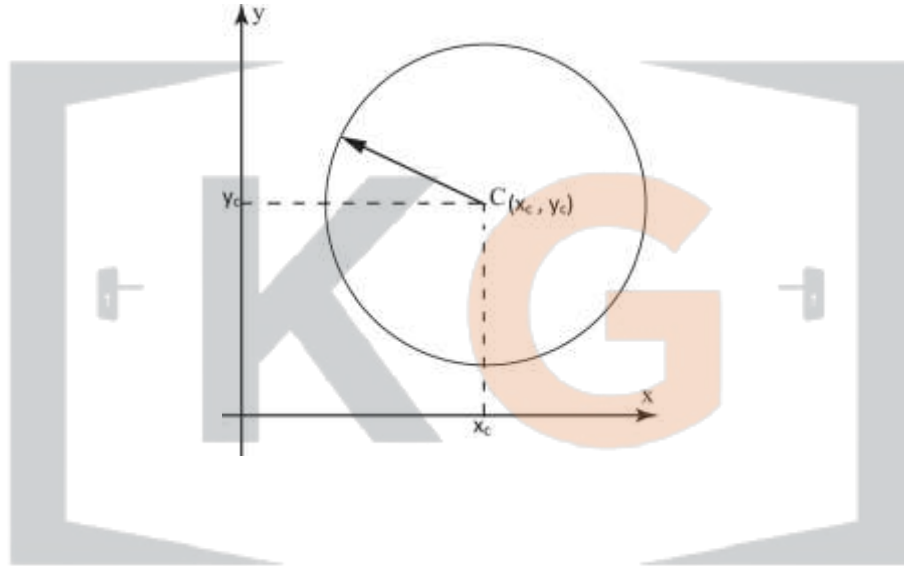
c) Q and R only

d) P, Q and R

Ans: B

CIRCLE-GENERATING ALGORITHMS

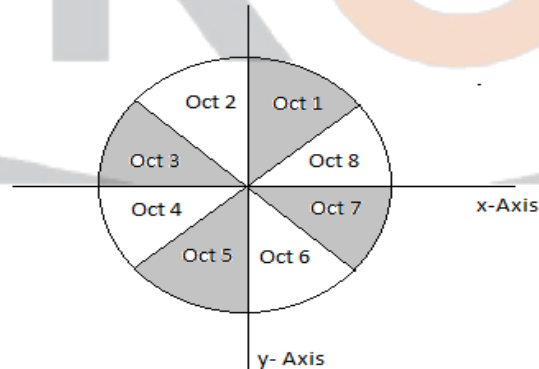
A circle is defined as the set of points that are all at a given distance r from a center position (x_c, y_c) .



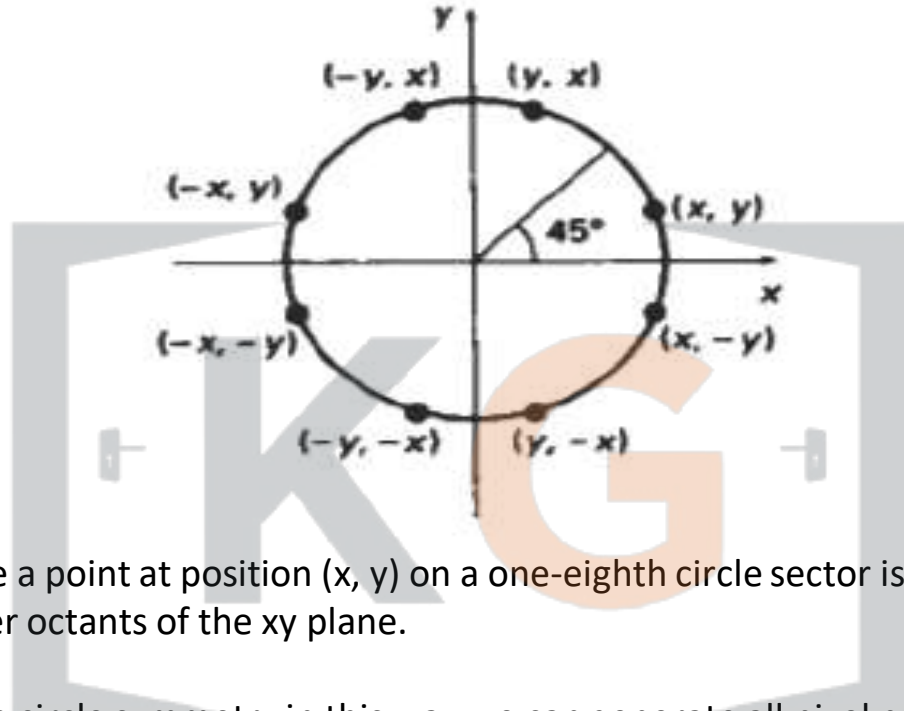
The two primary methods of drawing circle require a huge computational overhead. Methods such as:

- Pythagorean theorem in Cartesian coordinates:
 - $(x - x_c)^2 + (y - y_c)^2 = r^2$
- Expressing the circle equation in parametric polar form
 - $x = x_c + r \cos \theta$
 - $y = y_c + r \sin \theta$

The above two methods requires a lot of computation. The computation can be reduced by considering the symmetry of the circles i.e. the shape of the circle is similar in each quadrant or octant.



- Circle sections in adjacent octants within one quadrant are symmetric with respect to the 45° line dividing the two octants.
- Shown in Figure below



- In above figure we have a point at position (x, y) on a one-eighth circle sector is mapped into the seven circle points in the other octants of the xy plane.
- Taking advantage of the circle symmetry in this way we can generate all pixel positions around a circle.
- Bresenham's line algorithm for raster displays is adapted to circle generation by setting up decision parameters for finding the closest pixel to the circumference at each sampling step.

Aliased Circle Drawing Algorithms

1. Direct or Polynomial Method
2. Digital Differential Analyzer (DDA) Method
3. Midpoint Circle Algorithm
4. Bresenham's Circle Algorithm

Midpoint Circle Algorithm

Steps to take:

- Initially for a given radius r and screen center position (x_c, y_c) , we first set up our algorithm to calculate pixel positions around a circle path centered at the coordinate origin $(0,0)$.
- Then each calculated position (x, y) is moved to its proper screen position by adding x_c to x and y_c to y .
- In the first quadrant we can take unit steps in the positive x direction over this octant and use a decision parameter to determine which of the two possible y positions is closer to the circle path at each step.
- Positions in the other seven octants are then obtained by symmetry.

Derivation

Since we know that the circle function is;

$$f_{\text{circle}}(x, y) = x^2 + y^2 - r^2 \text{ (Since } x^2 + y^2 = r^2 \text{)}$$

Any point (x, y) on the boundary of the circle with radius r satisfies the equation

$$f_{\text{circle}}(x, y) = 0.$$

Now, two points to note here are –

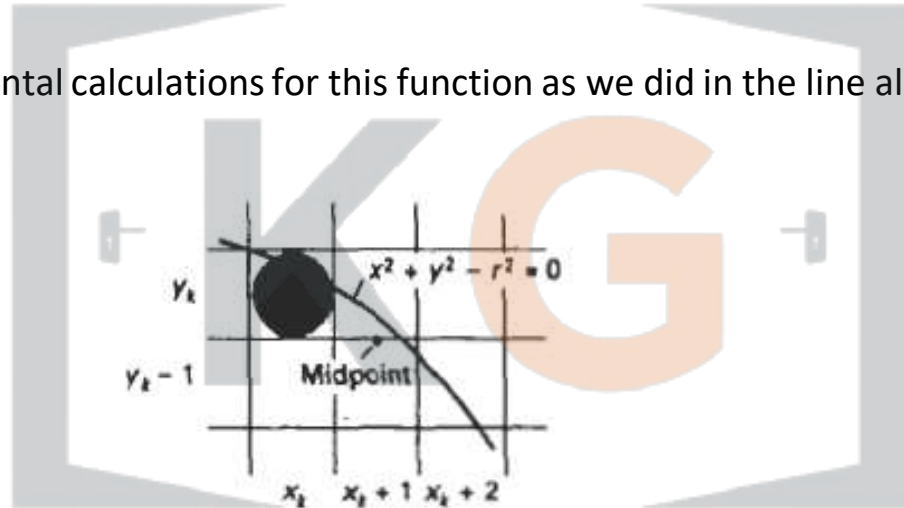
- If the point is in the interior of the circle, the circle function is negative.
- If the point is outside the circle, the circle function is positive.

Thus, the circle function is the decision parameter in the midpoint algorithm.

$$f_{\text{circle}}(x, y) \begin{cases} < 0, & \text{if } (x, y) \text{ is inside the circle boundary} \\ = 0, & \text{if } (x, y) \text{ is on the circle boundary} \\ > 0, & \text{if } (x, y) \text{ is outside the circle boundary} \end{cases}$$

Now we can set up incremental calculations for this function as we did in the line algorithm.

Consider the figure below



Assuming we have just plotted the pixel at (x_k, y_k) , we next need to determine whether the pixel at position $(x_k + 1, y_k)$ or the one at position $(x_k + 1, y_k - 1)$ is closer to the circle.

We have the two points $(x_k + 1, y_k)$ and $(x_k + 1, y_k - 1)$ and any one of them is going to be plotted.

Midpoint of these points will be:

$$(x_k + 1 + x_k + 1) / 2 \text{ and } (y_k + y_k - 1) / 2$$

and y

(Summing up and dividing by 2 both x coordinates
coordinates)

Midpoint:

$$(x_k + 1) \text{ and } (y_k - 1/2)$$

Our decision parameter (p_k) is the circle function evaluated at the midpoint between these two pixels:

$$p_k = f_{\text{circle}}(x_k + 1, y_k - 1/2)$$

Putting it in circle equation

$$p_k = (x_k + 1)^2 + (y_k - 1/2)^2 - r^2$$

- If $p_k < 0$, this midpoint is inside the circle and the pixel on scan line y_k is closer to the circle boundary.
- Otherwise, the midpoint is outside or on the circle boundary, and we select the pixel on scan line $y_k - 1$.

We obtain a recursive expression for the next decision parameter by evaluating the circle function at sampling position $x_k + 1 + 1 = x_k + 2$:

$$p_{k+1} = f_{\text{circle}}(x_k + 1 + 1, y_{\text{next}} - 1/2) = (x_k + 1 + 1)^2 + (y_{\text{next}} - 1/2)^2 - r^2$$

Subtracting p_k from p_{k+1}

$$p_{k+1} - p_k = [(x_k + 1 + 1)^2 + (y_{\text{next}} - 1/2)^2 - r^2] - [(x_k + 1)^2 + (y_k - 1/2)^2 - r^2]$$
$$p_{k+1} = p_k + 2x_k + 3 + y_{\text{next}}^2 - y_k^2 - y_{\text{next}} + y_k$$

Here y_{next} can either be y_k or y_{k-1} , depending on the sign of p_k .

Case 1:

If $p_k < 0$ thus we know $y_{\text{next}} = y_k$

Thus

$$p_{k+1} = p_k + 2x_k + 3 + y_k^2 - y_k^2 - y_k + y_k$$

$$p_{k+1} = p_k + 2x_k + 3$$

Case 2: If $p_k > 0$ thus we know $y_{\text{next}} = y_k - 1$

$$p_{k+1} = p_k + 2x_k + 3 + (y_k - 1)^2 - y_k^2 - y_{k-1} + y_k$$

$$p_{k+1} = p_k + 2x_k - 2y_k + 5$$

Case 3: Initial decision parameter (p_0)

Initial point to be plotted will be $(x_0, y_0) = (0, r)$

Since we have the equation as;

$$p_k = (x_k + 1)^2 + (y_k - 1/2)^2 - r^2$$

Putting $x_k = 0$ and $y_k = r$

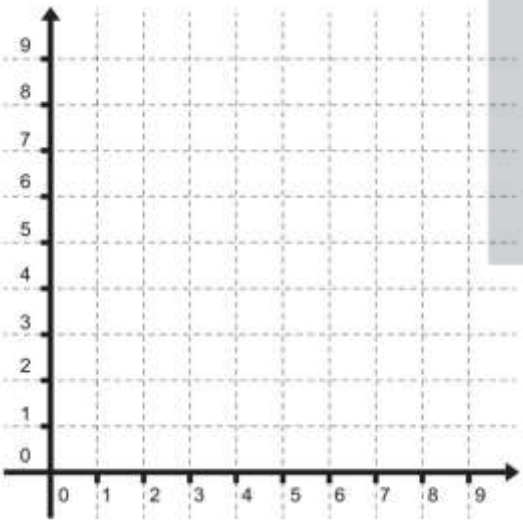
$$p_0 = 1 + (r - 1/2)^2 - r^2$$

$$p_0 = 1 + r^2 + (1/4) - 2r(1/2) - r^2$$

$$p_0 = (5/4) - r$$

Example: Given a circle radius $r = 10$, we demonstrate the midpoint circle algorithm by determining positions along the circle octant in the first quadrant from $x = 0$ to $x = y$. The initial value of the decision parameter is

x	y	p



Midpoint Circle Algorithm (x_c, y_c, r)

```
{
    x = 0;
    y = r;
    p = 5/4 - r;
    while(x < y)
    {
        putpixel(x, y)
        X++
        if(p < 0)
        {
            p = p + 2*x + 3
        }
        else
        {
            p = p + 2*(x-y)
            Y--
        }
    }
}
```

Example: Given a circle radius $r = 10$, we demonstrate the midpoint circle algorithm by determining positions along the circle octant in the first quadrant from $x = 0$ to $x = y$. The initial value of the decision parameter is

$$p_0 = 1 - r \text{ (rounded off 5/4 to 1)} = 1 - (10) = -9$$

The iterations are as follows

1. $p_0 < 0$: ($x_1 = 1, y_1 = 10$) and $p_1 = p_0 + 2x_0 + 3 = -9 + 0 + 3 = -6$

Similarly the other iterations and resulting values are:

Iteration	x	y	p
1	1	10	-9
2	2	10	-6
3	3	10	-1
4	4	9	6
5	5	9	-3
6	6	8	8
7	7	7	5

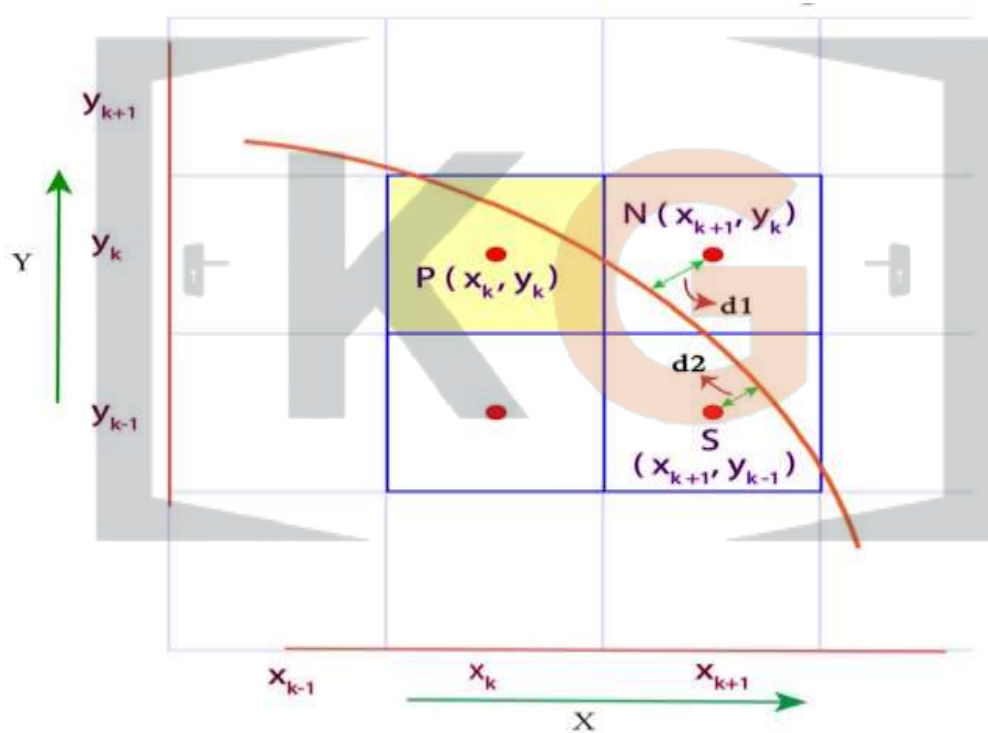
The octant will look somewhat like this:

10								
9								
8								
7								
	1	2	3	4	5	6	7	

The points on the other octants will be plotted using the symmetry of the circle.

Bresenham's circle drawing algorithm

- Given the centre point and radius of circle, Bresenham Circle Drawing Algorithm attempts to generate the points of one octant.



Function of Circle at N

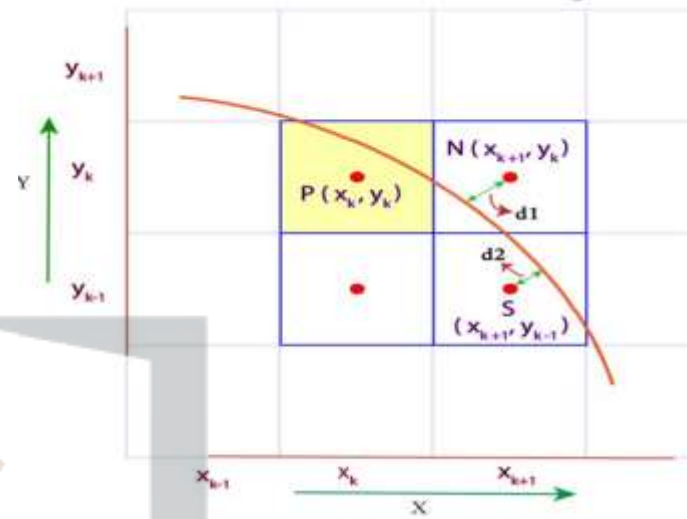
$$F(N) = (x_{k+1})^2 + (y_k)^2 - r^2 \quad (\text{Positive})$$

Here the value of $F(N)$ will be positive because N is out-side the circle that makes $(x_{k+1})^2 + (y_k)^2$ Greater than r^2

Function of Circle at S

$$F(S) = (x_{k+1})^2 + (y_{k-1})^2 - r^2 \quad (\text{Negative})$$

Here the value of $F(S)$ will be Negative because S is in-side the circle that makes $(x_{k+1})^2 + (y_{k-1})^2$ Less than r^2



Now we need a decision parameter which help us decide the next pixel

Say D_k

$$\text{And, } D_k = F(N) + F(S)$$

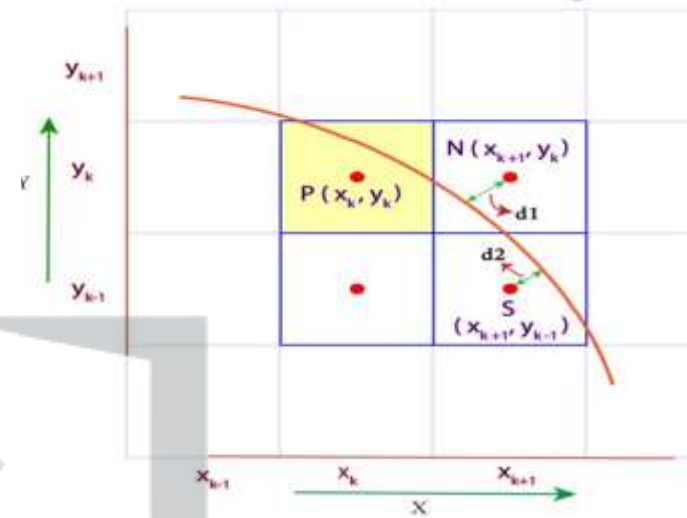
Here either we will get the positive or negative value of D_k

So if $D_k < 0$

that means the negative $F(S)$ is bigger then the positive $F(N)$, that implies Point N is closer to the circle than point S. So we will select pixel N as our next pixel.

and if $D_k > 0$

that means positive $F(N)$ is bigger and S is more closer as $F(S)$ is smaller. So we will Select S as our next pixel.



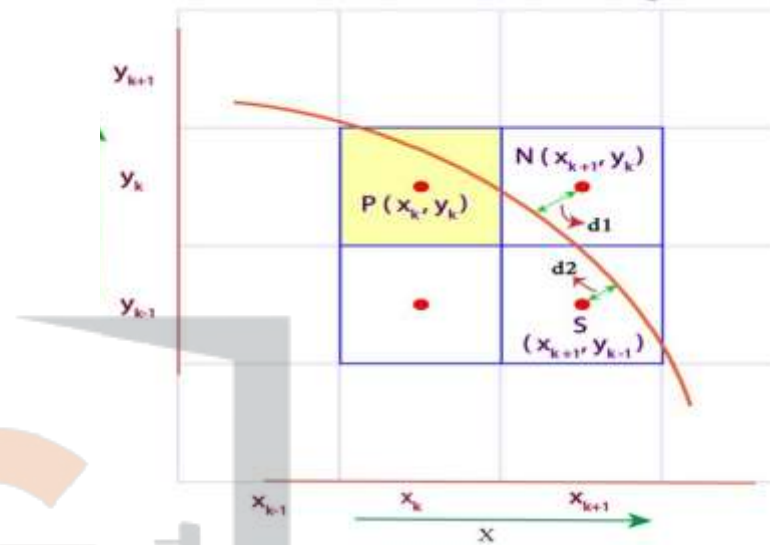
Now let's find D_k

$$D_k = (x_{k+1})^2 + (y_k)^2 - r^2 + (x_{k+1})^2 + (y_{k-1})^2 - r^2$$

(replacing x_{k+1} with $x_k + 1$ and y_{k-1} with $y_k - 1$)

$$= (x_k + 1)^2 + (y_k)^2 - r^2 + (x_k + 1)^2 + (y_k - 1)^2 - r^2$$

$$= 2(x_k + 1)^2 + (y_k)^2 + (y_k - 1)^2 - 2r^2 \quad \text{----- (i)}$$



Now let's find D_{k+1}

(Replacing every k with $k+1$)

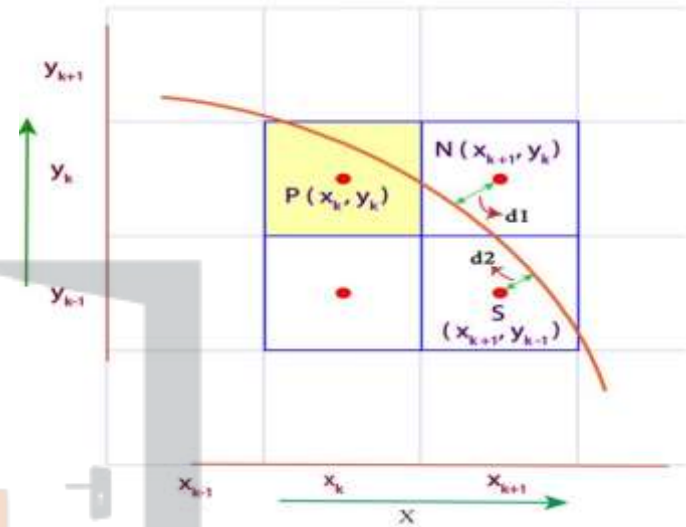
$$D_{k+1} = 2(x_{k+1} + 1)^2 + (y_{k+1})^2 + (y_{k+1} - 1)^2 - 2r^2$$

$$= 2(x_{k+1} + 1)^2 + (y_{k+1})^2 + (y_{k+1} - 1)^2 - 2r^2$$

(Replacing x_{k+1} with $x_k + 1$ but now we can't replace y_{k+1} because we don't know the exact value of y_k)

$$= 2(x_k + 1 + 1)^2 + (y_{k+1})^2 + (y_{k+1} - 1)^2 - 2r^2$$

$$= 2(x_k + 2)^2 + (y_{k+1})^2 + (y_{k+1} - 1)^2 - 2r^2 \quad \text{----- (ii)}$$



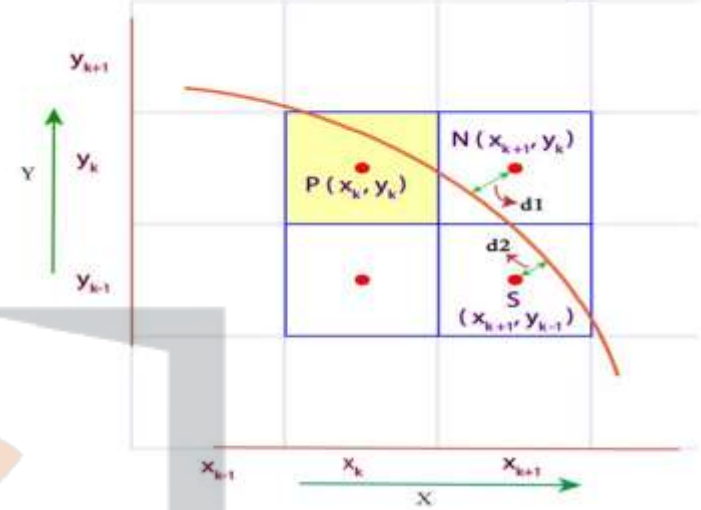
We need to find

$$\begin{aligned} D_{k+1} - D_k &= (ii) - (i) \\ &= 2(x_k+2)^2 + (y_{k+1})^2 + (y_{k+1}-1)^2 - 2r^2 \\ &\quad - [2(x_k+1)^2 + (y_k)^2 + (y_k-1)^2 - 2r^2] \end{aligned}$$

$$\begin{aligned} &= 2(x_k)^2 + 8x_k + 8 + (y_{k+1})^2 + (y_{k+1})^2 - 2y_{k+1} + 1 - 2r^2 \\ &\quad - 2x_k - 4x_k - 2 - (y_k)^2 - (y_k)^2 + 2y_k - 1 + 2r^2 \end{aligned}$$

$$= 4x_k + 2(y_{k+1})^2 - 2y_{k+1} - 2(y_k)^2 - 2y_k + 6$$

$$D_{k+1} = D_k + 4x_k + 2(y_{k+1})^2 - 2y_{k+1} - 2(y_k)^2 - 2y_k + 6 \quad \text{----- (iii)}$$



If ($D_k < 0$) then we will choose N point as discussed.

i.e. (x_{k+1}, y_k)

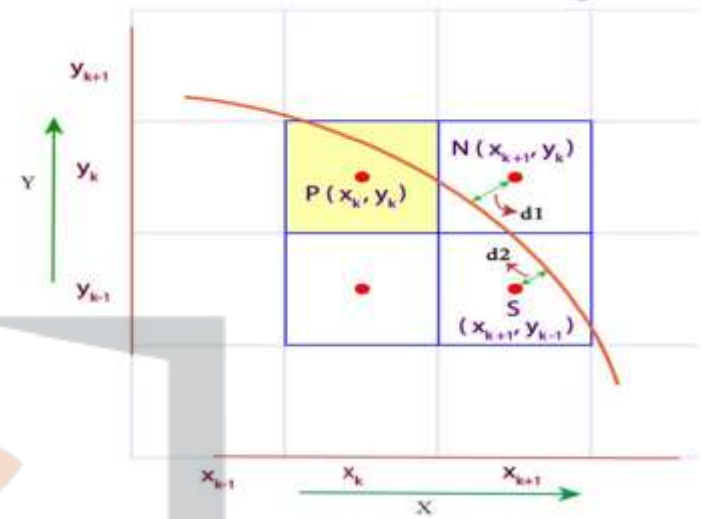
that means our next x coordinate is x_{k+1} and next y coordinate

is y_k i.e. $y_{k+1} = y_k$, putting y_k in (iii) in replace of y_{k+1}

now,

$$D_{k+1} = D_k + 4x_k + 2(y_k)^2 - 2y_k - 2(y_k)^2 - 2y_k + 6$$

$$= D_k + 4x_k + 6$$



If ($D_k > 0$) then we will choose S point.

i.e. (x_{k+1}, y_{k-1})

that means our next x coordinate is x_{k+1} and

next y coordinate is y_k i.e. $y_{k+1} = y_{k-1}$ putting y_{k-1} in (iii) in replace

of y_{k+1}

now,

$$D_{k+1} = D_k + 4x_k + 2(y_{k-1})^2 - 2y_{k-1} - 2(y_k)^2 - 2y_k + 6$$

Now we know

$$y_{k-1} = y_k - 1$$

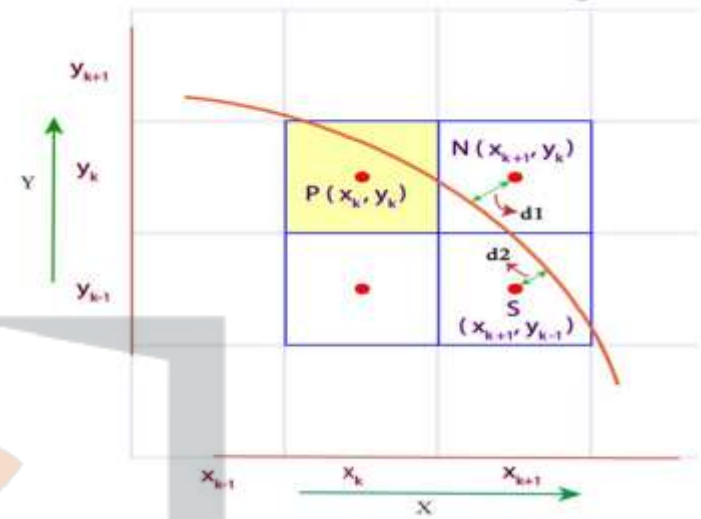
therefore,

$$D_{k+1} = D_k + 4x_k + 2(y_k - 1)^2 - 2(y_k - 1) - 2(y_k)^2 - 2y_k + 6$$

$$= D_k + 4x_k + 2(y_k)^2 + 2 - 4y_k - 2y_k + 2 - 2(y_k)^2 - 2y_k + 6$$

$$= D_k + 4x_k - 4y_k + 10$$

$$= D_k + 4(x_k - y_k) + 10$$



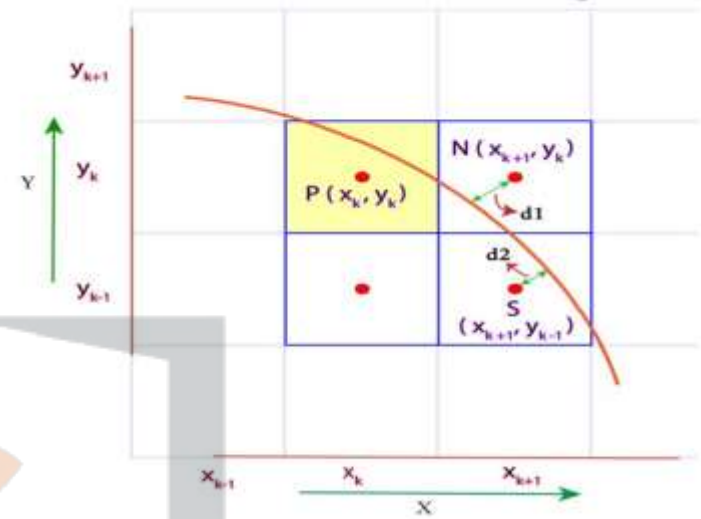
Now to find initial decision parameter means starting point
that is $(0, r)$, value of y is r
Putting $(0, r)$ in (i)

$$D_k = 2(x_k + 1)^2 + (y_k)^2 + (y_k - 1)^2 - 2r^2$$

$$D_0 = 2(0 + 1)^2 + r^2 + (r - 1)^2 - 2r^2$$

$$= 2 + r^2 + r^2 + 1 - 2r - 2r^2$$

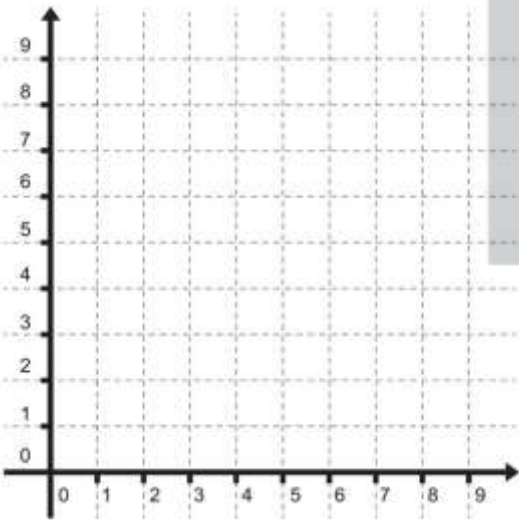
$$= 3 - 2r$$



Example: Given a circle radius $r = 10$, we demonstrate the midpoint circle Bresenham's circle drawing algorithm (x_c, y_c, r) algorithm by determining positions along the circle octant in the first quadrant { from $x = 0$ to $x = y$. The initial value of the decision parameter is

x	y	p

```
x = 0;
y = r;
d = 3 - 2r;
while(x < y)
{
    putpixel(x, y)
    if(d < 0)
    {
        d = d + 4*x + 6;
        X++;
    }
    else
    {
        d = d + 4*(x-y) + 10;
        Y--;
    }
}
```



Q. Consider the midpoint (or Bresenham) algorithm for rasterizing lines given below :
(UGC NET PAPER-2018)

(1) Input (x_1, y_1) and (x_2, y_2)

(2) $y = y_1$

(3) $d = f(x_1 + 1, y_1 + 1/2)$ // f is the implicit form of a line

(4) for $x = x_1$ to x_2

(5) do

(6) plot(x, y)

(7) if ($d < 0$)

(8) then

(9) $y = y + 1$

(10) $d = d + (y_1 - y_2) + (x_2 - x_1)$

(11) else

(12) $d = d + (y_1 - y_2)$

(13) end

(14) end



P: For a line with slope $m > 1$, we should change the outer loop in line (4) to be over y

Q: Lines (10) and (12) update the decision variable d through an incremental evaluation of the line equation f .

R: The algorithm fails if d is ever 0.

Choose the correct answer from the code given below:

- (a) P only
- (b) Q and R only
- (c) P and Q only
- (d) P, Q and R

Ans: 3

Q. Which of the statements given below are correct?
The midpoint (or Bresenham) algorithm for rasterizing lines is optimized relative to DDA algorithm in that **(UGC NET PAPER-2021)**

- (a) it avoids round-off operations.
- (b) it is incremental.
- (c) it uses only integer arithmetic.
- (d) all straight lines can be displayed as straight (exact).

Choose the correct answer from the options given below:

(a) (c) and (d) only

Two-Dimensional Geometric Transformations

- Changes in orientation, size, and shape are accomplished with geometric transformations that alter the coordinate descriptions of objects.

Types of Transformations:

- Translation
- Rotation
- Scaling
- Reflection
- Shearing



- Representation of a Point (x, y) in Matrix Form:

$$2 \times 1 \text{ Matrix} = \begin{bmatrix} x \\ y \end{bmatrix}$$

- General Transformation Problem can be represented as:

$$[P'] = [T] [P]$$

- Here, Matrix P' is the new transformed matrix, T is a transformation matrix and P is the original matrix.

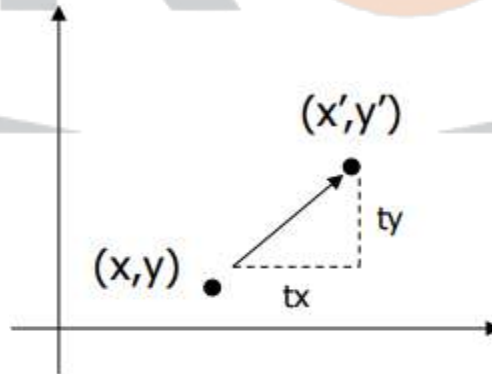
Translation

- A translation is applied to an object by repositioning it along a straight-line path from one coordinate location to another.
- We translate a two-dimensional point by adding translation distances, t_x and t_y , to the original coordinate position (x, y) to move the point to a new position (x', y') . Where,

$$x' = x + t_x$$

$$y' = y + t_y$$

- The translation distance pair (t_x, t_y) is called a **translation vector** or **shift vector**.



- It be represented in the matrix equation using column vectors as:

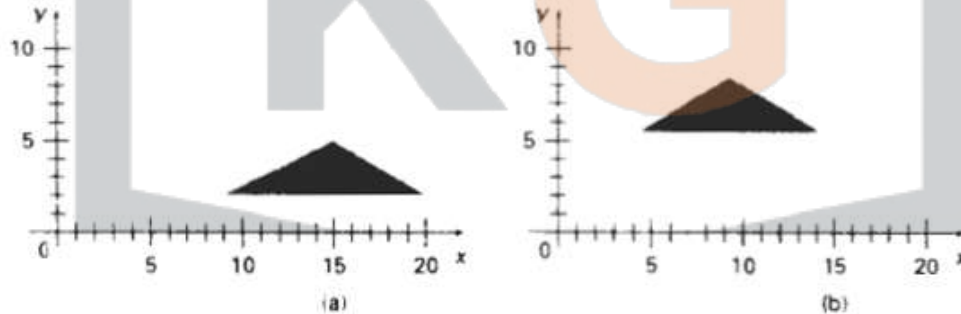
$$\mathbf{P}' = \begin{bmatrix} t_x \\ t_y \end{bmatrix} + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

- The two-dimensional translation equations in the matrix form can be rewritten as:

$$\mathbf{P}' = \mathbf{P} + \mathbf{T}$$

- Translation is a **rigid-body transformation** that moves objects without deformation.

Example:



- We cannot directly represent translation as a **matrix multiplication** as we did in all the previous transformations.

Rotation

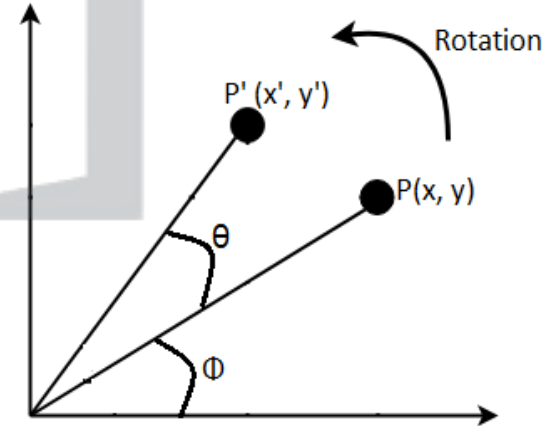
- Rotation is a process of rotating an object with respect to an angle in a two or a three dimensional plane as shown below.

Here, the initial coordinates of the point P is (x, y)

Initial angle of P with respect to origin is Φ

The Rotation Angle is θ

The new coordinates of P after rotation is (x', y')



$$op' = op = r$$

$$\cos \phi = x/r$$

$$x = r \cos \phi \dots \dots \dots 1$$

$$\sin \phi = y/r$$

$$y = r \sin \phi \dots \dots \dots 1$$

$$\cos (\phi + \theta) = x'/r$$

$$x' = r \cos (\phi + \theta)$$

$$x' = r \cos \phi \cos \theta - r \sin \phi \sin \theta$$

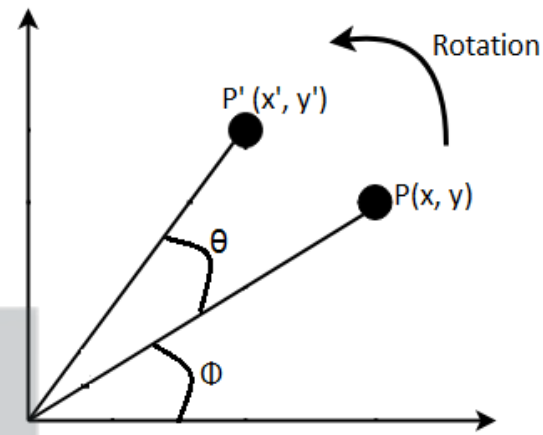
$$x' = x \cos \theta - y \sin \theta \dots \dots \dots \text{using 1}$$

$$\sin (\phi + \theta) = y'/r$$

$$y' = r \sin (\phi + \theta)$$

$$y' = r \cos \phi \sin \theta + r \cos \theta \sin \phi$$

$$y' = x \sin \theta + y \cos \theta \dots \dots \dots \text{using 2}$$



- Rotation is achieved by using the following rotation equations:

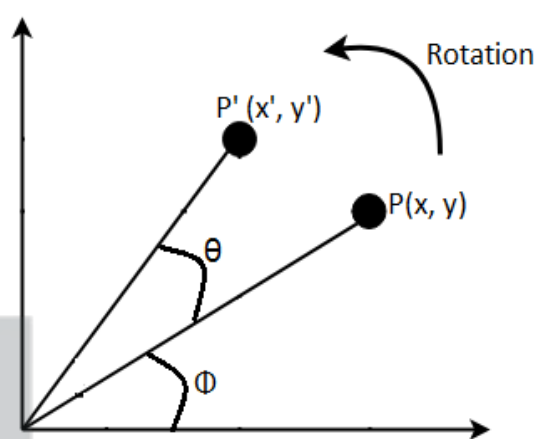
$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

- In Matrix form, the above rotation equations is represented as:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

- The transformation matrix: $T = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$



Example: Given a polygon with corner coordinates as A (0, 0), B (1, 0) and C (1, 1). Rotate the polygon by 90 degree anticlockwise direction and find out the new coordinates.

Solution: We rotate figure by rotating each vertex of it with the same rotation angle i.e. $\theta = 90^\circ$

For A (0, 0)

Let the new coordinates of corner A after rotation = (x', y') .

Applying the rotation equations, we get

$$x' = x \cos\theta - y \sin\theta = 0 \cos 90^\circ - 0 \sin 90^\circ = 0$$

$$y' = x \sin\theta + y \cos\theta = 0 \sin 90^\circ + 0 \cos 90^\circ = 0$$

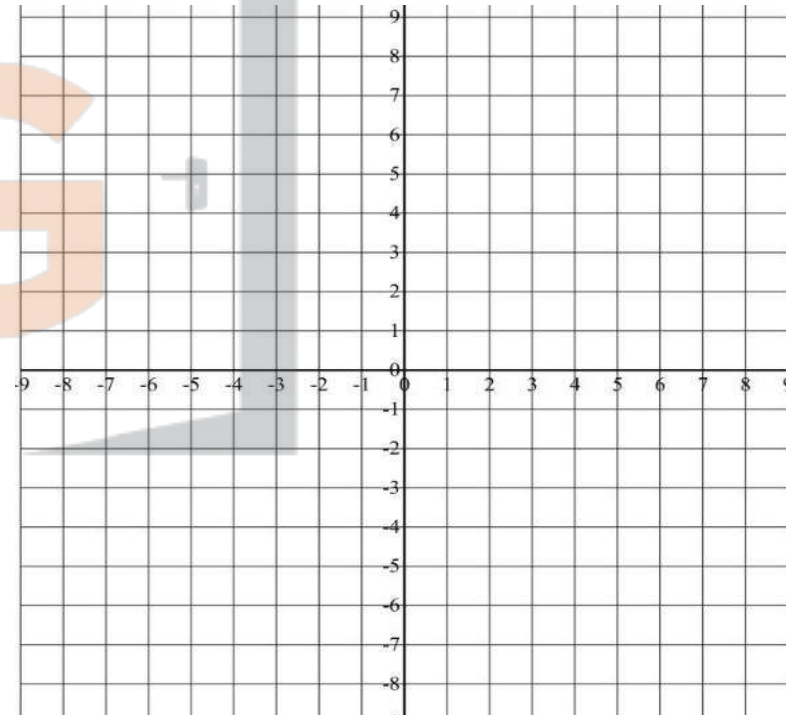
New coordinates of A after rotation are (0, 0).

For B (1, 0)

$$x' = 1 \cos 90^\circ - 0 \sin 90^\circ = 0$$

$$y' = 1 \sin 90^\circ + 0 \cos 90^\circ = 1 + 0 = 1$$

New coordinates of B after rotation are (0, 1).



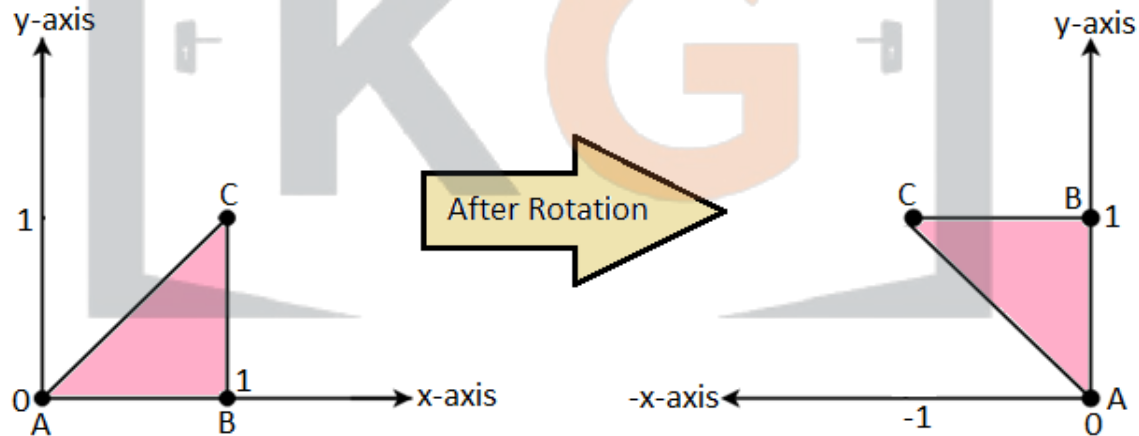
For C (1, 1)

$$x' = 1 \cos 90^\circ - 1 \sin 90^\circ = 0 - 1 = -1$$

$$y' = 1 \sin 90^\circ + 1 \cos 90^\circ = 1 + 0 = 1$$

New coordinates of C after rotation are (-1, 1).

Thus we get the new coordinates as: A (0, 0), B(0, 1), C(-1, 1).



Scaling

- A scaling transformation alters the size of an object.
- This operation can be carried out for polygons by multiplying the coordinate values (x, y) of each vertex by scaling factors s_x and s_y , to produce the transformed coordinates (x', y') such that:

$$x' = x \cdot s_x$$

$$y' = y \cdot s_y$$

- Scaling factor s_x , scales objects in the x direction, while s_y scales in the y direction.
- The above transformation equations can also be written in the matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Or,

$$P' = S.P$$

Points to Note

- Any positive numeric values can be assigned to the scaling factors s_x and s_y .
- Values less than 1 but greater than 0 reduce the size of objects and values greater than 1 produce an enlargement.
- Scaling factors with values less than 1 move objects closer to the coordinate origin whereas values greater than 1 move coordinate positions farther from the origin.
- Specifying a value of 1 for both s_x and s_y leaves the size of objects unchanged.
- When s_x and s_y are assigned the same value, a uniform scaling is produced that maintains relative object proportions.
- Unequal values for s_x and s_y result in a differential (non-uniform) scaling.

Example: Given coordinate points for a polygon are A (0, 3), B (3, 3), C (3, 0) and D (0, 0). Apply the scaling parameter 2 along the X axis and 3 along the Y axis and obtain the new coordinates for the polygon.

Solution:

Given corner coordinates = A (0, 3), B (3, 3), C (3, 0) and D (0, 0)

Scaling factor along X axis (s_x) = 2

Scaling factor along Y axis (s_y) = 3

For A (0, 3)

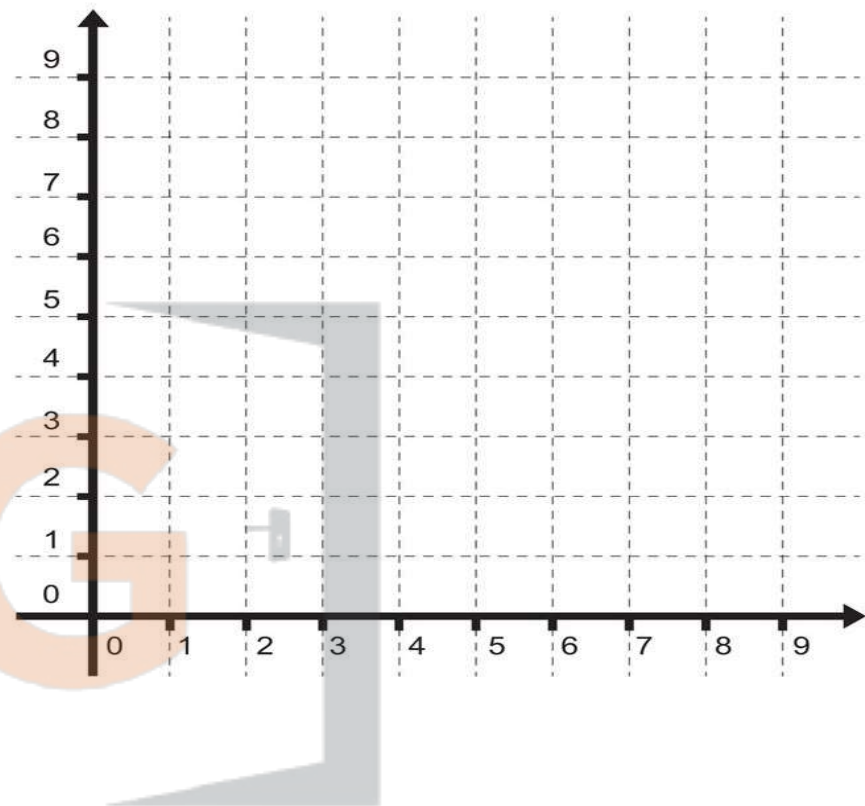
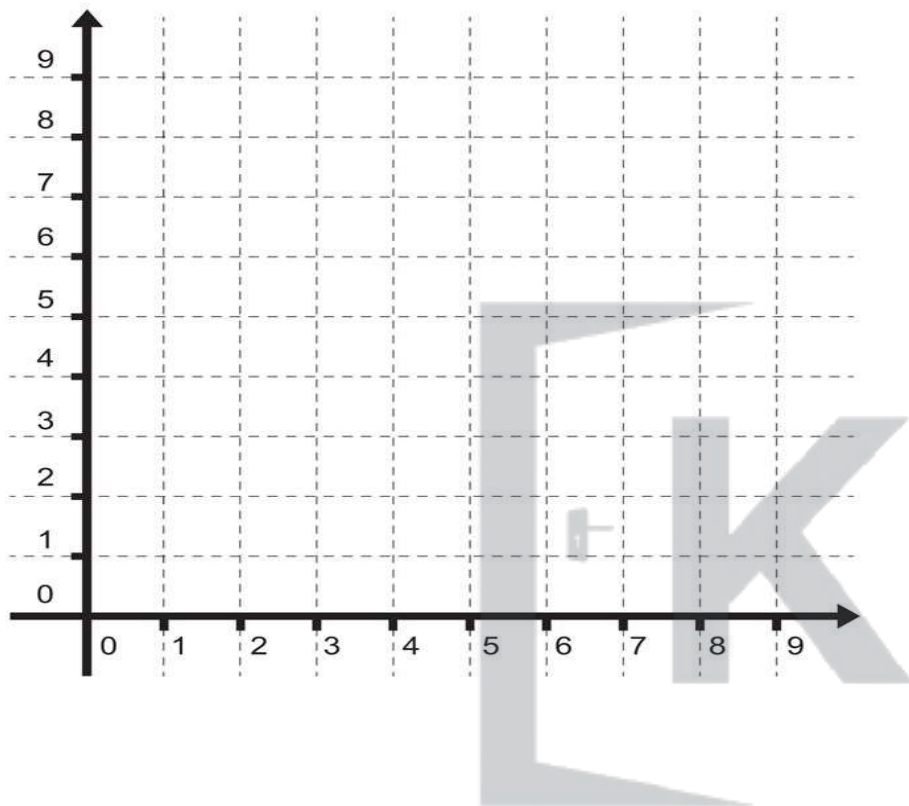
Let the new coordinates of point A after scaling be (x' y').

Applying the scaling equations, we get:

$$x' = x \times s_x = 0 \times 2 = 0$$

$$y' = y \times s_y = 3 \times 3 = 9$$

New coordinates of point A after scaling = (0, 9).



For B (3, 3)

$$x' = x \times s_x = 3 \times 2 = 6$$

$$y' = y \times s_y = 3 \times 3 = 9$$

New coordinates of point B after scaling = (6, 9).

For C (3, 0)

$$x' = x \times s_x = 3 \times 2 = 6$$

$$y' = y \times s_y = 0 \times 3 = 0$$

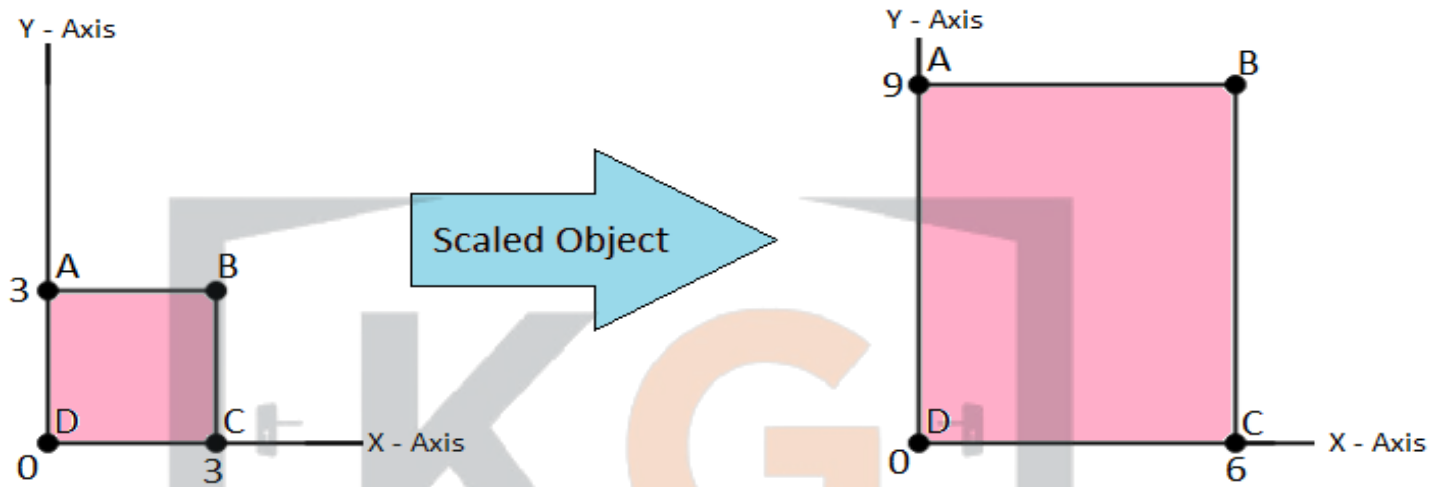
New coordinates of point C after scaling = (6, 0).

For D (0, 0)

$$x' = x \times s_x = 0 \times 2 = 0$$

$$y' = y \times s_y = 0 \times 3 = 0$$

New coordinates of point D after scaling = (0, 0).



Reflection

- If s_x and/or s_y (i.e. scaling factors) is less than zero (i.e. negative) then we get the reflection through xy-plane.

- **Reflection around x- axis**

Reflection around x-axis is achieved through if: $y' = -y$, in matrix form it is represented as:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Thus we get: $x' = x$ and $y' = -y$

- **Reflection around y- axis**

Reflection around y-axis is achieved through if: $x' = -x$, in matrix form it is represented as:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Thus we get: $x' = -x$ and $y' = y$

Example: A triangle with following coordinate points: A (3, 4), B (6, 4), C (5, 6). If we apply the reflection around the x-axis find the new coordinates of the object.

Solution:

For A (3, 4)

Let the new coordinates of the point A after reflection be (x', y')

Applying the reflection equations, we get

$$x' = x = 3$$

$$y' = -y = -4$$

New coordinates of A after reflection are (3, -4).

For B (6, 4)

$$x' = x = 6$$

$$y' = -y = -4$$

New coordinates of B after reflection are (6, -4).

Other transformations

Reflection:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

x-axis



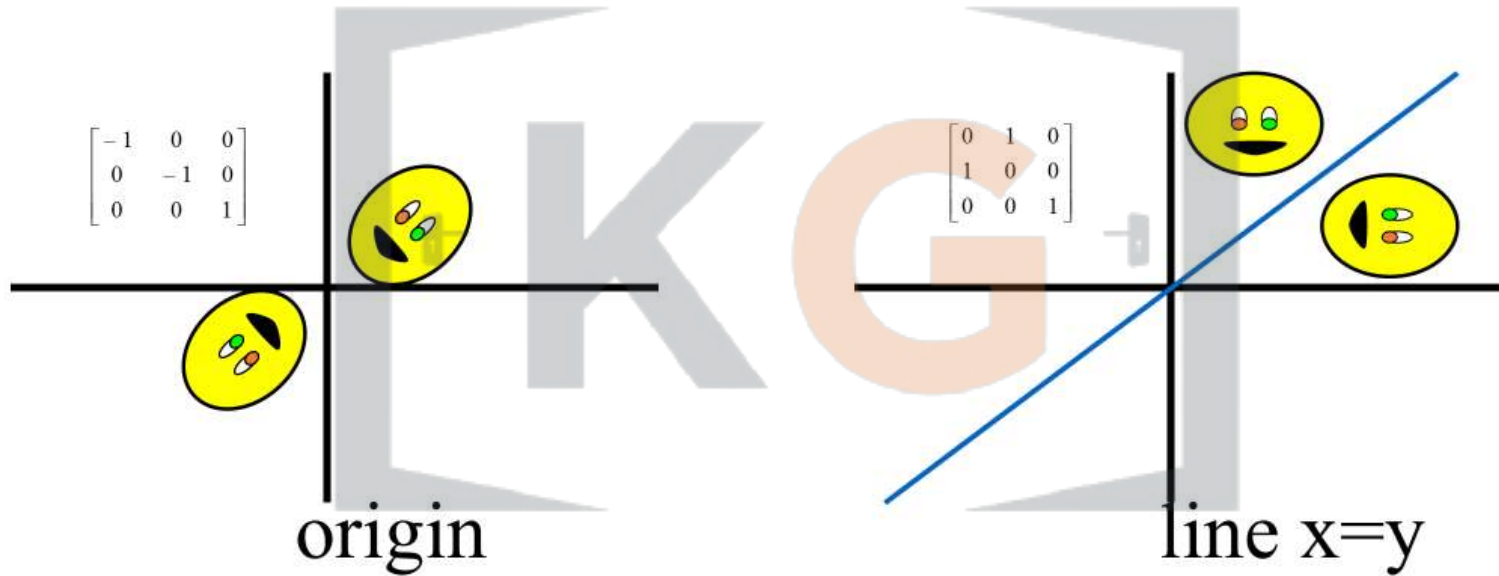
y-axis

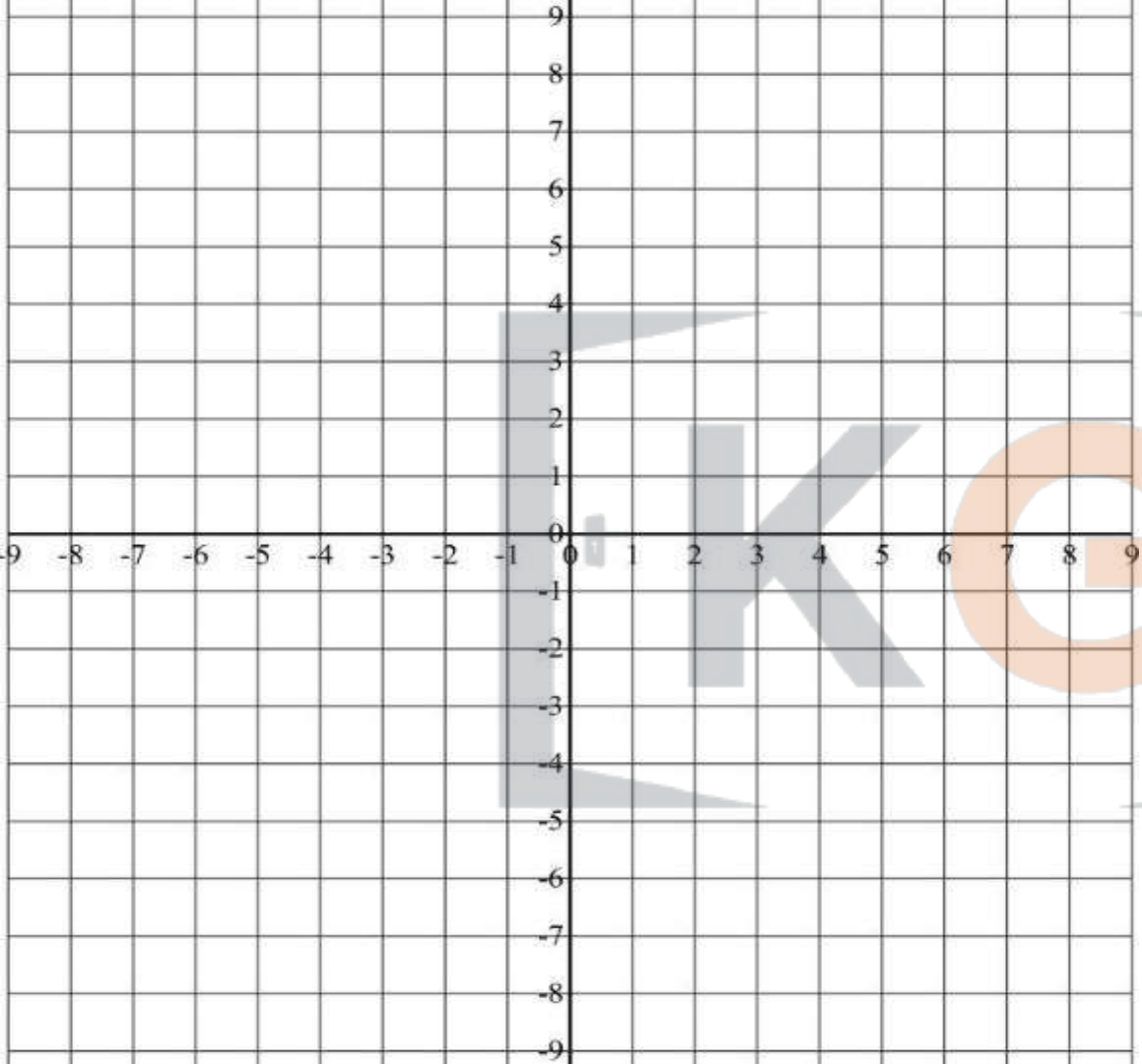


$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Other transformations

Reflection:





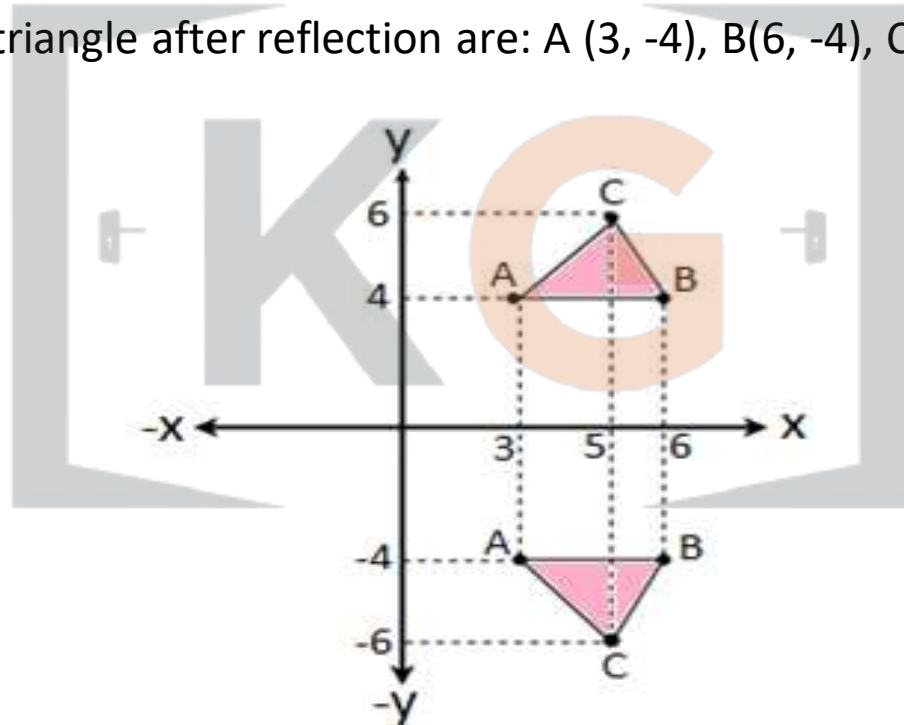
For C (5, 6)

$$x' = x = 5$$

$$y' = -y = -6$$

New coordinates of B after reflection are (5, -6).

New coordinates of the triangle after reflection are: A (3, -4), B(6, -4), C(5, -6).



Shearing

- Shearing is a technique to change the shape of an existing object in a two dimensional plane.
- The object size can be changed along x direction as well as y direction.
- Off diagonal Elements (b and c) are involved in shearing.

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & c \\ b & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Where, $a = d = 1$

Let $c = 0$ and $b = 2$

Thus, $x' = x$ and $y' = 2x + y$;

Here y' linearly depends on x and this effect is known as shear.

Shearing along x-axis

Shearing in X axis is achieved by using the following shearing equations-

$$x' = x + sh_x \times y$$

$$y' = y$$

The matrix form is represented as:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & sh_x \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Shearing along y-axis

Shearing in X axis is achieved by using the following shearing equations-

$$x' = x$$

$$y' = y + sh_y \times x$$

The matrix form is represented as:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ sh_y & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Example: Given a triangle with points as: A (1, 1), B (0, 0) and C (1, 0). Apply shear parameter 2 along the x-axis and find out the new coordinates of the object.

Solution:

Shearing along x-axis

For A (1, 1)

Let the new coordinates for A after shearing be (x', y')

Applying the shearing equations, we have-

$$x' = x + sh_x \times y = 1 + 2 \times 1 = 3$$

$$y' = y = 1$$

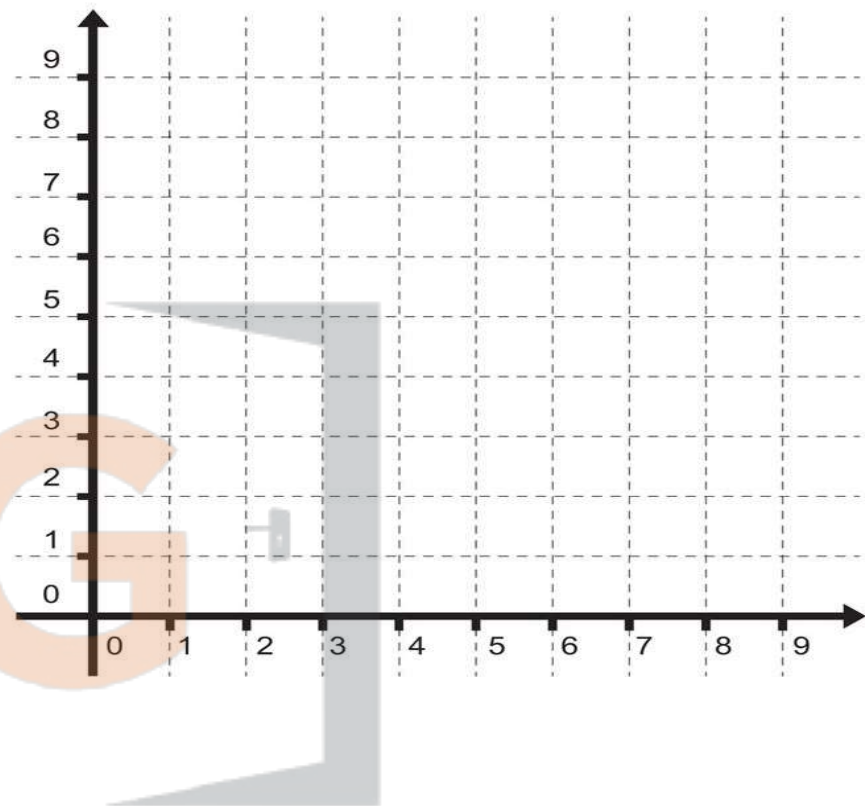
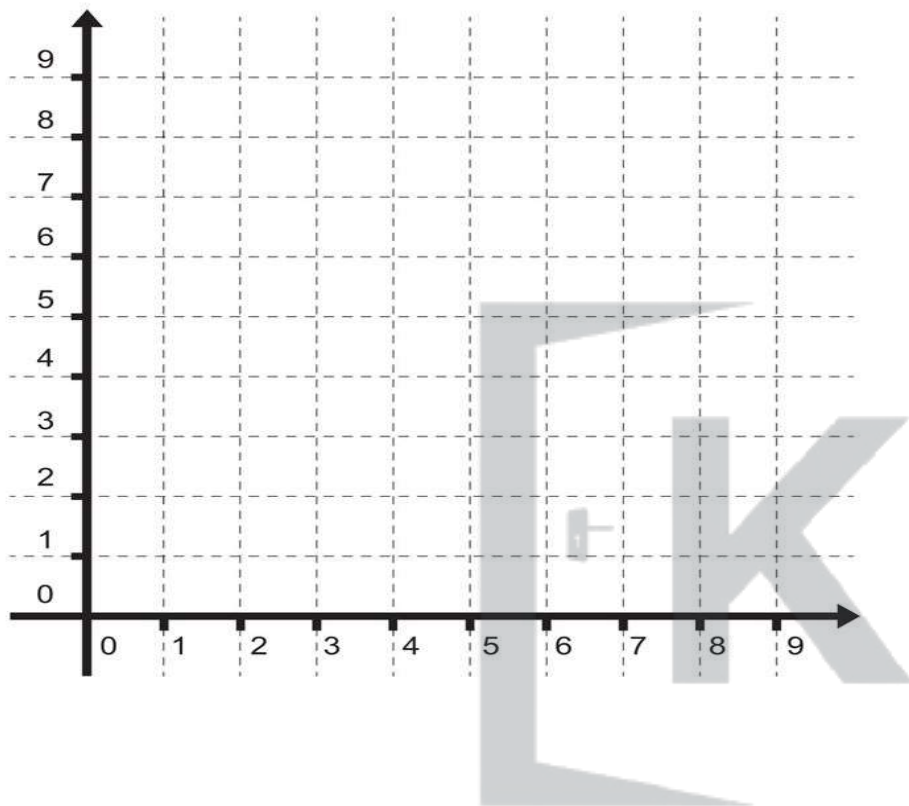
New coordinates for A after shearing are (3, 1).

For B

$$x' = 0 + 2 \times 0 = 0$$

$$y' = 0$$

New coordinates of point B after shearing = (0, 0).



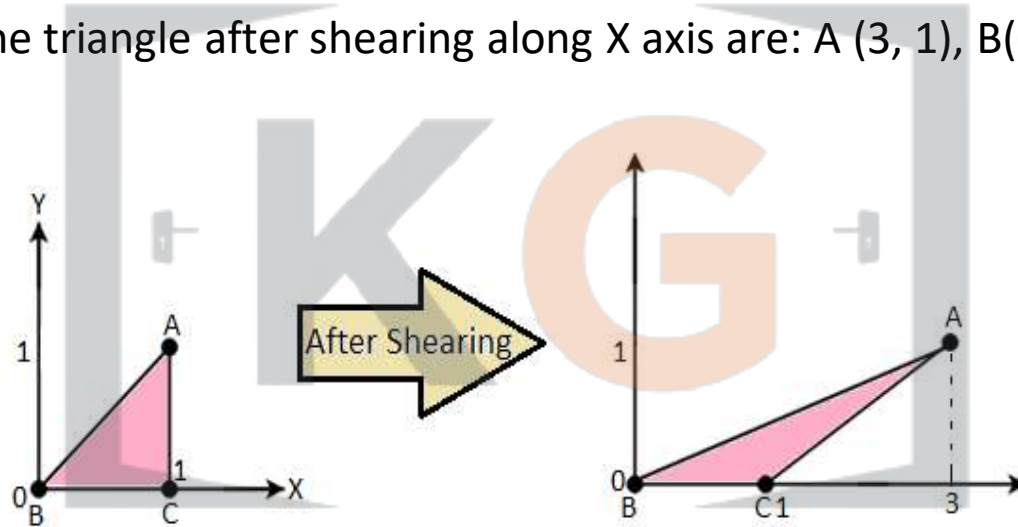
For C

Applying the shearing equations, we have-

$$x' = 1 + 2 \times 0 = 1$$

$$y' = 0$$

New coordinates of the triangle after shearing along X axis are: A (3, 1), B(0, 0), C(1, 0).



Q. Match List-I with List-II. List-I gives 3x3 matrices representing 2 D transformations and List-II shows the corresponding transformation diagrams. (UGC NET PAPER-2020)

List I (Matrix)

(A) $\begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

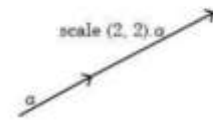
(B) $\begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}$

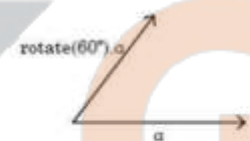
(C) $\begin{bmatrix} 0.5 & -0.866 & 0 \\ 0.866 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

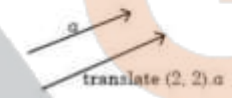
(D) $\begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

List II (Transformation Diagram)

(I) 

(II) 

(III) 

(IV) 

Choose the correct answer from the options given below:

- (a) (b) (c) (d)

(a) (iv) (ii) (iii) (iv)

(b) (iv) (iii) (ii) (i)

(c) (iii) (ii) (iv) (i)

(d) (ii) (iv) (iii) (i)

Ans: D

Homogenous Coordinates

- Many graphics applications involve sequences of geometric transformations.
- To produce a sequence of transformations, such as scaling followed by rotation then translation, we must calculate the transformed coordinates one step at a time.
- First, coordinate positions are scaled, then these scaled coordinates are rotated, and finally the rotated coordinates are translated.

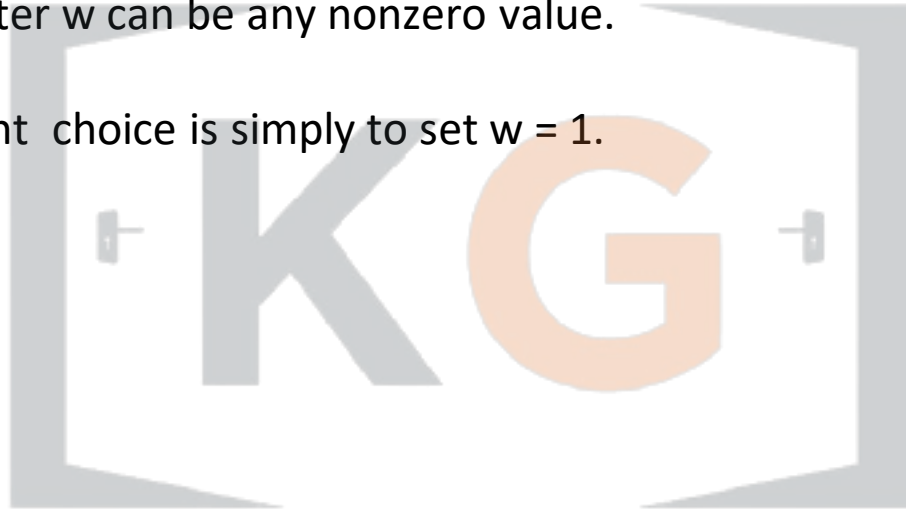
- A more efficient approach would be to combine the transformations so that the final coordinate positions are obtained directly from the initial coordinates, thereby eliminating the calculation of intermediate coordinate values.
- To be able to do this, we reformulate the translation equation to eliminate the matrix addition associated with the translation.



- We do this by expanding the 2 by 2 matrix representations to 3 by 3 matrices.
- This allows us to express all transformation equations as matrix multiplications, providing that we also expand the matrix representations for coordinate positions.
- We represent the translation as our general transformation matrix using homogenous coordinates.



- In homogenous coordinate system we represent each Cartesian coordinate position (x, y) with the homogeneous coordinate triple (x_h, y_h, w) .
- Here, $x = x_h/w$ and $y = y_h/w$ are called the Cartesian coordinates of the homogenous points.
- Homogenous parameter w can be any nonzero value.
- In general a convenient choice is simply to set $w = 1$.



- Two homogenous coordinates (x_1, y_1, w_1) and (x_2, y_2, w_2) may represent a same point iff they are multiples of one another.

Example: A (1, 2, 3) and B (3, 6, 9)

We can obtain the coordinates for 2D for A as: $1/3$ and $2/3$

Similarly, for B we have: $3/9 = 1/3$ and $6/9 = 2/3$

- Thus, there is an infinite number of equivalent homogeneous representations for each coordinate point (x, y) i.e. there is no unique homogenous representation for a point.

Q. A scaling transformation changes the (UP LT - 2018)

- A) size of an object**
- B) location of an object**
- C) shape of an object**
- D) Both (a) and (b)**



Q: Consider the rectangle with vertices $(0,0),(0,2),(3,0),(3,2)$. There is scaling of 2 towards x-axis and 3 towards y-axis. The new coordinates of the rectangle are.

(UGC NET PAPER-2023)

(a) $(0,0),(6,0),(0,4),(6,4)$

(b) $(0,0),(6,0),(0,4),(3,2)$

(c) $(0,0),(6,0),(0,6),(6,6)$

(d) $(0,0),(4,0),(0,6),(4,6)$

Ans: C



**Q: Which of the following transform in 2 dimension is used to resize a 2 - dimensional object?
(UGC NET PAPER-2023)**

- (a)** Translation
- (b)** Rotation
- (c)** Scaling
- (d)** Shearing

Ans: C



General purpose 2D Matrix representation in Homogenous Coordinate System

- The 2D Matrix representation of the homogenous coordinate as:

$$\begin{bmatrix} a & c & p \\ b & d & q \\ m & n & s \end{bmatrix}$$

- In the above matrix:
 - Parameters involved in scaling, rotation, reflection and sheer are: a, b, c and d.
 - Since we are using the pre-multiplication order of matrix in $P' = T \cdot P$, thus the Translation parameters are: p and q.
 - In case we are using the post-multiplication order of matrix in $P' = P \cdot T$, the translation parameters will be: m and n.

- A 3×3 Matrix for translation in homogenous coordinate system is represented as:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & c & t_x \\ b & d & t_y \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad *$$

In abbreviated form we can write as:

$$P' = T(t_x, t_y) \cdot P$$

With $T(t_x, t_y)$ as the 3 by 3 translation matrix

- We can represent 3×3 Matrix for rotation as:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ w \end{bmatrix} \quad *$$

Or as,

$$P' = R(\theta) \cdot P$$

- Other transformations can be represented similarly.

Q. Which homogeneous 2D matrix transforms the figure (a) on the left side to the figure (b) on the right? (UGC NET PAPER-2018)

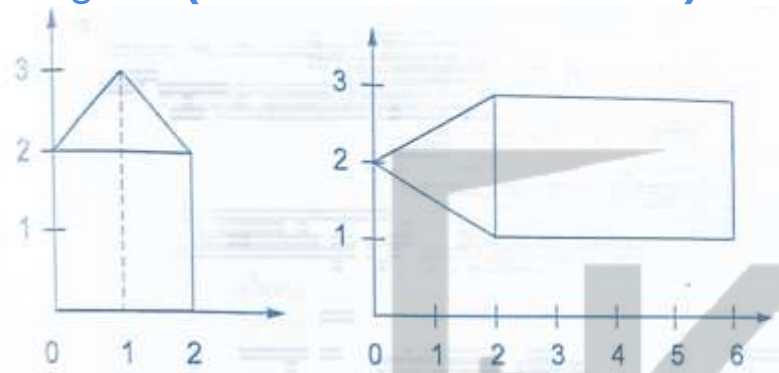


Figure (a)

Figure (b)

(a) $\begin{pmatrix} 1 & -2 & 6 \\ 1 & 0 & 2 \\ 0 & 0 & 1 \end{pmatrix}$

(b) $\begin{pmatrix} 0 & 2 & 6 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}$

(c) $\begin{pmatrix} 0 & -2 & 6 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}$

(d) $\begin{pmatrix} 0 & 2 & -6 \\ 2 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}$

Ans: C

Composite Transformations

- Forming products of transformation matrices is often referred to as a **concatenation**, or **composition**, of matrices.
- In case we want to apply a series of transformations T_1, T_2, T_3 to a set of points, we can do it in two different ways:
 - We can calculate it as:
 - $P' = T_1 \cdot P$
 - Followed by $P'' = T_2 \cdot P'$
 - Followed by $P''' = T_3 \cdot P''$Or,
 - Calculate $T = T_1 * T_2 * T_3$
 - Followed by $P' = T \cdot P$
- Method 2 is more optimal as it saves a lot of computational time.

Composite Translation:

- In case of a translation let us say we want to translate a point by (tx_1, ty_1) and then by (tx_2, ty_2) . Thus we get the matrix as:

$$\begin{bmatrix} 1 & 0 & (tx_2) \\ 0 & 1 & (ty_2) \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & (tx_1) \\ 0 & 1 & (ty_1) \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & (tx_1 + tx_2) \\ 0 & 1 & (ty_1 + ty_2) \\ 0 & 0 & 1 \end{bmatrix}$$

- Thus two successive translations are additive.

Composite Scaling

- Concatenating transformation matrices for two successive scaling operations produces the following composite scaling matrix:

$$\begin{bmatrix} s_{x2} & 0 & 0 \\ 0 & s_{y2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} s_{x1} & 0 & 0 \\ 0 & s_{y1} & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} s_{x1} \cdot s_{x2} & 0 & 0 \\ 0 & s_{y1} \cdot s_{y2} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Successive scaling operations are multiplicative.
- That is, if we were to triple the size of an object twice in succession, the final size would be nine times that of the original.

Composite Rotation

- Two successive rotations applied to point P produce the transformed position

$$P' = R(\theta_2) \cdot R(\theta_1) \cdot P$$

- By multiplying the two rotation matrices, we can verify that two successive rotations are additive:

$$\text{i.e. } [R(\theta_2) \cdot R(\theta_1)] = [R(\theta_2 + \theta_1)]$$

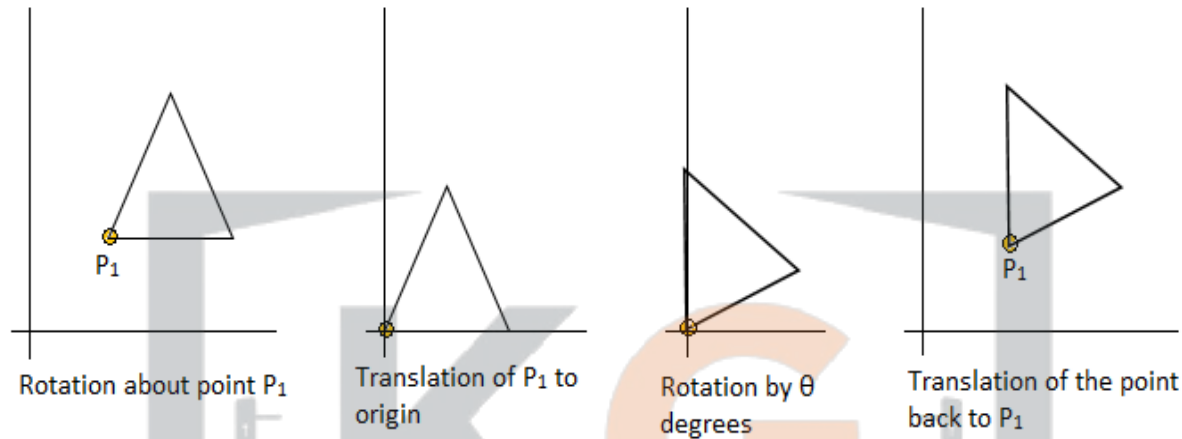
Thus we get:

$$P' = R(\theta_2 + \theta_1) \cdot P$$

General Pivot-Point Rotation

- We can generate rotations about any selected pivot point (x_r, y_r) by performing the following sequence of translate-rotate-translate operations:
 1. Translate the object so that the pivot-position is moved to the coordinate origin using $(-P_x, -P_y)$.
 1. Rotate the object about the coordinate origin.
 1. Translate the object so that the pivot point is returned to its original position using (P_x, P_y) .

Example: Consider the figure given below showing how to rotate about any selected pivot point.



- The composite transformation matrix of this sequence can be expressed as:

$$T = T_1(-P_x, -P_y) * T_2(\theta) * T_3(P_x, P_y)$$

- In matrix form it is represented as:

$$\begin{bmatrix} 1 & 0 & -P_x \\ 0 & 1 & -P_y \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & P_x \\ 0 & 1 & P_y \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & P_x * (\cos \theta - 1) - p_y * \sin \theta \\ \sin \theta & \cos \theta & P_y * (\cos \theta - 1) + p_x * \sin \theta \\ 0 & 0 & 1 \end{bmatrix}$$

General Pivot-Point Scaling

- Scaling about a pivot also follows the same set of operation:

1. Translate P to origin
2. Scale
3. Translate P back

- The composite transformation matrix of this sequence can be expressed as:

$$T = T_1(P_x, P_y) * T_2(s_x, s_y) * T_3(-P_x, -P_y)$$

- In matrix form:

$$T = \begin{bmatrix} s_x & 0 & (P_x(1 - s_x)) \\ 0 & s_y & (P_y(1 - s_y)) \\ 0 & 0 & 1 \end{bmatrix}$$

Q: Given a vector with Cartesian components (x,y) , if scaling is done with Matrix Row1[0.5 0], Row2[0 1.5] which of the following are true:

- (A) Decrease the vertical by three halves**
- (B) Increase the vertical by three halves**
- (C) Double the horizontal**
- (D) Halves the horizontal**

Choose the correct from the options given below:

(UGC NET PAPER-2022)

- (a) A and C only**
- (b) A and D only**
- (c) B and C only**
- (d) B and D only**

Ans: D

Q. Consider the following statements regarding 2D transforms in computer graphics:

S1 $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ is a 2x2 matrix that reflects (mirrors) only 2D point

about the X axis

S2= A 2x2 matrix which mirrors any 2D point about the X axis is a rotation matrix

What can you say about the statement S1 and S2? (**UGC NET PAPER-2019**)

1. Both S1 and S2 are true
2. Only S1 is true
3. Only S2 is true
4. Both S1 and S2 are false

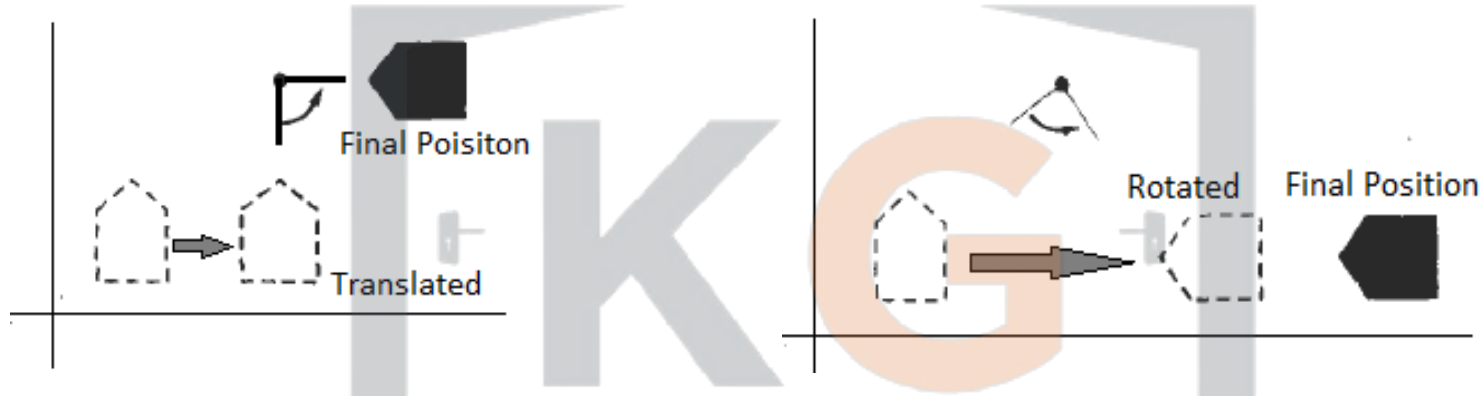
Ans: B

Concatenation Properties

- Matrix multiplication is associative. For any three transformation matrices, T_1 , T_2 , and T_3
$$T_1 \cdot T_2 \cdot T_3 = (T_1 \cdot T_2) \cdot T_3 = T_1 \cdot (T_2 \cdot T_3)$$
- Transformation products may not be commutative: The matrix product $T_1 \cdot T_2$ is not equal to $T_2 \cdot T_1$, in general.
- There might be some cases in which $T_1 \cdot T_2 = T_2 \cdot T_1$ such as:

T_1	T_2
Rotation	Rotation
scaling	scaling
Translation	Translation
Scaling (uniform)	rotation

- The order in which we apply transformation is very important leaving aside the above mentioned cases.
- **Example:** In the below first figure an object is first translated, then rotated. In the next figure the object is rotated first and then translated.



- Thus reversing the order in which a sequence of transformation is performed may affect the transformed position of an object.

THE VIEWING PIPELINE

- It is formal mechanism for displaying views of a picture on an output device.
- Any convenient Cartesian coordinate system, referred to as the world-coordinate reference frame, can be used to define the picture.
- For a two-dimensional picture, a view is selected by specifying a subarea of the total picture area.
- A world-coordinate area selected for display is called a **window**.
- An area on a display device to which a window is mapped is called a **viewport**.
- The window defines what is to be viewed; the viewport defines where it is to be displayed.

2D Viewing

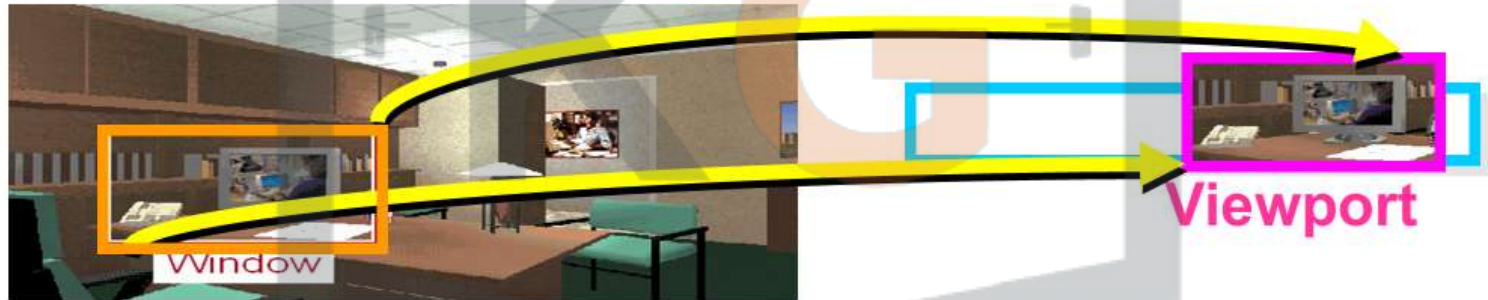
- The geometric description of the object or scene provided by the model, is converted into a set of graphical primitives, which are displayed where desired on a 2D display.
- The same abstract model may be viewed in many different ways:
 - e.g. faraway, near, looking down,

The Viewing Pipeline

- A world coordinate area selected for display is called **window**.
- An area on a display device to which a window is mapped a **viewport**.
- Windows and viewports are rectangular in standard position.

The Viewing Pipeline

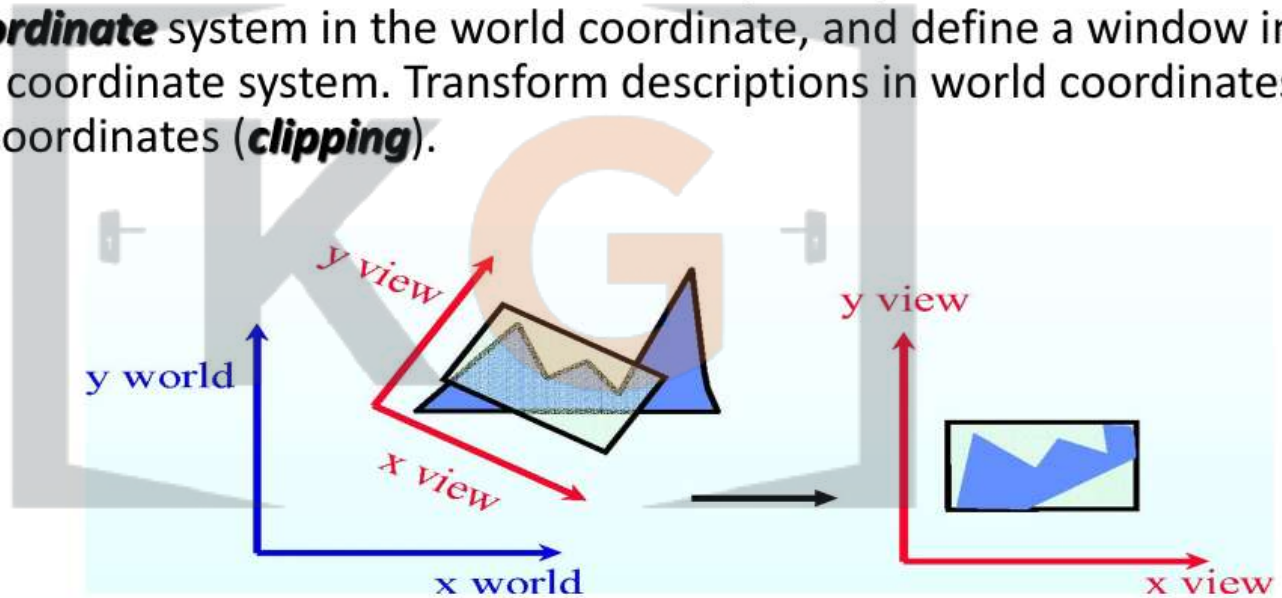
- The mapping of a part of a world coordinate scene to device coordinate is referred to as **viewing transformation** or **window-to-viewport transformation** or **windowing transformation**.



window-to-viewport transformation

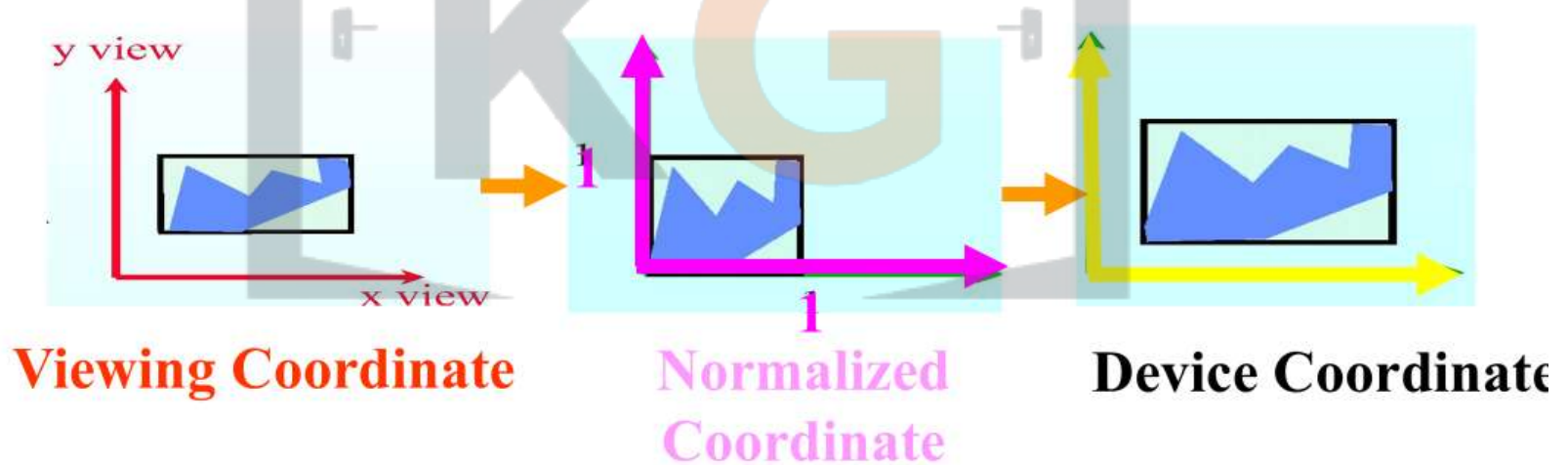
The Viewing Pipeline

1. Construct the scene in world coordinate using the output primitives.
2. Obtain a particular orientation for the window by set up a two dimensional **viewing coordinate** system in the world coordinate, and define a window in the viewing coordinate system. Transform descriptions in world coordinates to viewing coordinates (**clipping**).

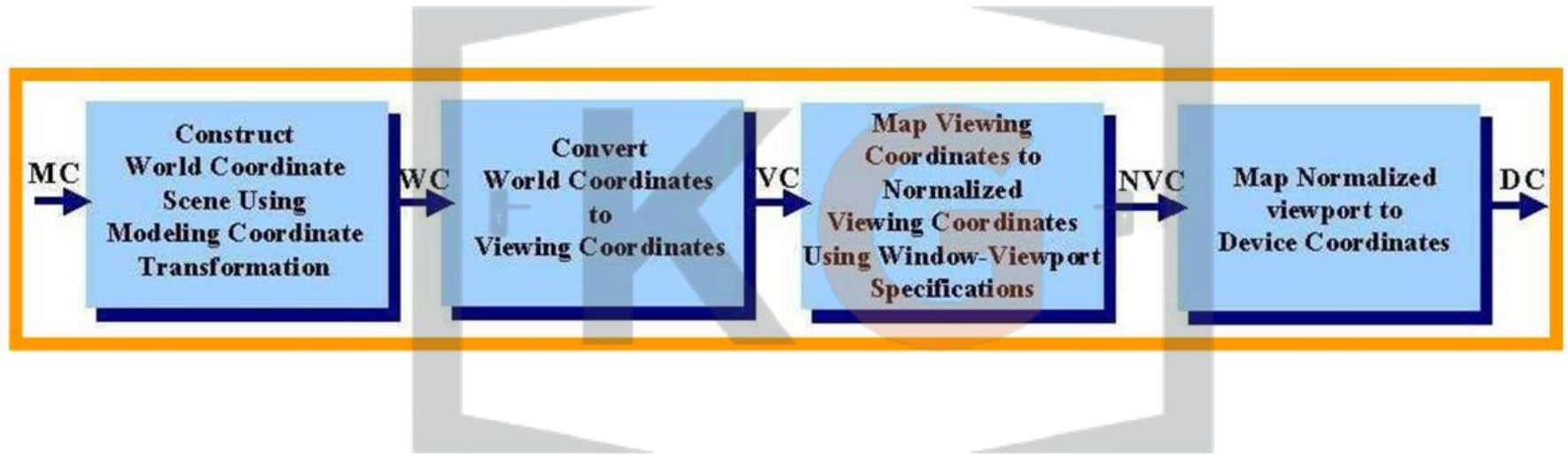


The Viewing Pipeline

3. Define a viewport in normalized coordinate, and map the viewing coordinate description of the scene to normalized coordinate
4. (All parts lie outside the viewport are **clipped**), and contents of the viewport are transferred to device coordinates



The Viewing Pipeline

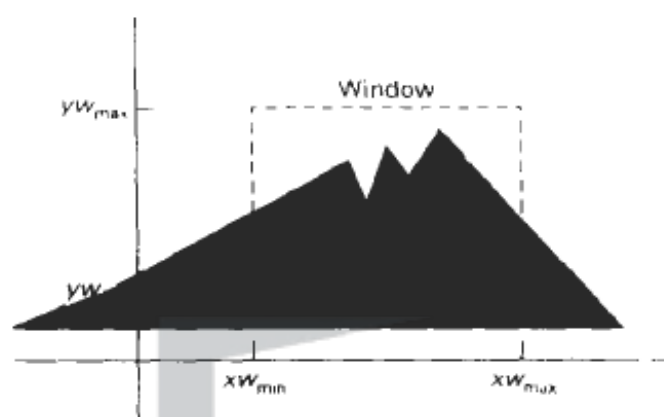


WINDOW-TO-VIEWPORT COORDINATE TRANSFORMATION

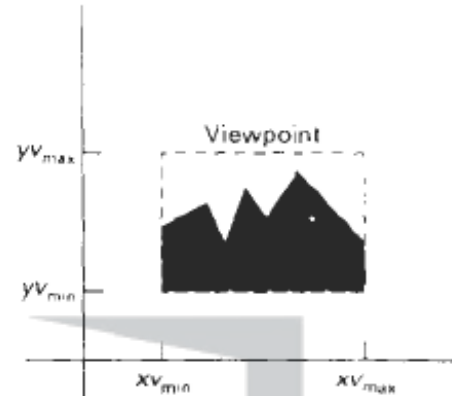
- Graphics applications often require the transformation of object descriptions from one coordinate system to another.
- The purpose is to find a transformation matrix that maps the window in world coordinate system to viewport in screen coordinates.
- Window xy space is generally denoted as: $x_{min}, y_{min}, x_{max}, y_{max}$.
- Viewport uv space is generally denoted as: $u_{min}, v_{min}, u_{max}, v_{max}$.

Steps involved in transformation w.r.t. to the origin:

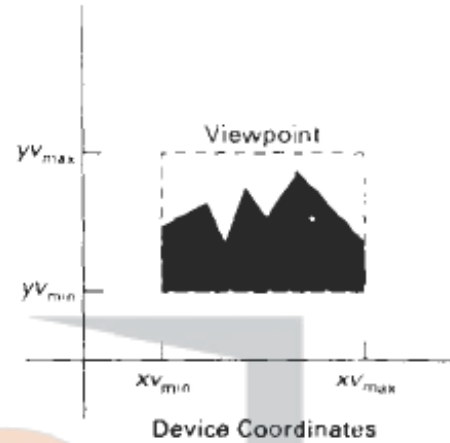
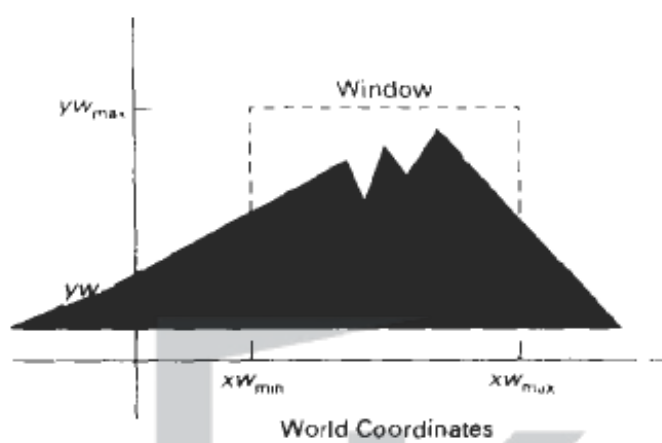
- Translate the window to the origin.
- Scale it to the size of the viewport.
- Translate it to the viewport location.



World Coordinates



Device Coordinates



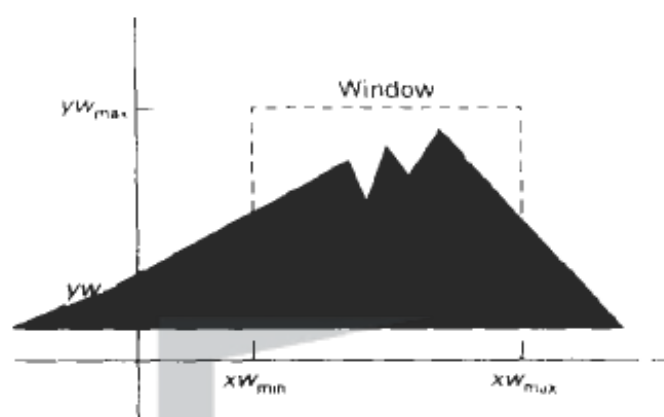
$$(x_v - x_{vmin}) / (x_{vmax} - x_{vmin}) = (x_w - x_{wmin}) / (x_{wmax} - x_{wmin})$$

$$(x_v - x_{vmin}) = [(x_w - x_{wmin}) / (x_{wmax} - x_{wmin})] * (x_{vmax} - x_{vmin})$$

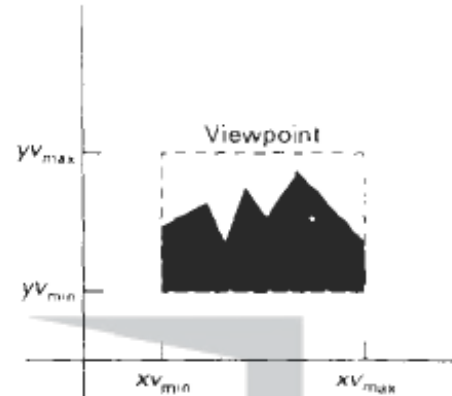
$$(x_v - x_{vmin}) = [(x_{vmax} - x_{vmin}) / (x_{wmax} - x_{wmin})] * (x_w - x_{wmin})$$

$$(x_v - x_{vmin}) = S_x * (x_w - x_{wmin})$$

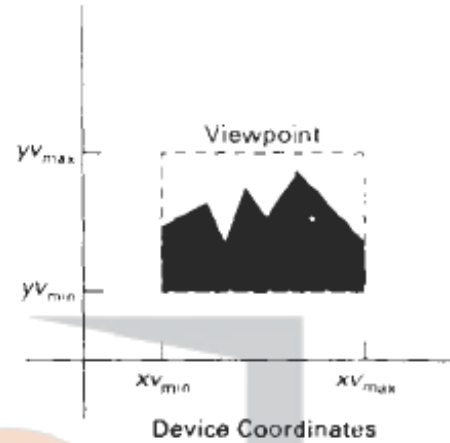
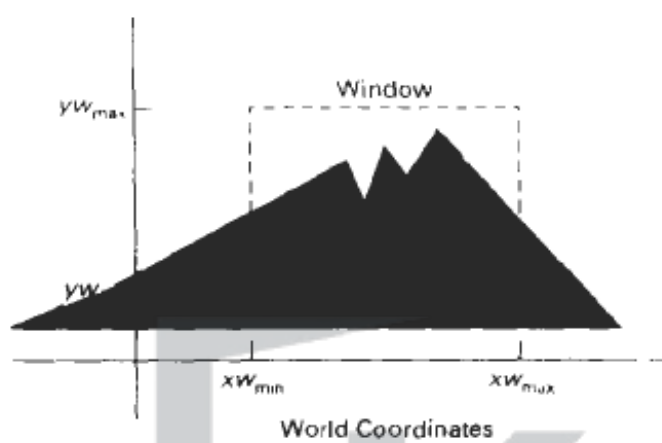
$$x_v = x_{vmin} + [S_x * (x_w - x_{wmin})]$$



World Coordinates



Device Coordinates



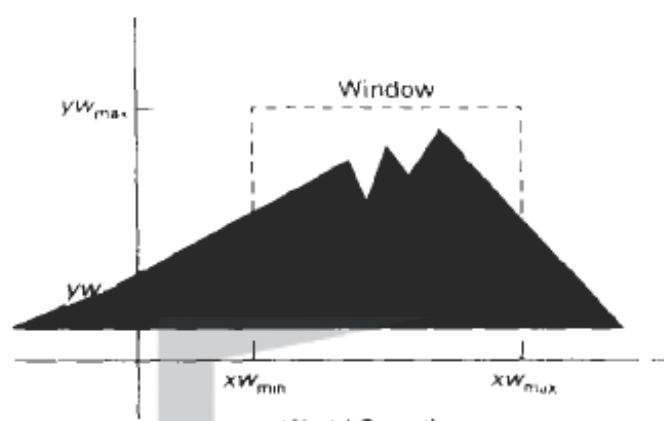
$$(Y_v - Y_{vmin}) / (Y_{vmax} - Y_{vmin}) = (Y_w - Y_{wmin}) / (Y_{wmax} - Y_{wmin})$$

$$(Y_v - Y_{vmin}) = [(Y_w - Y_{wmin}) / (Y_{wmax} - Y_{wmin})] * (Y_{vmax} - Y_{vmin})$$

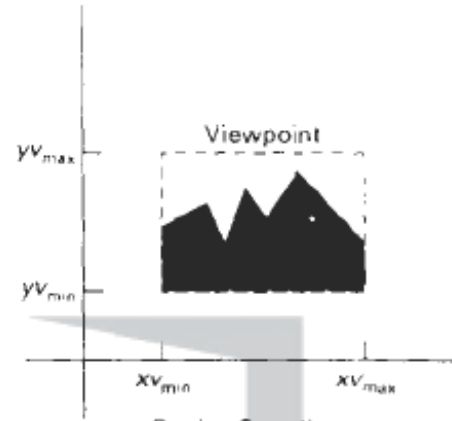
$$(Y_v - Y_{vmin}) = [(Y_{vmax} - Y_{vmin}) / (Y_{wmax} - Y_{wmin})] * (Y_w - Y_{wmin})$$

$$(Y_v - Y_{vmin}) = S_Y * (Y_w - Y_{wmin})$$

$$Y_v = Y_{vmin} + [S_Y * (Y_w - Y_{wmin})]$$



World Coordinates



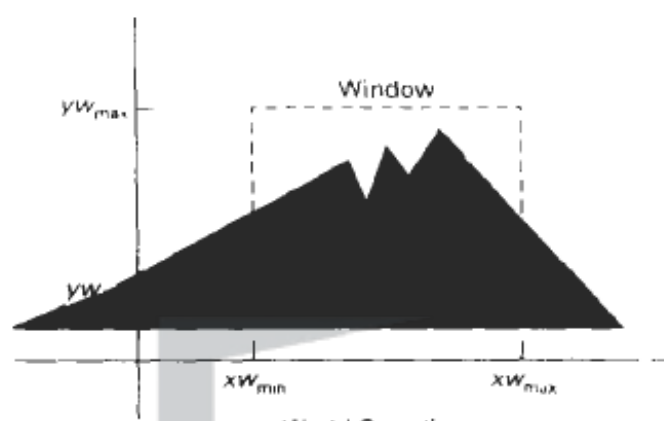
Device Coordinates

$$W = TST^{-1}$$

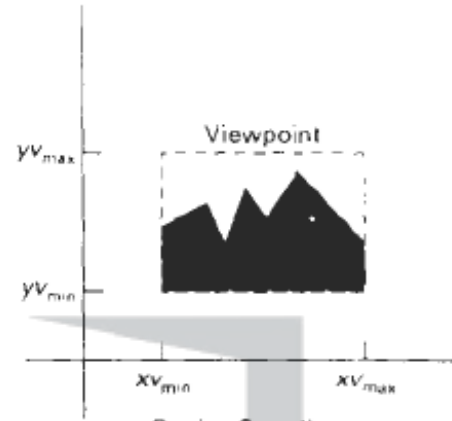
$$W = T$$

$$T^{-1}$$

S



World Coordinates



Device Coordinates

$$W = TST^{-1}$$

$$W = T$$

$$T^{-1}$$

S

The Viewing Pipeline

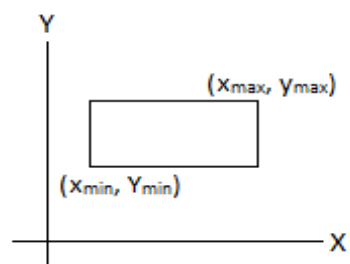
- By Changing the position of the viewport, we can view objects at different position on the display area of an output device.



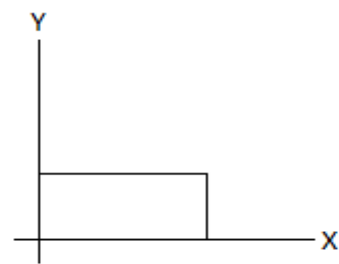
The Viewing Pipeline

- By Changing the position of the viewport, we can view objects at different position on the display area of an output device.





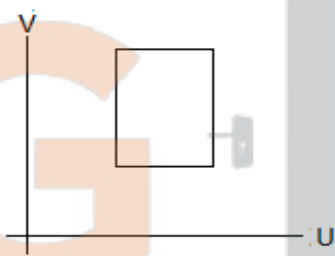
Window in world
Coordinate



Window Translated to
origin



Window scaled to the
viewport size



View port translated to
final position

Affine Transformations

- Affine transformation is a linear mapping method that preserves points, straight lines, and planes.
- Sets of parallel lines remain parallel and finite point's maps to finite points after an affine transformation.
- **Examples:** translation, scale, shear, reflection and rotation.
- Any general two-dimensional affine transformation can always be expressed as a composition of these five transformations.
- Another affine transformation is the conversion of coordinate descriptions from one reference system to another, which can be described as a combination of translation and rotation.

Affine Transform

Example

Translation



Scale



Shear



Rotation



CLIPPING OPERATIONS

- Any procedure that identifies those portions of a picture that are either inside or outside of a specified region of space is referred to as a **clipping algorithm**, or simply **clipping**.
- The region against which an object is to be clipped is called a clip **window**.

Applications of clipping

- Extracting part of a defined scene for viewing (**viewing transformations**);
- Identifying visible surfaces in three-dimensional views;
- Antialiasing line segments or object boundaries;
- Creating objects using solid-modeling procedures;
- Displaying a multi-window environment;
- Drawing and painting operations that allow parts of a picture to be selected for copying, moving, erasing, or duplicating.

- Depending on the application, the clip window can be a general polygon or it can even have curved boundaries.

We consider algorithms for clipping the following primitive types

- Point Clipping
- Line Clipping (straight-line segments)
- Area Clipping (polygons)
- Curve Clipping
- Text Clipping



LINE CLIPPING

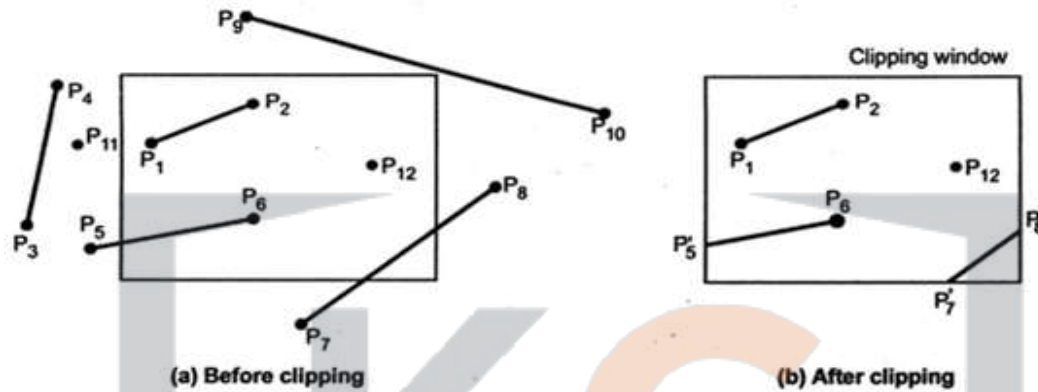


Figure shown above illustrates possible relationships between line positions and a standard rectangular clipping region.

Four Standard Cases:

1. A line lies completely inside the clipping window (p_1 and p_2).
2. A line lies completely outside the clipping window (p_3 and p_4).
3. A line having one endpoint inside the clipping window (p_5 and p_6).
4. A line having both end points outside the window but a portion of line lies inside the clipping window (p_7 and p_8).

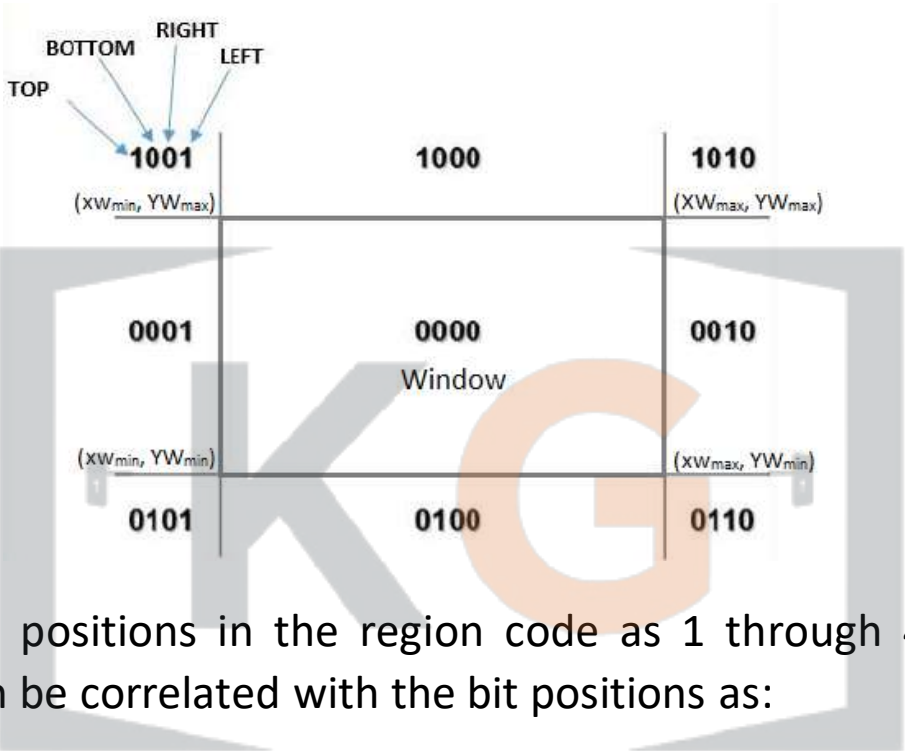
Cohen Sutherland Line Clipping

- This is one of the oldest and most popular line-clipping procedures.

Binary Region Codes

- Every line end-point in a picture is assigned a four-digit binary code, called a region code that identifies the location of the point relative to the boundaries of the clipping rectangle.
- Regions are set up in reference to the boundaries.
- Each bit position in the region code is used to indicate one of the four relative coordinate positions of the point with respect to the clip window: to the left, right, top, or bottom.

Consider the figure below:



- By numbering the bit positions in the region code as 1 through 4 from right to left, the coordinate regions can be correlated with the bit positions as:
 - bit 1: left
 - bit 2: right
 - bit 3: below
 - bit 4: above

- A value of 1 in any bit position indicates that the point is in that relative position; otherwise, the bit position is set to 0.
- As shown in the figure above a point in the coded region 1001: represents that the point is in the left top w.r.t to the clipping window.
- If a point is within the clipping rectangle, the region code is 0000.
- Bit values in the region code are determined by comparing endpoint coordinate values (x, y) to the clip boundaries.
- Bit 1 is set to 1 if $x < XW_{min}$. The other three bit values can be determined using similar comparisons.

Working

Step 1: Establish region codes for all line endpoints.

Step 2: If: both end points of the line have the region code: 0000 (done by performing a logical OR operation between the two end points) the line lies inside the window and we trivially accept these lines.

Step 3: Else if: We perform a logical AND between the two endpoints region codes:

- If Result $\neq$ 0000, we reject the line as it will lie outside the window.

Else if: Result = 0000 then the line lies partially inside the window.

- Select any of the endpoint.
- Find the intersection point.
- We clip the line and replace the endpoint with the intersection point and update the region code.
- Repeat Step 2 till the lines are trivially accepted or rejected.

Step 4: Repeat the process for other lines.

Pseudo code

1. Assign region codes to point P_1 and P_2 .
2. If $P_1 \text{ OR } P_2 = 0000$, Accept the line.
3. Else if $P_1 \text{ AND } P_2 \neq 0000$; Reject the line.
4. Else clip the line till the intersection point:
 - For a line with endpoint coordinates (x_1, y_1) and (x_2, y_2) , the coordinate of the intersection point with a vertical boundary ($XW_{\min}$ or $XW_{\max}$) can be obtained with the calculation:

$$y = y_1 + m(x - x_1)$$

Where the x value is set either to $XW_{\min}$ or $XW_{\max}$ (based on the intersection), and the slope of the line is calculated as

$$m = (y_2 - y_1) / (x_2 - x_1)$$

- For Horizontal boundary ($YW_{\min}$ or $YW_{\max}$) we calculate the x-coordinate intersection points as:

$$x = x_1 + \frac{1}{m} (y - y_1)$$

With y set either to $YW_{\min}$ or $YW_{\max}$.

5. Update the points.

6. Verify if: $XW_{\min} \leq x \leq XW_{\max}$ and $YW_{\min} \leq y \leq YW_{\max}$. If not then repeat the process.



Example: Let ABCD be the rectangular window with the lower left hand corners at (20, 20) and the upper right hand corner at (90, 70). Use Cohen Sutherland algorithm to clip the points $P_1 = (10, 30)$ and $P_2 = (80, 90)$.

Solution:

The region codes of $P_1 = 0001$ and $P_2 = 1000$.

Performing the AND operation we get: 0000. Thus line is partially inside the window.

Calculating the slope: $m = (y_2 - y_1) / (x_2 - x_1) = (90 - 30) / (80 - 10) = 6/7$

Calculate the intersection point for P_1 : $y = y_1 + m(x - x_1)$

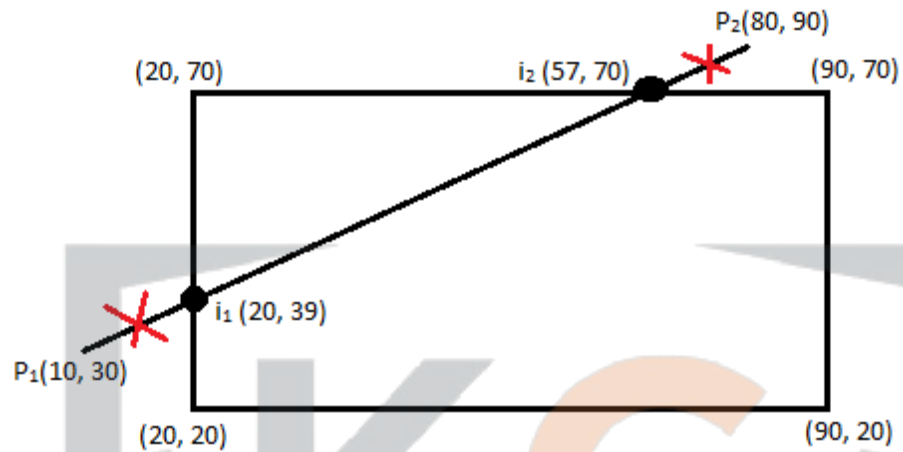
$$y = 30 + 6/7 (20 - 10) = 38.57 = 39$$

Intersection point for P_1 : (20, 39)

Similarly, calculate the intersection point for P_2 : $x = x_1 + \frac{1}{m} (y - y_1)$

$$x = 10 + 7/6 (70 - 30) = 56.66 = 57$$

Intersection point for P_2 : (57, 70)



We clip up to the intersection point

Example: Let R be the rectangular window against which the lines are to be clipped using 2D Sutherland-Cohen line clipping algorithm. The rectangular window has lower left-hand corner at $(-5, 1)$ and upper right-hand corner at $(3, 7)$. Consider the following three lines for clipping with the given end point co-ordinates:

Line AB : A $(-6, 2)$ and B $(-1, 8)$

Line CD : C $(-1, 5)$ and D $(4, 8)$

Line EF : E $(-2, 3)$ and F $(1, 2)$

Which of the following line(s) is/are candidate for clipping?

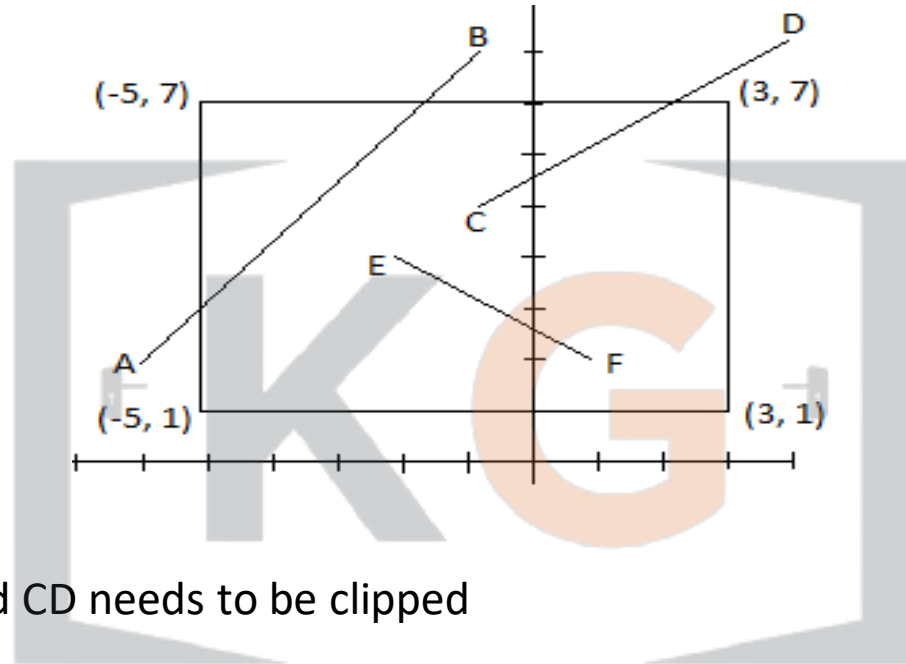
A. AB

B. DC

C. EF

D. AB and CD

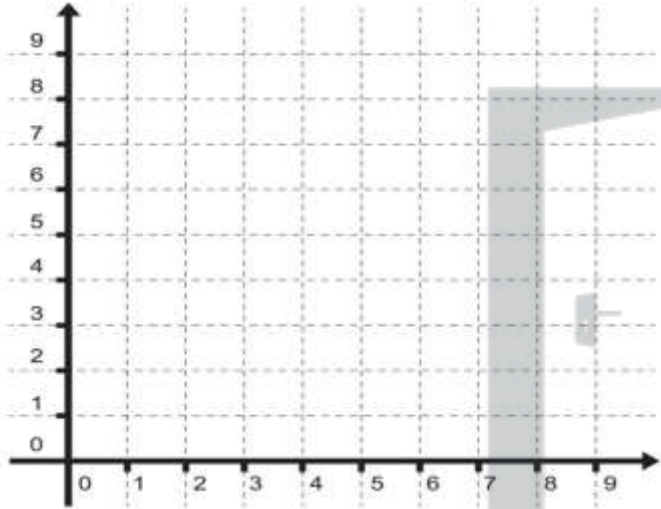
Solution: Drawing the window along with lines we will get to the answer.



Thus answer is D. AB and CD needs to be clipped

Liang-Barsky Line Clipping

- Liang and Barsky independently devised a parametric line-clipping algorithm known as Liang-Barsky Line clipping Algorithm.



Parametric Equation of a line

- Consider a line with end point (x_1, y_1) and (x_2, y_2) the parametric equation of the line will be:

- $x = (1-t)x_1 + tx_2$
- $y = (1-t)y_1 + ty_2$

- $x = x_1 + t \Delta x$
- $y = y_1 + t \Delta y.$

Where $0 \leq t \leq 1$ and $\Delta x = x_2 - x_1$, and $\Delta y = y_2 - y_1$.

- We first write the point-clipping conditions discussed previously in the parametric form:

- $x_{wmin} \leq x \leq x_{wmax}$

- $x_{wmin} \leq x_1 + t \Delta x \leq x_{wmax}$

- $y_{wmin} \leq y \leq y_{wmax}$

- $y_{wmin} \leq y_1 + t \Delta y \leq y_{wmax}$



• We first write the point-clipping conditions discussed previously in the parametric form:

- $x_{wmin} \leq x \leq x_{wmax}$
- $x_{wmin} \leq x_1 + t \Delta x \leq x_{wmax}$

- $y_{wmin} \leq y \leq y_{wmax}$
- $y_{wmin} \leq y_1 + t \Delta y \leq y_{wmax}$

- $x_{wmin} \leq x_1 + t \Delta x$
- $t \Delta x \geq x_{wmin} - x_1 \dots\dots\dots 1$

- $x_1 + t \Delta x \leq x_{wmax}$
- $t \Delta x \leq x_{wmax} - x_1 \dots\dots\dots 2$

- $y_{wmin} \leq y_1 + t \Delta y$
- $t \Delta y \geq y_{wmin} - y_1 \dots\dots\dots 3$

- $y_1 + t \Delta y \leq y_{wmax}$
- $t \Delta y \leq y_{wmax} - y_1 \dots\dots\dots 4$

- $x_{wmin} \leq x_1 + t \Delta x$
- $t \Delta x \geq x_{wmin} - x_1 \dots\dots\dots 1$

- $x_1 + t \Delta x \leq x_{wmax}$
- $t \Delta x \leq x_{wmax} - x_1 \dots\dots\dots 2$

- $y_{wmin} \leq y_1 + t \Delta y$
- $t \Delta y \geq y_{wmin} - y_1 \dots\dots\dots 3$

- $y_1 + t \Delta y \leq y_{wmax}$
- $t \Delta y \leq y_{wmax} - y_1 \dots\dots\dots 4$

$t \Delta x \geq x_{wmin} - x_1 \dots\dots\dots 1$	$-t \Delta x \leq x_1 - x_{wmin} \dots\dots\dots 1$
$t \Delta x \leq x_{wmax} - x_1 \dots\dots\dots 2$	$t \Delta x \leq x_{wmax} - x_1 \dots\dots\dots 2$
$t \Delta y \geq y_{wmin} - y_1 \dots\dots\dots 3$	$-t \Delta y \leq y_1 - y_{wmin} \dots\dots\dots 3$
$t \Delta y \leq y_{wmax} - y_1 \dots\dots\dots 4$	$t \Delta y \leq y_{wmax} - y_1 \dots\dots\dots 4$

- Each of these four inequalities can be expressed as:
- $\mathbf{t} \mathbf{p}_k \leq \mathbf{q}_k$ where $k = 1, 2, 3, 4$

where parameters p and q are defined as:

$p_1 = -\Delta x$	$(q_1 = x_1 - x_{wmin})$
$p_2 = \Delta x$	$(q_2 = x_{wmax} - x_1)$
$p_3 = -\Delta y$	$(q_3 = y_1 - y_{wmin})$
$p_4 = \Delta y$	$(q_4 = y_{wmax} - y_1)$

$$-t \Delta x \leq x_1 - x_{wmin} \dots\dots 1$$

$$t \Delta x \leq x_{wmax} - x_1 \dots\dots 2$$

$$-t \Delta y \leq y_1 - y_{wmin} \dots\dots 3$$

$$t \Delta y \leq y_{wmax} - y_1 \dots\dots 4$$

Steps to Follow

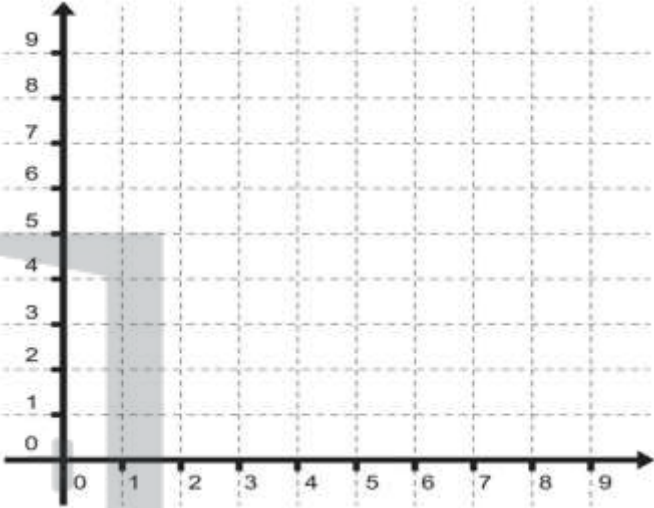
- Start with the endpoints (x_1, x_2) and (y_1, y_2)
- Calculate the respective values of: Δx , Δy , p_1 , p_2 , p_3 , p_4 , q_1 , q_2 , q_3 and q_4 .
- Assign values $t_1 = 0$ and $u_2 = 1$.
- Check for $p_k = 0$
 - If $p_k = 0$ for $k = 1, 2, 3, 4$, If yes then the line is parallel to one of the clipping window boundary.
 - Further check If $q_k < 0$ for $k = 1, 2, 3, 4$, Then the line is outside the clipping window and can be eliminated. Else it is inside and is parallel to boundary.

- Check for $p_k \neq 0$
 - If $p_k < 0$ (line proceeds from outside to inside); Update the value of t_1 as:
 - **$t_1 = \max(0, q_k / p_k)$**
 - If $p_k > 0$ (line proceeds from inside to outside); Update the value of t_2 as:
 - **$t_2 = \min(1, q_k / p_k)$**
- If $t_1 > t_2$; Then line is completely outside and we reject it
- Else the endpoints of the clipped line are calculated from the two values of parameter t through:
 - $x = x_1 + t \Delta x$
 - $y = y_1 + t \Delta y$.

- $t_1 > t_2$; Then line is completely outside and we reject it
- Else the endpoints of the clipped line are calculated from the two values of parameter t through:
 - $x = x_1 + t \Delta x$
 - $y = y_1 + t \Delta y$.
- Putting t_1 and t_2 value in both equation we will get first and second intercept



Example: The Liang-Barsky line clipping algorithm uses the parametric equation of a line from (10, 30) to (80, 90) , window A(20,20), B(90,20), C(90,70), D(20,70)



Example A line AB with end points A(-1, 7) and B(11, 1) is to be clipped against a rectangular window with $x_{\min} = 1$, $x_{\max} = 9$, $y_{\min} = 2$, and $y_{\max} = 8$. The lower and upper bound values of the parameter u for the clipped line using Liang-Barsky algorithm is given as :

(A) (0, 2/3)

(B) (1/6 , 5/6)

(C) (0, 1/3)

(D) (0, 1)

Ans. B



Solution:

$$\Delta x = x_2 - x_1 = 11 - (-1) = 11 + 1 = 12.$$

$$\Delta y = y_2 - y_1 = 1 - 7 = -6$$

$$\text{Now } p_1 = -\Delta x = -12; p_2 = \Delta x = 12; p_3 = -\Delta y = 6; p_4 = \Delta y = -6$$

$$q_1 = x_1 - x_{\min} = -1 - 1 = -2; q_2 = x_{\max} - x_1 = 9 - (-1) = 9 + 1 = 10; q_3 = y_1 - y_{\min} = 7 - 2 = 5; q_4 = y_{\max} - y_1 = 8 - 7 = 1.$$

Here $p_1, p_4 < 0$ and $p_2, p_3 > 0$.

Initially: $u_1 = 0, u_2 = 1$

$$u_1 = \max(0, q_1 / p_1 \text{ and } q_4 / p_4) = \max(0, 2 / 12, 1 / -6) = 1 / 6.$$

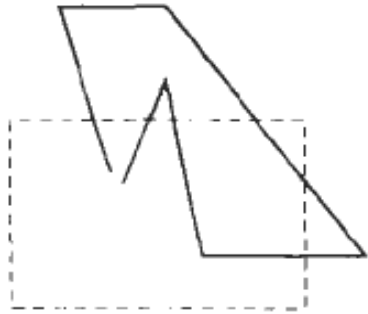
$$u_2 = \min(1, q_1 / p_1, q_4 / p_4) = \min(1, 10 / 12, 5 / 6) = 5 / 6.$$

u ranges between $(1 / 6, 5 / 6)$.

So, option (B) is correct.

POLYGON CLIPPING

- To clip polygons, we need to modify the line-clipping procedures.
- A polygon boundary processed with a line clipper may be displayed as a series of unconnected line segments depending on the orientation of the polygon to the clipping window.



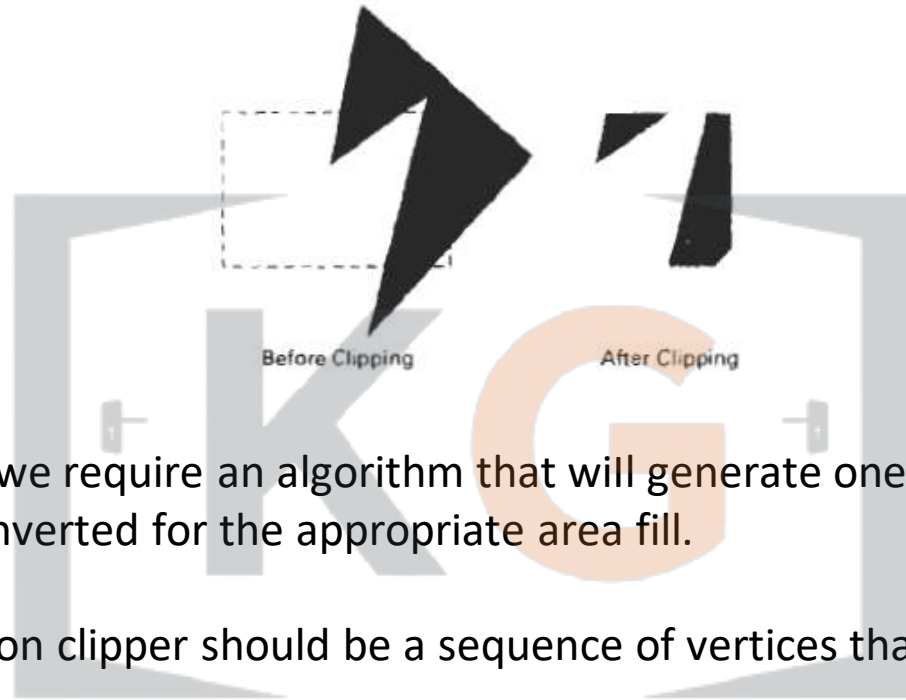
Before Clipping



After Clipping



- We want to display a bounded area after clipping:



- For polygon clipping, we require an algorithm that will generate one or more closed areas that are then scan converted for the appropriate area fill.
- The output of a polygon clipper should be a sequence of vertices that defines the clipped polygon boundaries.

Sutherland-Hodgeman Polygon Clipping

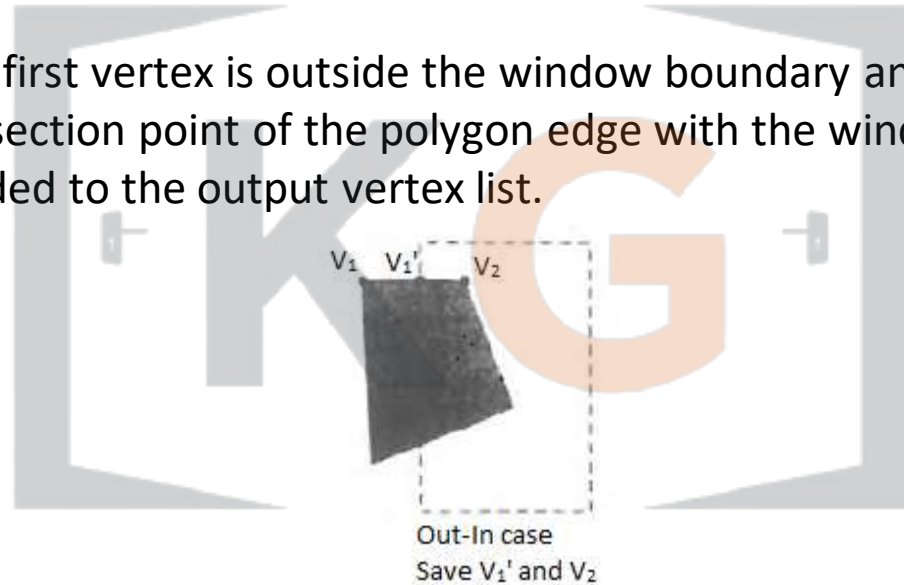
- We correctly clip a polygon by processing the polygon boundary as a whole against each window edge.
- Beginning with the initial set of polygon vertices, we could first clip the polygon against the left rectangle boundary to produce a new sequence of vertices.
- The new set of vertices could then be successively passed to a right boundary clipper, a bottom boundary clipper, and a top boundary clipper.



- At each step, a new sequence of output vertices is generated and passed to the next window boundary clipper.

Four Cases:

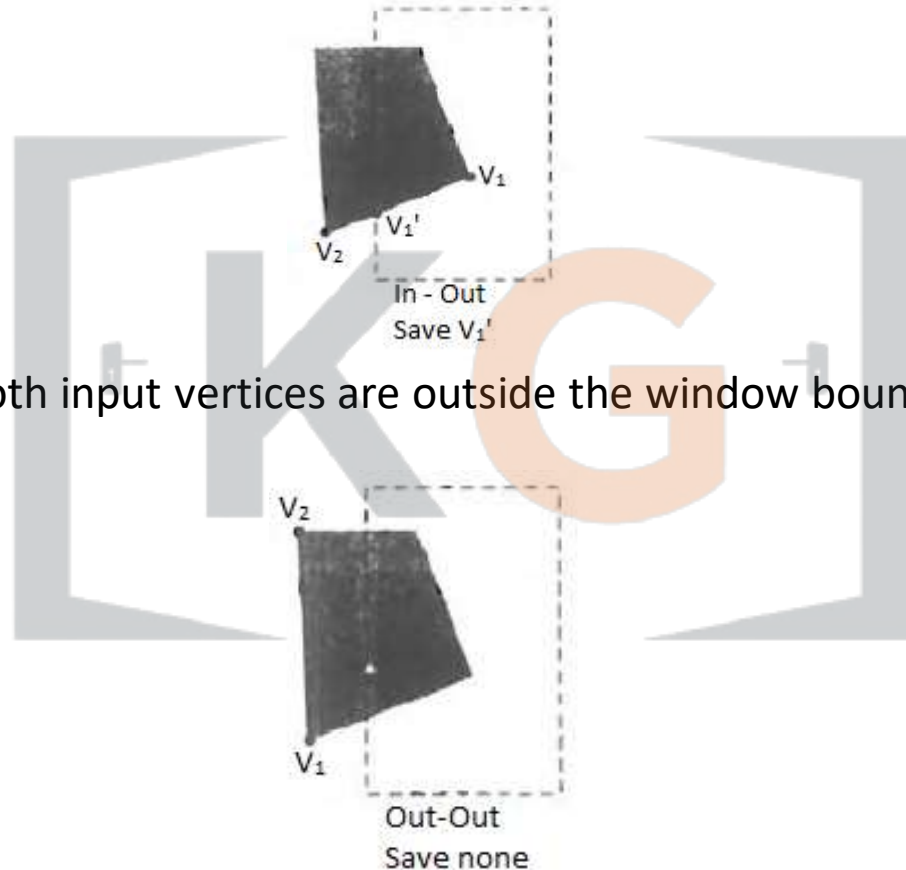
- There are four possible cases when processing vertices in sequence around the perimeter of a polygon. As each pair of adjacent polygon vertices is passed to a window boundary clipper, we make the following tests:
- Case 1 (out-in):** If the first vertex is outside the window boundary and the second vertex is inside, both the intersection point of the polygon edge with the window boundary and the second vertex are added to the output vertex list.



- **Case 2 (in-in):** If both input vertices are inside the window boundary, only the second vertex is added to the output vertex list.

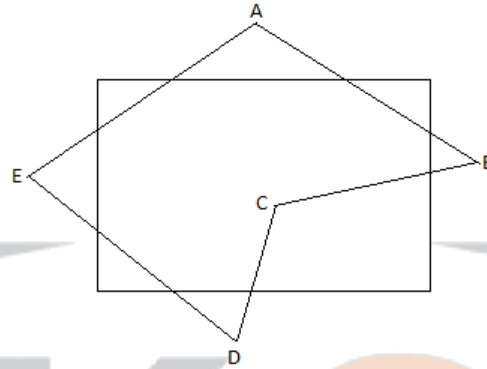


- **Case 3 (in-out):** If the first vertex is inside the window boundary and the second vertex is outside, only the edge intersection with the window boundary is added to the output vertex list.



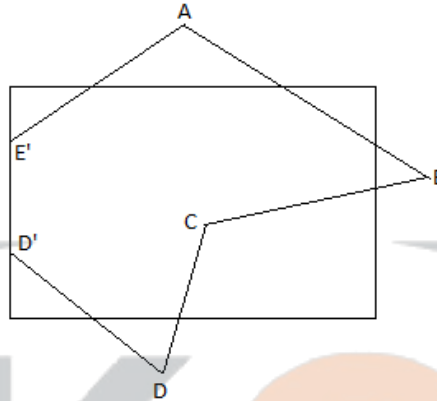
- **Case 4 (out-out):** If both input vertices are outside the window boundary, nothing is added to the output list.

Example: Consider the figure below:



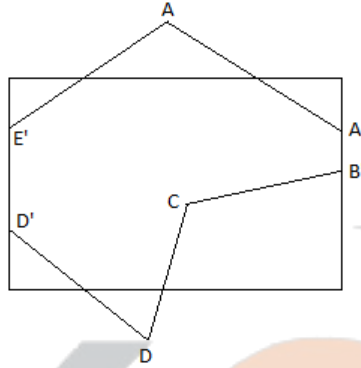
- We will clip the above figure with respect to all the window boundaries.
- **Clipping against the left window boundary:**
 - The edge AB lies in the in-in case w.r.t to left edge: Thus we add only B to the output list.
 - The edge BC lies in the in-in case w.r.t to left edge: Thus we add only C to the output list.
 - The edge CD lies in the in-in case w.r.t to left edge: Thus we add only D to the output list.
 - The edge DE lies in the in-out case w.r.t to left edge: Thus we add a new vertex D' (intersection of DE along the left boundary) to the output list.
 - The edge EA lies in the out-in case w.r.t to left edge: Thus we add new vertex E' and A to the output list

After clipping against the left boundary got:



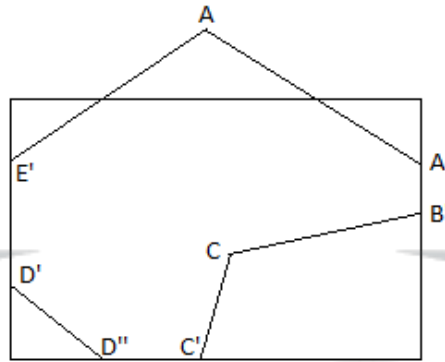
- **Clipping against the right window boundary:**
 - The edge AB lies in the in-out case w.r.t to right edge: Thus we add new vertex A' to the output list.
 - The edge BC lies in the out-in case w.r.t to right edge: Thus we add new vertex B' and C to the output list.
 - The edge CD lies in the in-in case w.r.t to right edge: Thus we add only D to the output list.
 - The edge DD' lies in the in-in case w.r.t to right edge: Thus we add D' to the output list.
 - The edge D'E' lies in the in-in case w.r.t to right edge: Thus we add E' to the output list.
 - The edge E'A lies in the in-in case w.r.t to right edge: Thus we add A to the output list.

Clipping against the right window boundary we get:



- **Clipping against the bottom window boundary:**
 - The edge AA' lies in the in-in case w.r.t to bottom edge: Thus we add A' to the output list.
 - The edge A'B' lies in the in-in case w.r.t to bottom edge: Thus we add B' to the output list.
 - The edge B'C lies in the in-in case w.r.t to bottom edge: Thus we add C to the output list.
 - The edge CD lies in the in-out case w.r.t to bottom edge: Thus we add new vertex C' to the output list.
 - The edge DD' lies in the out-in case w.r.t to bottom edge: Thus we add new vertex D'' and D' to the output list.
 - The edge D'E' lies in the in-in case w.r.t to bottom edge: Thus we add E' to the output list.
 - The edge E'A lies in the in-in case w.r.t to bottom edge: Thus we add A to the out put list.

Clipping against the bottom window boundary we get:

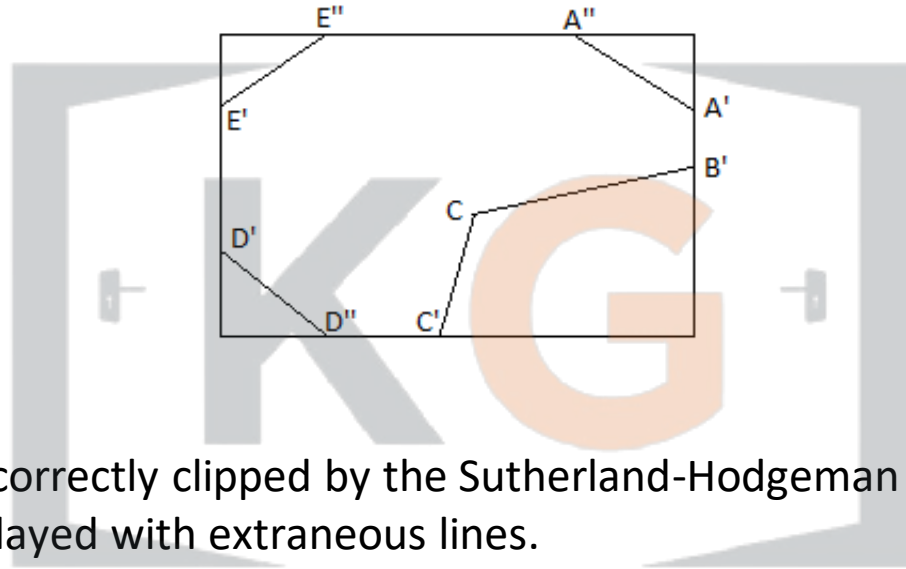


- **Clipping against the top window boundary:**

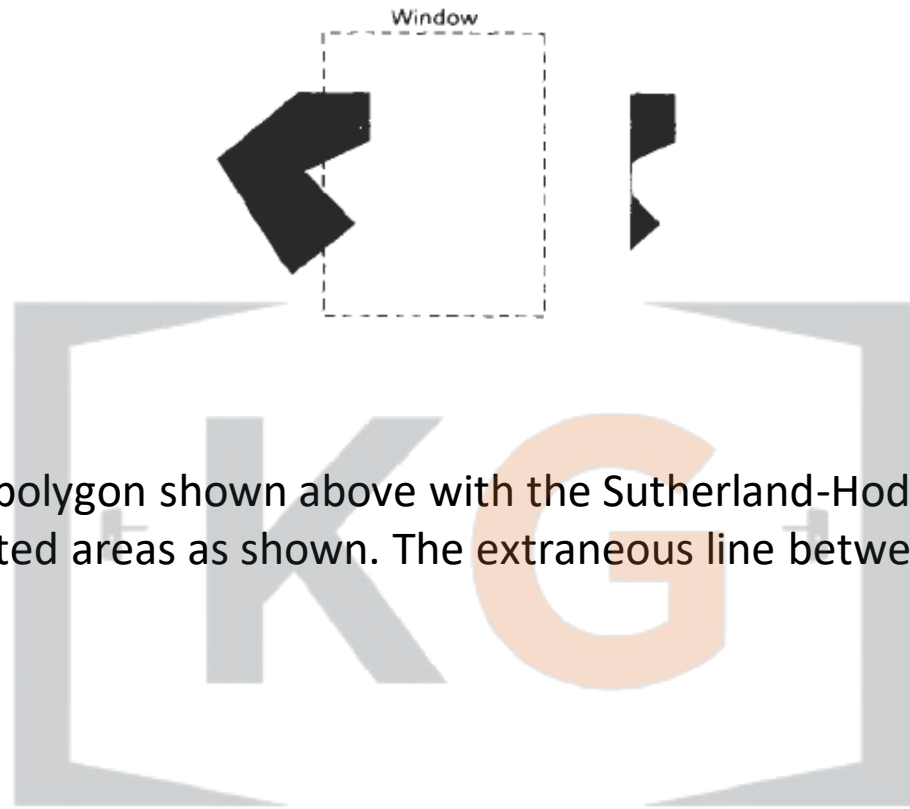
- The edge AA' lies in the out-in case w.r.t to top edge: Thus we add new vertex A'' and A' to the output list.
- The edge $A'B'$ lies in the in-in case w.r.t to top edge: Thus we add B' to the output list.
- The edge $B'C$ lies in the in-in case w.r.t to top edge: Thus we add C to the output list.
- The edge CC' lies in the in-in case w.r.t to top edge: Thus we add C' to the output list.
- The edge $C'D''$ lies in the in-in case w.r.t to top edge: Thus we add D'' to the output list.
- The edge $D''D'$ lies in the in-in case w.r.t to top edge: Thus we add D' to the output list.
- The edge $D'E'$ lies in the in-in case w.r.t to top edge: Thus we add E' to the output list.

- The edge $E'A$ lies in the in-out case w.r.t to top edge: Thus we add new vertex E'' to the output list.

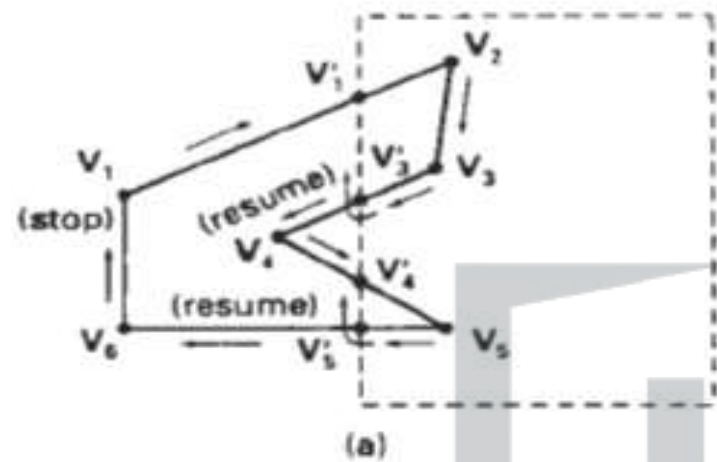
Clipping against the top window boundary we get the final clipped polygon as:



- Convex polygons are correctly clipped by the Sutherland-Hodgeman algorithm, but concave polygons may be displayed with extraneous lines.



- Clipping the concave polygon shown above with the Sutherland-Hodgeman Algorithm produces two connected areas as shown. The extraneous line between the areas should not have been there

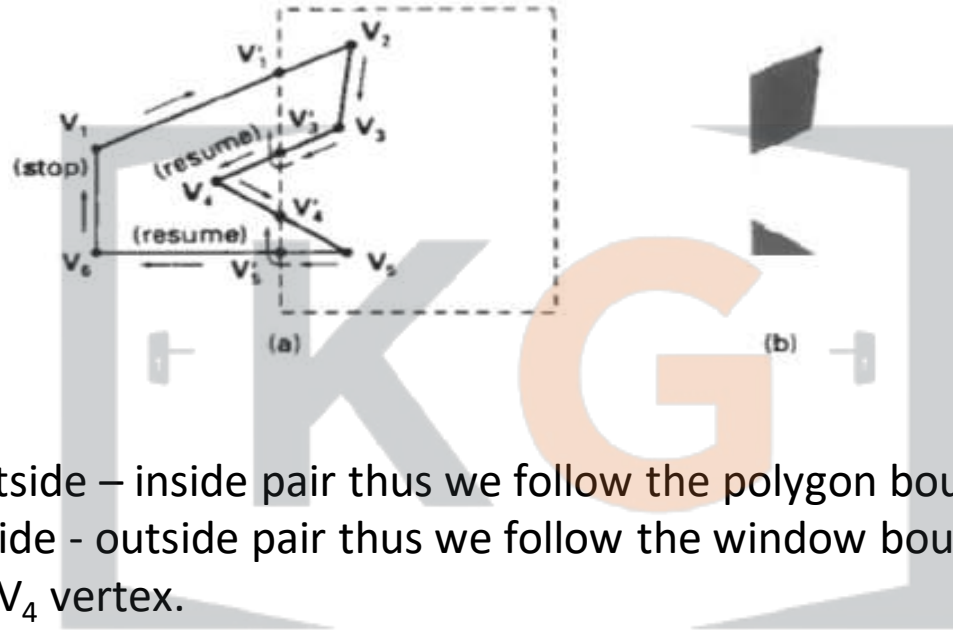


(b)

Weiler-Atherton Polygon Clipping

- In this clipping algorithm the vertex-processing procedures for window boundaries are modified so that concave polygons are displayed correctly.
- The basic idea in this algorithm is that instead of always proceeding around the polygon edges as vertices are processed, we sometimes follow the window boundaries.
- Which path we follow depends on the polygon-processing direction (clockwise or counterclockwise) and whether the pair of polygon vertices currently being processed represents an outside-to-inside pair or an inside-outside pair.
- For clockwise processing of polygon vertices, we use the following rules:
 - For an outside-to-inside pair of vertices, follow the polygon boundary.
 - For an inside-to-outside pair of vertices, follow the window boundary in a clockwise direction.

- The processing direction in the Weiler-Atherton algorithm and the resulting clipped polygon is shown for a rectangular clipping window.



- V_1 and V_2 are an outside – inside pair thus we follow the polygon boundary for clipping.
- V_3 and V_4 are an inside - outside pair thus we follow the window boundary for clipping and resume again from V_4 vertex.

**Q: The clipping process in computer graphics is used for
(UGC NET PAPER-2023)**

- (a) Adding Graphics**
- (b) Copying**
- (c) Zooming**
- (d) Removing objects and lines**

Ans: D



Visible Surface Detection

- A major consideration in the generation of realistic graphics displays is identifying those parts of a scene that are visible from a chosen viewing position.
- When we project 3-D objects on a 2-D screen, we need to detect the faces that are hidden on 2D.
- The various algorithms referred to as **visible-surface detection / Hidden Surface Elimination** methods are used for this purpose.

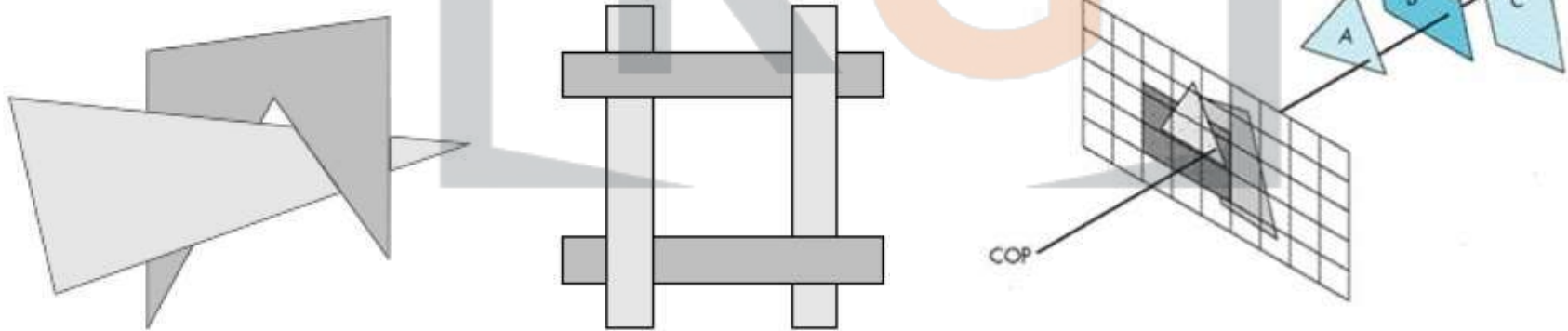


Fig.3.3. Z- buffer algorithm

CLASSIFICATION OF VISIBLE-SURFACE DETECTION ALGORITHMS

- Two approaches are called **object-space methods** and **image-space methods**.
- An **object-space method** compares objects and parts of objects to each other within the scene definition to determine which surfaces, as a whole, we should label as visible.
- An **image-space algorithm**, visibility is decided point by point at each pixel position on the projection plane.
- Most visible-surface algorithms use image-space methods, although object space methods can be used effectively to locate visible surfaces in some cases

Object-space method	Image-space method
Object space is the 3 dimensional space in which a graphic object is defined.	Image space is the projection of the object defined in 3D to two dimensional screen space.
a. Deals with object definition directly.	a. Deals with the projected images of the objects.
b. It compares the objects and part of objects to each other within the scene definition to determine which surfaces, as a whole to be labeled as visible, Geometrical relationship among objects	b. In this method, visibility is decided point by point at each pixel position on the projection plane.
c. Line-display algorithm	c. Line/surface algorithm
d. Used for accuracy and continuous operation, using float data processing	d. User for time saving and discrete operation.
Physical Coordinates	View port/Projection Coordinate
consider object only one time, but pixel can be process time to time	consider pixel only one time, but object can be process time to time
For e.g. Back-Face Detection	For e.g. Depth-Buffer Method (Z-Buffer)

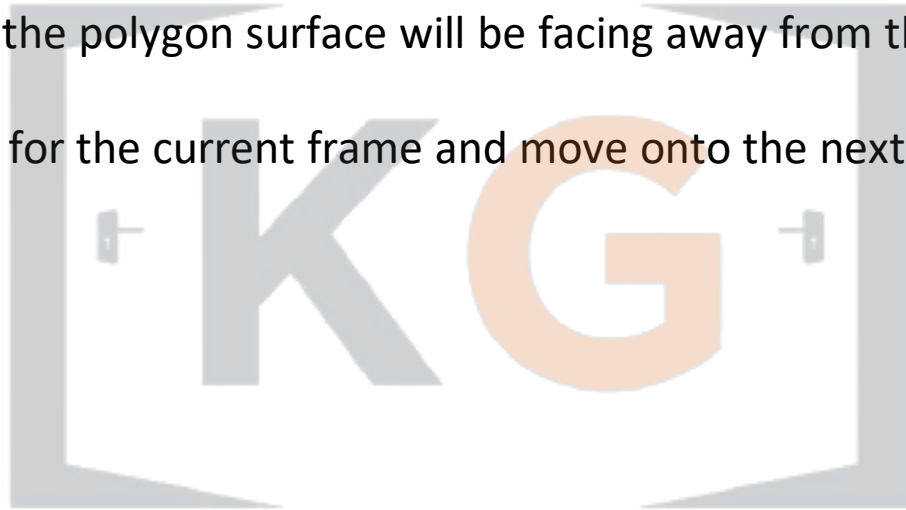
Q.The best suited hidden surface algorithm to deal with non-polygonal, non-planar surface patches is **(UP LT - 2018)**

- A)** Z-buffer algorithm
- B)** ray tracing algorithm
- C)** painter's algorithm
- D)** scan line algorithm



BACK-FACE DETECTION

- A fast and simple **object-space method** for identifying the back faces of a polyhedron is based on the "inside-outside" tests.
- The idea is to check if the polygon surface will be facing away from the viewer or not.
- If it does so, discard it for the current frame and **move onto** the next one.



- Each surface has a normal vector.
- If the normal vector is pointing in the direction of the center of projection, then it is a front face and can be seen by the viewer.
- If the normal vector is pointing away from the center of projection, then it is a back face and can not be seen by the viewer.

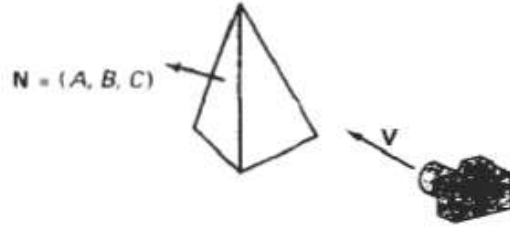


- A point (x, y, z) is "inside" a polygon surface with plane parameters A, B, C, and D if:

$$\mathbf{Ax + By + Cz + D < 0}$$

- When an inside point is along the line of sight to the surface, the polygon must be a back face (we are inside that face and cannot see the front of it from our viewing position).
- The back faces needs to be eliminated as they are not visible on the surfaces and they just take memory unnecessarily.
- Considering the normal vector N to a polygon surface, which has Cartesian components (A, B, C).
- In general, if V is a vector in the viewing direction from the eye (or "camera") position, then this polygon is a back face if:

$$\mathbf{V.N > 0}$$



- If object descriptions have been converted to projection coordinates and our viewing direction is parallel to the viewing z_v axis, then $V = (0, 0, V_z)$ and:

$$V \cdot N = V_z \cdot C$$

- Now we only need to consider the sign of C , the z component of the normal vector N .
- In a right-handed viewing system with viewing direction along the negative z_v axis.
- The polygon is a back face if $C < 0$.
- Also, we cannot see any face whose normal has z component $C = 0$, since our viewing direction is grazing that polygon.

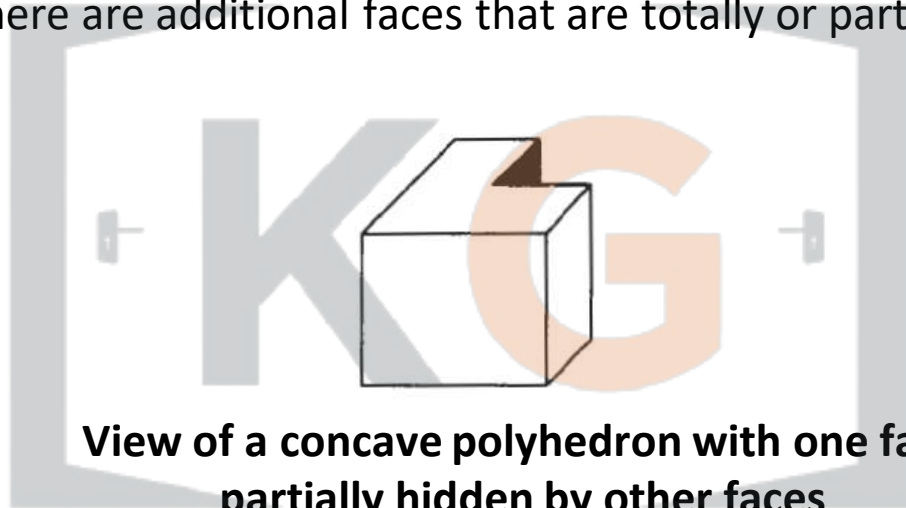
- In general, we can label any polygon as a back face if its normal vector has a z component value:

$$C \leq 0$$

- By examining parameter C for the different planes defining an object, we can immediately identify all the back faces.
- In general, back-face removal can be expected to eliminate about half of the polygon faces. surfaces in a scene from further visibility tests

Limitations

- If a scene contains only non-overlapping convex polyhedra, then only all hidden surfaces are identified with the back-face method.
- For other objects, such as the concave polyhedron, more tests need to be carried out to determine whether there are additional faces that are totally or partly obscured by other faces.



View of a concave polyhedron with one face partially hidden by other faces

Q. In 3D Graphics, which of the following statements is/are true? **(UGC NET PAPER-2018)**

P: Back-face culling is an example of an image precision visible-surface determination procedure.

Q. Z-Buffer is a 16-bit, or 64 bit field associated with each pixel in a frame buffer that can be used to determine the visible surfaces at each pixel.

Choose the correct answer from the code given below:

- (a)** Q only
- (b)** P and Q
- (c)** P only
- (d)** Neither P nor Q

Ans: A

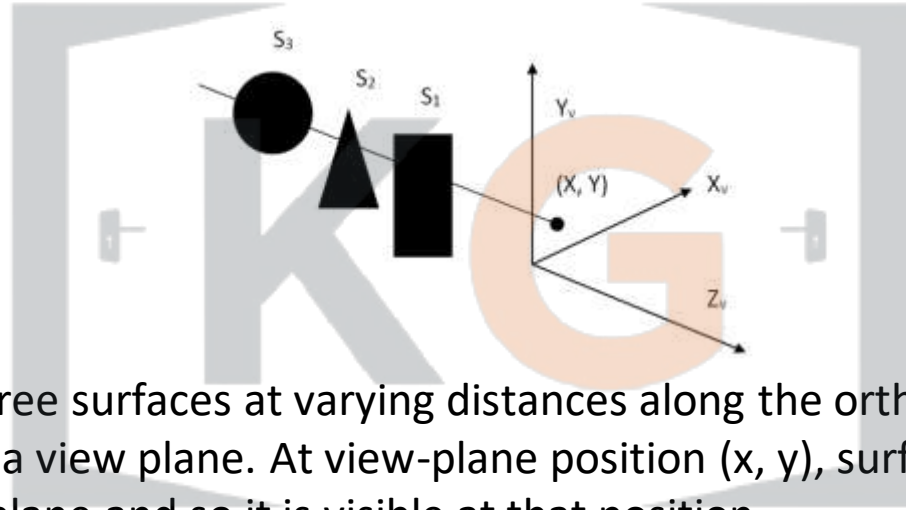
DEPTH-BUFFER (Z-Buffer) METHOD

- A commonly used **image-space approach** to detecting visible surfaces is the depth-buffer method, which compares surface depths at each pixel position on the projection plane.
- This procedure is also referred to as the **z-buffer method**, since object depth is usually measured from the view plane along the z axis of a viewing system.
- Each surface of a scene is processed separately, one point at a time across the surface.

- The method is usually applied to scenes containing only polygon surfaces, because depth values can be computed very quickly and the method is easy to implement.
- But the method can be applied to non-planar surfaces.



- With object descriptions converted to projection coordinates, each (x, y, z) position on a polygon surface corresponds to the orthographic projection point (x, y) on the view plane.
- Therefore, for each pixel position (x, y) on the view plane, object depths can be compared by comparing z values.



- In the above figure three surfaces at varying distances along the orthographic projection line from position (x, y) in a view plane. At view-plane position (x, y) , surface S_1 has the smallest depth from the view plane and so it is visible at that position.

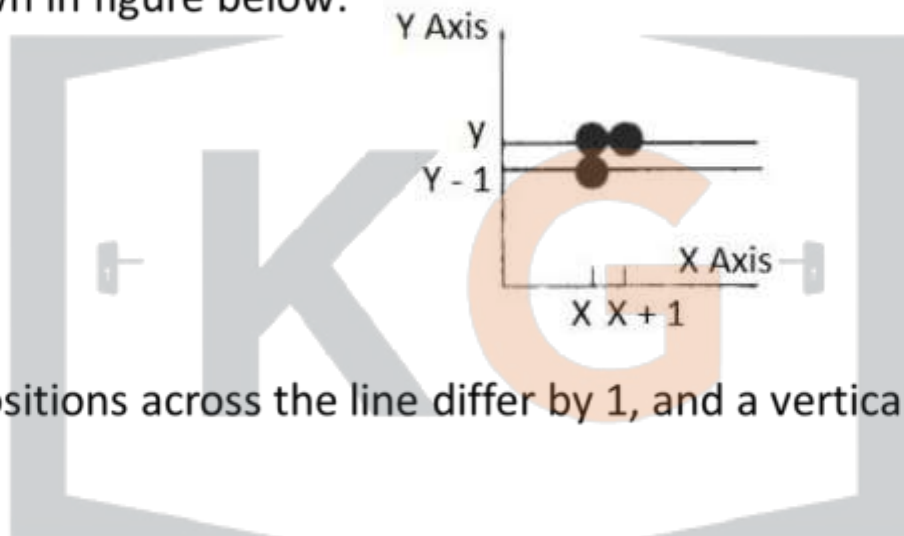
Working

- In Z-buffer algorithm a **depth buffer** is used to store depth values for each (x, y) position as surfaces are processed, and the **refresh buffer** stores the intensity values for each position.
- Initially, all positions in the depth buffer are set to 0 (minimum depth), and the refresh buffer is initialized to the background intensity.
- Each surface listed in the polygon tables is then processed, one scan line at a time, calculating the depth (z value) at each (x, y) pixel position.
- The calculated depth is compared to the value previously stored in the depth buffer at that position.
- If the calculated depth is greater than the value stored in the depth buffer, the new depth value is stored, and the surface intensity at that position is determined and placed in the same xy location in the refresh buffer.

- Depth values for a surface position (x, y) are calculated from the plane equation for each surface as:

$$z = \frac{-Ax - By - D}{C}$$

- For any scan line shown in figure below:



- Adjacent horizontal positions across the line differ by 1, and a vertical y value on an adjacent scan line differs by 1.
- If the depth of position (x, y) has been determined to be z, then the depth z' of the next position (x + 1, y) along the scan line is obtained as:

$$z' = \frac{-(Ax+1)-By-D}{C} \text{ or } z' = z - \frac{A}{C}$$

- The ratio $-A/C$ is constant for each surface, so succeeding depth values across a scan line are obtained from preceding values with a single addition.



Example The Z-buffer algorithm is used for Hidden surface removal of objects. The maximum number of objects that can be handled by this algorithm shall

- a) Depend on the application
- b) Be arbitrary no. of objects**
- c) Depend on the memory availability
- d) Depend on the processor

Ans. B

Z-Buffer algorithm can handle arbitrary number of objects as each object is processed one at a time. The object are only limited by the memory of computer to store objects.

Painter's Algorithm (Depth Sorting)

- The painter's algorithm, also known as a priority fill, is one of the simplest solutions to the visibility problem in 3D computer graphics.
- The name "painter's algorithm" refers to the technique employed by many painters for painting distant parts of a scene before parts which are nearer thereby covering some areas of distant parts.
- The painter's algorithm sorts all the polygons in a scene by their depth and then paints them in this order, **farthest to closest**.

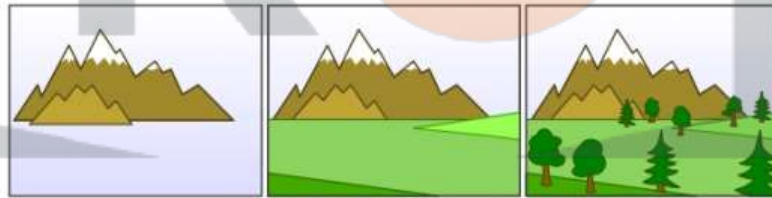


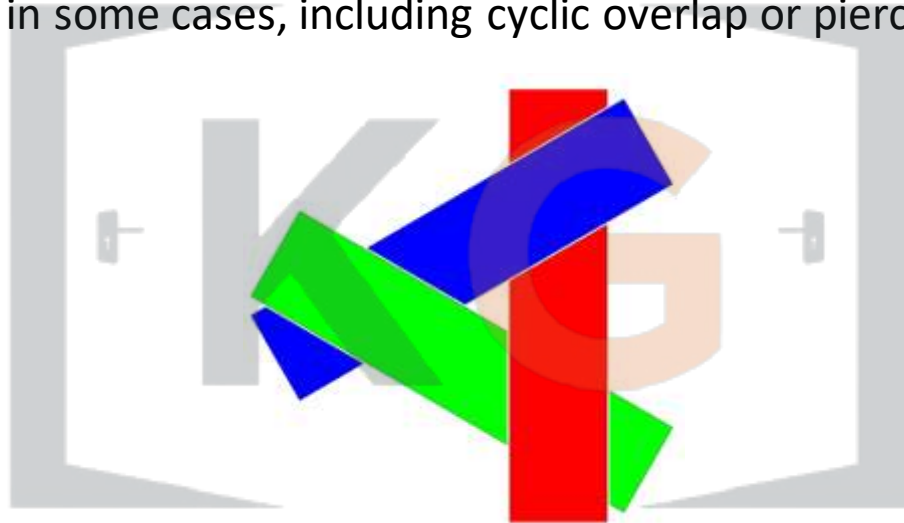
Fig.3.4. Painter's algorithm

- It will paint over the parts that are normally not visible — thus solving the visibility problem — at the cost of having painted invisible areas of distant objects.

- The ordering used by the algorithm is called a '*depth order*', the essential property of this ordering is, rather, that if one object obscures part of another then the first object is painted after the object that it obscures.

Limitation

- The algorithm can fail in some cases, including cyclic overlap or piercing polygons.



Q. Hidden surface removal problem with minimal 3 D pipeline can be solved with (UGC NET PAPER-2022)

- (a)** Painter's algorithm
- (b)** Window clipping algorithm
- (c)** Brute force rasterization algorithm
- (d)** Flood fill algorithm

Ans: C



Q. Which of the following statements is/ are **True** regarding the solution to the visibility problem in 3D graphics? (**UGC NET PAPER-2018**)

S_1 : The Painter's algorithm sorts polygons by depth and then paints (scan -converts)

S_2 : Backface Culling refers to eliminating geometry with back facing normals.

Code:

- (a) S_1 only
- (b) S_2 only
- (c) Both S_1 and S_2 only
- (d) Neither of S_1 nor S_2

Ans: B

Spline Representation

- **Planar surface**

- A planar surface is a flat 2D surface.

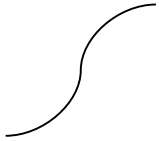
- **Curved surface**

- **Single curved surface:** It is a simple curved surface.
- **Double curved surface:** It is a complex surface generated not by a straight line but a curved surface.

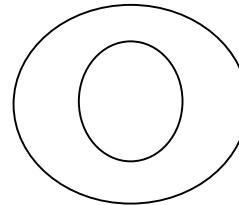
- **Ruled surface:** Ruled surface is a surface constructed by transitioning between two or more curves by using linear blending between each section of the surface.

Curves and its representation

- When a set of points finite or infinite are joined in a **continuous** fashion then what we get is a curve.
- There should be no discontinuity or gaps in between for a figure to be a curve.



- **Is a straight line a Curve?**
 - Technically Yes, a straight line is curve with no curvature.
- The curve is basically a generalization of a Line where there may or may not be any curvature the only condition is it needs to be continuous.



This is not a curve as there is discontinuity

. Polygons Surfaces

- Polygons are used in computer graphics to compose images that are three-dimensional in appearance. Usually triangular, polygons arise when an object's surface is modeled, vertices are selected, and the object is rendered in a wire frame model.
- A polygonal surface can be thought of as a surface composed of polygonal faces.

Polygons Surfaces

- The information defining the polygonal surface can be stored in two simple files :
 - 1 The **node file lists** a set of (x,y,z) values, which are vertices of the polygons.
 - 1 The **element file lists**, for each polygonal face, the indices of the nodes that form its vertices.

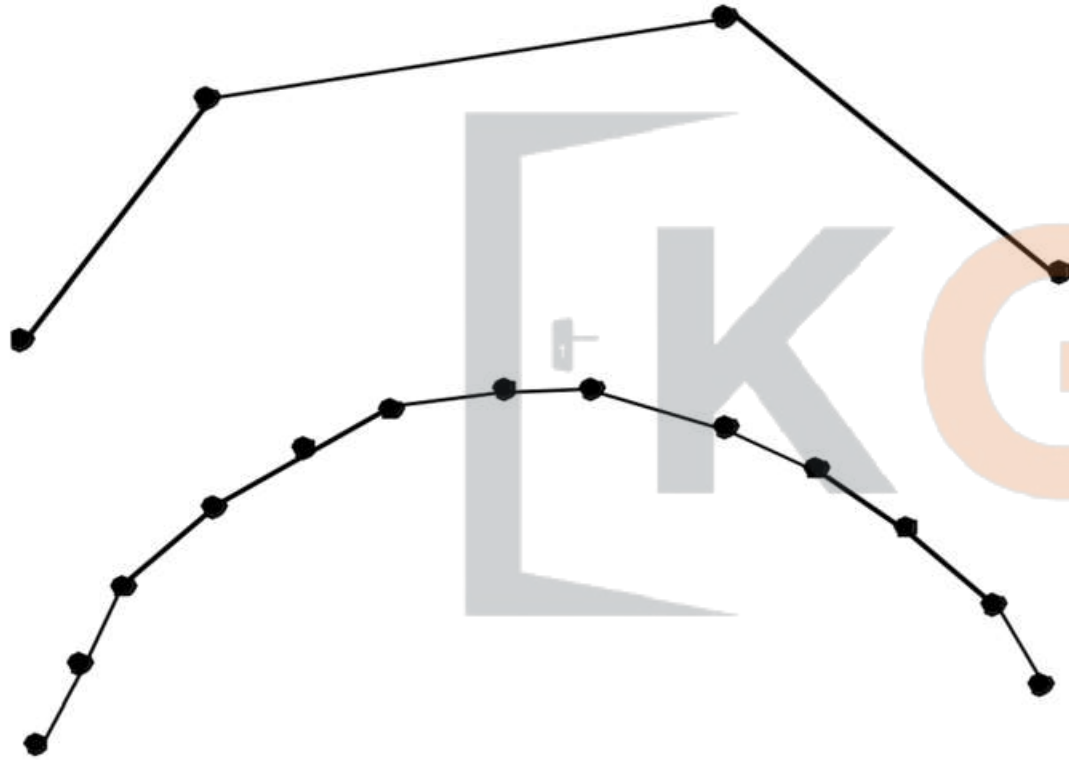
Quadric Surfaces

- These surfaces are described with second-degree equations (quadratics).
- It includes spheres ellipsoid, tori, parabolic and hyperbolised spheres and ellipsoid, we know that these are basic elements of any graphics scene with the help of which we can generate complex objects in the scene.
- Eg: A spherical surface with radius (r) cantered on the coordinate origin is defined as the set of points (x, y, z) represented by the equation.

$$x^2 + y^2 + z^2 = r^2$$

-

“Computers can’t draw curves.”

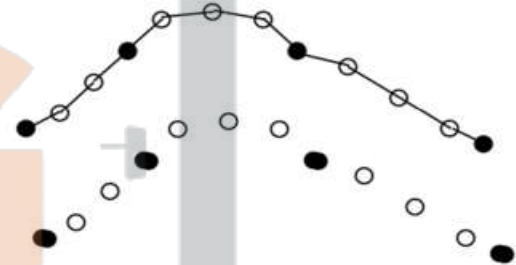


The more points/line segments that are used, the smoother the curve.

- Problem: How to represent a curve easily and efficiently
- “Brute force and ignorance” approaches:
 - storing a curve as many small straight line segments
 - doesn't work well when scaled
 - inconvenient to have to specify so many points
 - need lots of points to make the curve look smooth
 - working out the equation that represents the curve
 - difficult for complex curves
 - moving an individual point requires re-calculation of the entire curve

Solution - Interpolation

- Define a small number of points
- Use a technique called “interpolation” to invent the extra points for us.
- Join the points with a series of (short) straight lines



The need for smoothness

- So far, mostly polygons can approximate any geometry, but Only approximate
- Need lots of polygons to hide discontinuities Storage problems
- Math problems
- Not very convenient as modeling tool
- Gets even worse in animation

Requirements

- Want mathematical smoothness
- Some number of continuous derivatives of P
- Local control
- Local data changes have local effect
- Continuous with respect to the data
- No wiggling if data changes slightly
- Low computational effort

A solution

- Use SEVERAL polynomials
- Complete curve consists of several pieces
- All pieces are of low order
 - Third order is the most common
- Pieces join smoothly
- This is the idea of spline curves
 - or just “splines”

Interpolation methods	Best fit methods
It is necessary that the curve produced will have to go through all the data points	The curve will not pass through all the points, but will result in a curve that will be closest to as many points as possible.
Cubic Splines, and Lagrange interpolation methods are used.	Regression and least square methods are used for the purpose. Bezier curves also fall in this category.
Shape of the curve is affected to a great extent by manipulating a single data point. The nature of tweaking is unpredictable.	It is possible to have a local modification easily by tweaking a single point where the behaviour is more predictable.

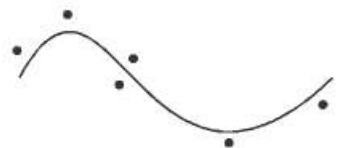
Spline

- Drafting terminology
 - Spline is a flexible strip that is easily flexed to pass through a series of design points (control points) to produce a smooth curve.
- Spline curve – a piecewise polynomial (cubic) curve whose first and second derivatives are continuous across the various curve sections.



Spline

- We specify **spline curve** by set of coordinate position called **control points**.
- These control points are fitted in two ways.
 - (A) When the polynomial sections are fitted so that the curve passes through each control point. and the curve is said to interpolate the set of control points and method is **called curve fitting method**.
 - (B) If polynomials are fitted to control point path without necessarily passing through each control point, **then the curve is said to approximate the set of control points and technique is called fairing**.



Approximation or curve fairing



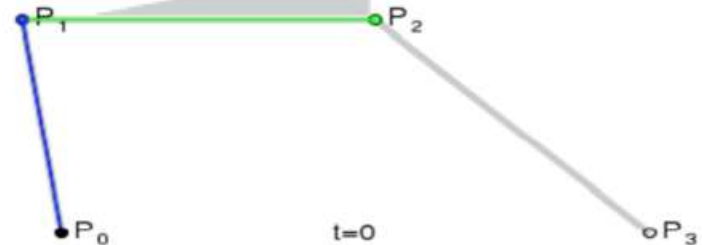
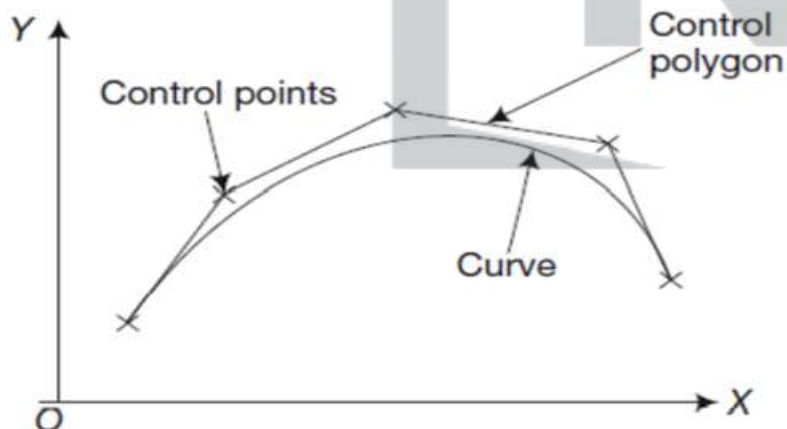
Interpolation or curve fitting

Synthetic curves

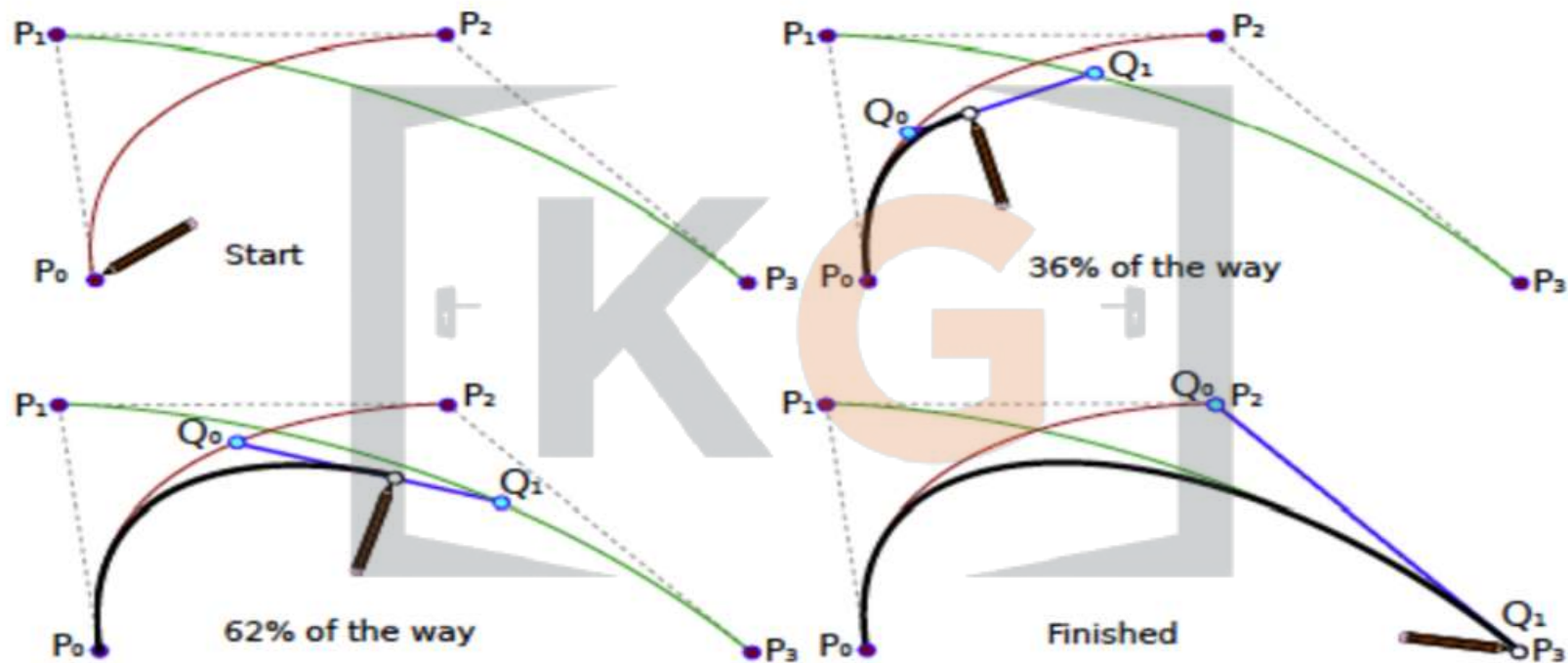
- Easy to enter the data and easy to control the continuity of the curves to be designed.
- Requires much less computer storage for the data representing the curve.
- Having no computational problems and faster in computing time.
 - Bezier curves
 - Hermite cubic spline
 - B-spline curves
 - Rational B-splines (including Non-uniform rational B-splines – NURBS)

Bezier curve

- A Bezier curve is a **mathematically defined curve** used in two-dimensional graphic applications.
- The curve is defined by points: the **initial position** and the **terminating position** (which are called "anchors") and **middle points** (which are called "handles or control points").
- The **shape** of a Bezier curve can be **altered** by moving the **control points**.



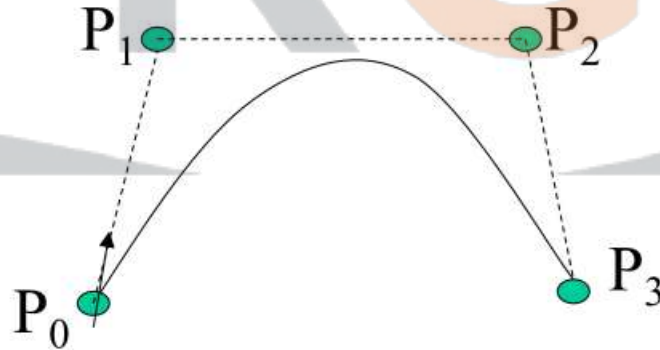
Bezier curves and the Associated Control Polygon

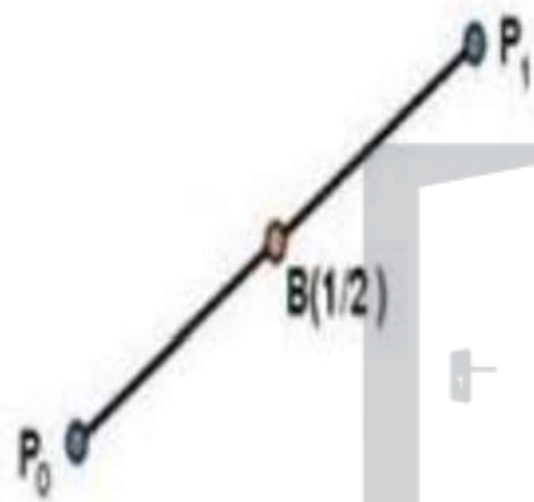


Bezier curve

The polynomial degree is directly related to the number of control points.

- The control of the continuity between Bezier curves is not trivial.
- Bezier curve is determined by defining a polygon.
- Bezier curve section can be fitted to any no. of control points

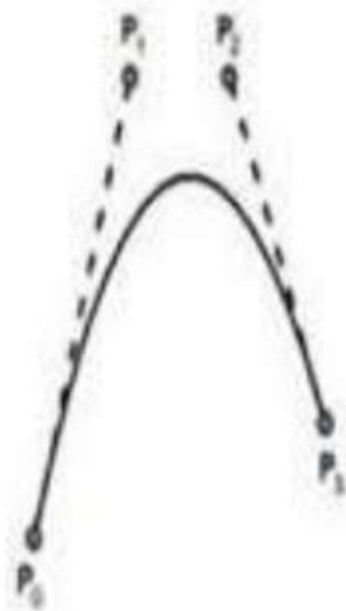




Simple Bezier Curve



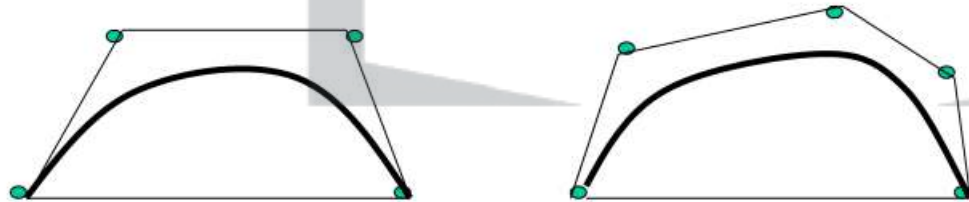
Quadratic Bezier Curve



Cubic Bezier Curve

Properties (continued)

- Convex hull
 - Convex polygon formed by connecting the control points of the curve.
 - Curve resides completely inside its convex hull

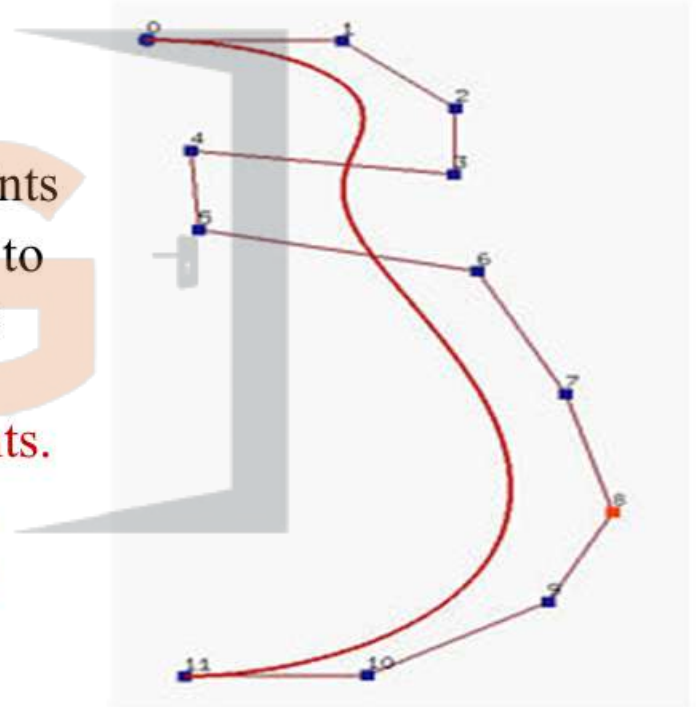


Properties

- The Bézier curve passes through the first and last control points while it maintains proximity to the intermediate control points.
- As such the entire Bézier curve lies in the interior of the convex hull of the control points.
- If a control point is moved the entire curve moves.
- Being polynomial functions, Bézier curves are easily computed, and infinitely differentiable.
- If the control points of the Bézier curve are transformed, the curve moves to the corresponding new coordinate frame without changing its shape.

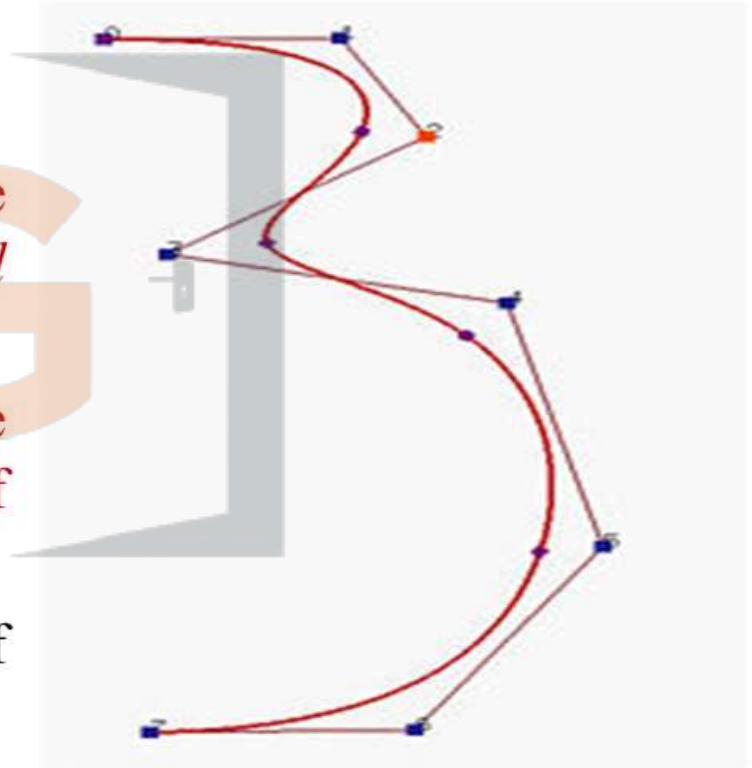
B-Spline

- Motivation (recall bezier curve)
 - The degree of a Bezier Curve is determined by the number of control points
 - E. g. (bezier curve degree 11) – difficult to bend the "neck" toward the line segment P_4P_5 .
 - Of course, we can add more control points.
 - BUT this will increase the degree of the curve $\rightarrow$ increase computational burden



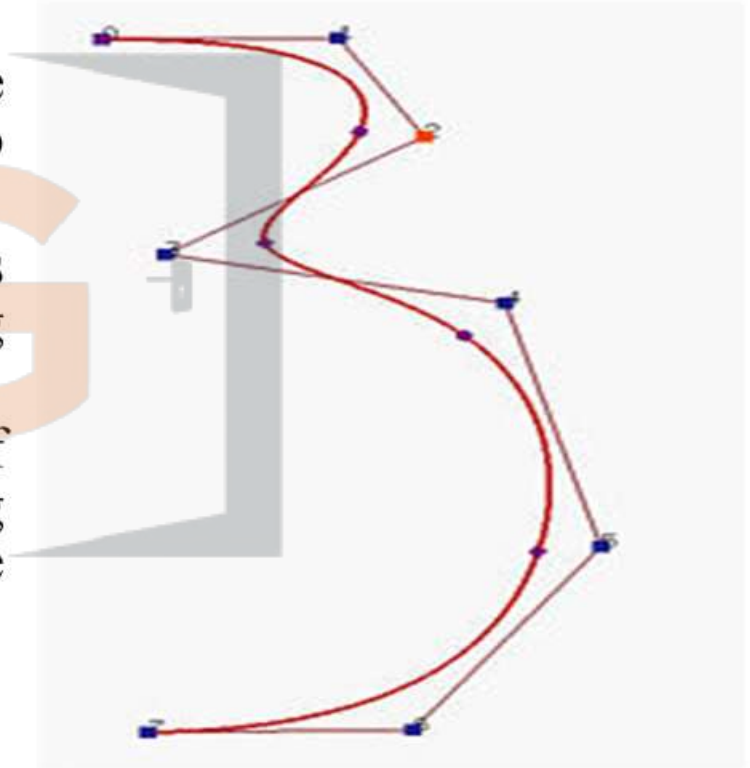
B-Spline

- Motivation (recall bezier curve)
 - moving a control point affects the shape of the entire curve- (*global modification property*) – undesirable.
 - Thus, the solution is B-Spline – the degree of the curve is independent of the number of control points
 - E.g - right figure – a B-spline curve of degree 3 defined by 8 control points



B-Spline

- This method is better than bezier spline curves method because of two characteristics.
- (i) The number of polygons vertices fixes the order of the resulting polynomial which defines the curve.
- (ii) Bezier curve is the result of blending the values of all defining vertices, change in one vertex is felt in the entire curve.

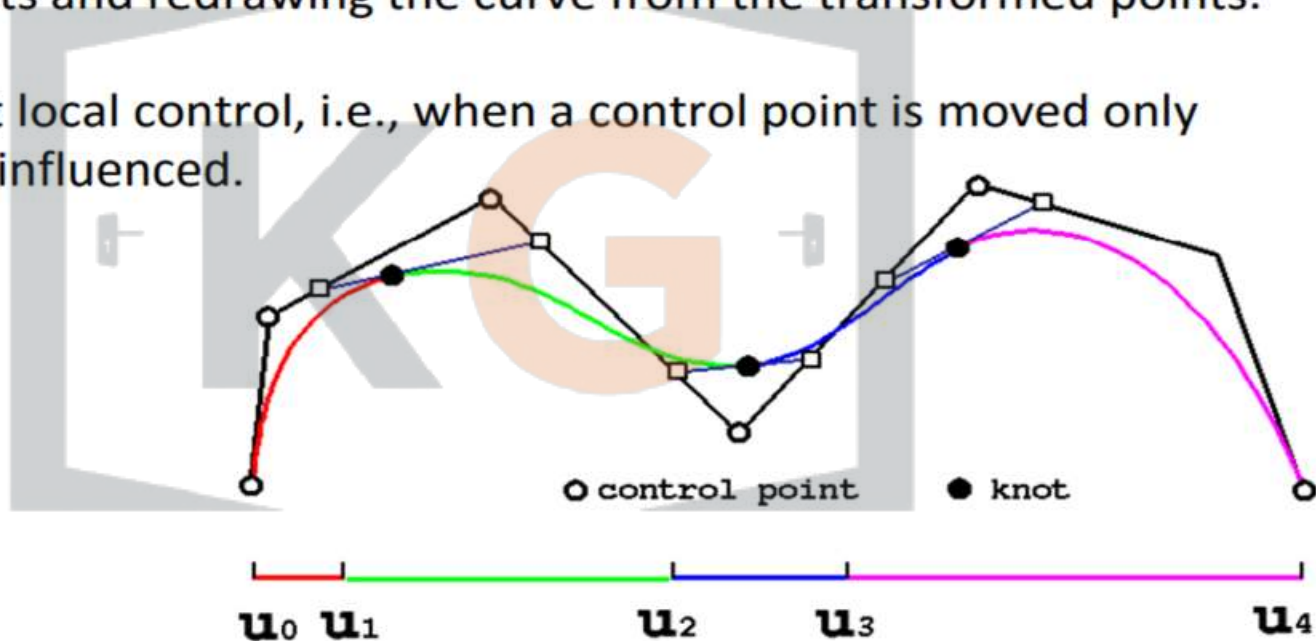


- B-spline curves have the flexibility of choosing the degree of the curve irrespective of the number of control points.
- With four control points, it is possible to get a cubic Bézier curve, while with B-spline curve one can get a linear, quadratic or cubic curve.
- B-spline also uses the basis (blending) functions and the equation is of the form

$$0 \leq u \leq u_{\max}$$

$$p(u) = \sum_{i=0}^n p_i N_{i,k}(u)$$

- The curve follows the shape of the control points and lie in the convex hull of the control points.
- The entire B-spline curve can be affinely transformed by transforming the control points and redrawing the curve from the transformed points.
- B-splines exhibit local control, i.e., when a control point is moved only that segment is influenced.



B-Spline

- Subdividing the curve directly is difficult to do → so, **subdivide the domain of the curve by points called *knots***



Type of B-Spline uniform knot vector

Non-periodic knots (open knots)

- First and last knots are duplicated k times.
- E.g (0,0,0,1,2,2,2)
- Curve pass through the first and last control points

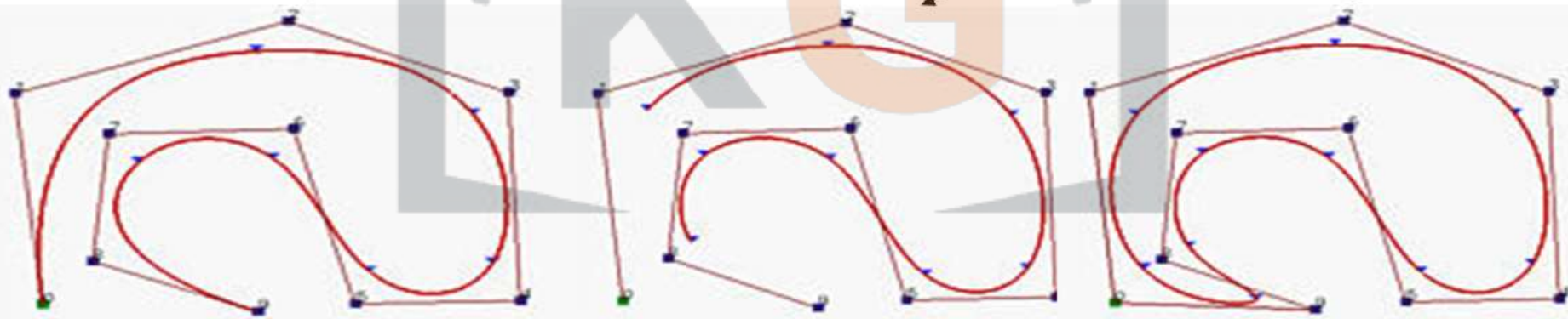
Periodic knots (non-open knots)

- First and last knots are not duplicated – same contribution.
- E.g (0, 1, 2, 3)
- Curve doesn't pass through end points.
- used to generate closed curves (when first = last control points)

Type of B-Spline knot vector

Non-periodic knots
(open knots)

Periodic knots
(non-open knots)



(Closed knots)

- The B-spline curve is contained in the convex hull of its control points.
- Affine transformations of the coordinate system do not change the shape of the B-spline curve.
- Increasing the degree of the curve makes it more difficult to control, and hence a cubic B-spline is sufficient for a majority of applications.

UGCNET-Oct2020-II: 82

Given below are two statements:

Statement *I*: Bezier curves are curves that interpolate all of their control points

Statement *II*: A cubic bezier curve has four control points

In the light of the above statements, choose the correct answer from the options given below

- A. Both Statement *I* and Statement *II* are true
- B. Both Statement *I* and Statement *II* are false
- C. Statement *I* is correct but Statement *II* is false
- D. Statement *I* is incorrect but Statement *II* is true

UGCNET-Oct2020-II: 82

Given below are two statements:

Statement *I*: Bezier curves are curves that interpolate all of their control points

Statement *II*: A cubic bezier curve has four control points

In the light of the above statements, choose the correct answer from the options given below

- A. Both Statement *I* and Statement *II* are true
- B. Both Statement *I* and Statement *II* are false
- C. Statement *I* is correct but Statement *II* is false
- D. Statement *I* is incorrect but Statement *II* is true

Which of the following statements is/are correct with reference to curve generation?

- I. Hermite curves are generated using the concepts of interpolation
 - II. Bezier curves are generated using the concepts of approximation
 - III. The Bezier curve lies entirely within the convex hull of its control points
 - IV. The degree of Bezier curve does not depend on the number of control points
- A. I, II and IV only
 - B. II and III only
 - C. I and II only
 - D. I, II and III only

UGC NET CSE | June 2016 | Part 3 | Question: 18

Which of the following statements is/are correct with reference to curve generation?

- I. Hermite curves are generated using the concepts of interpolation
- II. Bezier curves are generated using the concepts of approximation
- III. The Bezier curve lies entirely within the convex hull of its control points
- IV. The degree of Bezier curve does not depend on the number of control points

A. I, II and IV only

B. II and III only

C. I and II only

 D. I, II and III only

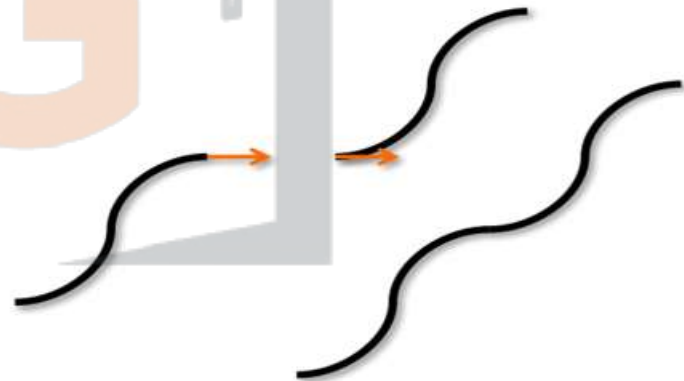
A Hermite curve is a **spline where every piece is a third degree polynomial**

Hermite Curves

We want curves that fit **together** smoothly.

To accomplish this, we would like to specify a curve by providing:

- The endpoints
- The 1st derivatives at the endpoints

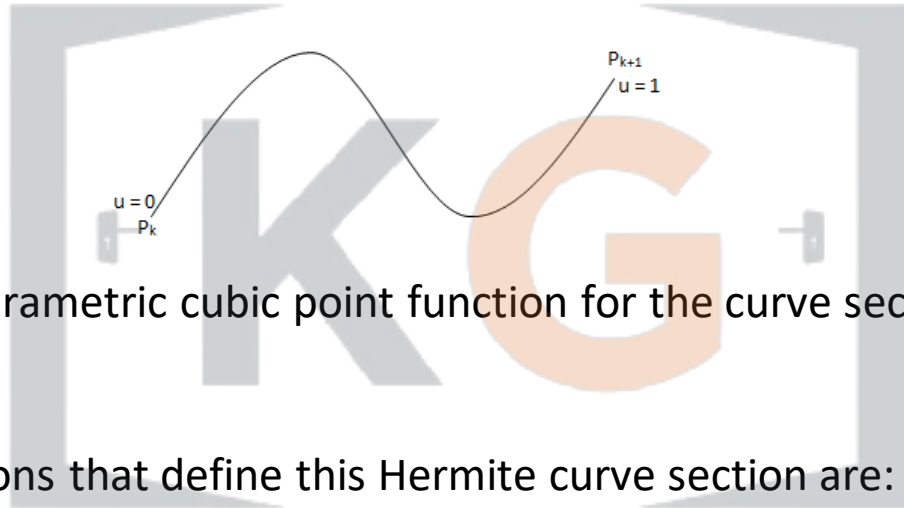


The result is called a *Hermite Curve*.

Hermite Interpolation

- A Hermite spline (named after the French mathematician Charles Hermite) is an **interpolating** piece wise cubic polynomial with a specified tangent at each control point.

- Consider the curve:



- If $P(u)$ represents a parametric cubic point function for the curve section between control points p_k and p_{k+1} .
- The boundary conditions that define this Hermite curve section are:
 - $P(0) = p_k$
 - $P(1) = p_{k+1}$
 - $P'(0) = Dp_k$
 - $P'(1) = Dp_{k+1}$